

INFORMATIQUE • 1ÈRE TI

MODULE 1 : ALGORITHMIQUE

LEÇON N03

CONSTRUCTION D'ALGORIGRAMMES ET EXÉCUTION DES ALGORITHMES À LA MAIN

DURÉE CONSEILLÉE : 6 H

DANS CETTE LEÇON

Cette leçon introduit l'algorithme comme représentation graphique d'un algorithme et l'exécution manuelle (trace) comme outil de vérification. Elle couvre les symboles normalisés, la correspondance avec les structures de contrôle, le déroulement pas à pas, le comptage d'instructions et l'exécution de fonctions récursives simples.

- À la fin de cette leçon, je sais définir un algorithme et le distinguer d'un organigramme
- À la fin de cette leçon, je sais identifier les symboles de base d'un algorithme (terminaux, traitement, entrée/sortie, décision, flèches, connecteurs)
- À la fin de cette leçon, je sais associer à chaque structure de contrôle (séquence, alternative, répétitive) la portion d'algorithme correspondante
- À la fin de cette leçon, je sais dérouler un algorithme en construisant une table de trace
- À la fin de cette leçon, je sais transformer un algorithme simple en algorithme en pseudo-code
- À la fin de cette leçon, je sais compter le nombre d'instructions exécutées par un algorithme



TÉLÉCHARGE ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir

**Signé • OnBuch+**

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. Algorithme et organigramme : définitions et symboles
2. Structures de contrôle en algorithmes
3. Déroulement d'un algorithme et table de trace
4. De l'algorithme à l'algorithmique ; comptage d'instructions
5. Exécution pas à pas : structures de données et sous-algorithmes
6. Fonctions récursives simples
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais définir un algorithme et le distinguer d'un organigramme
- À la fin de cette leçon, je sais identifier les symboles de base d'un algorithme (terminaux, traitement, entrée/sortie, décision, flèches, connecteurs)
- À la fin de cette leçon, je sais associer à chaque structure de contrôle (séquence, alternative, répétitive) la portion d'algorithme correspondante
- À la fin de cette leçon, je sais dérouler un algorithme en construisant une table de trace
- À la fin de cette leçon, je sais transformer un algorithme simple en algorithmique en pseudo-code
- À la fin de cette leçon, je sais compter le nombre d'instructions exécutées par un algorithme
- À la fin de cette leçon, je sais exécuter pas à pas un algorithme intégrant une structure de contrôle, un tableau ou un sous-algorithme
- À la fin de cette leçon, je sais définir une fonction récursive et exécuter à la main des fonctions récursives simples (factorielle, somme)

PRÉREQUIS

- Notion d'algorithme et de pseudo-code (Leçons 1 et 2 du Module 1)

- Variables, affectation et types de données simples
- Structures de contrôle : alternative (Si...Alors...Sinon) et répétitives (TantQue, Pour)
- Notion de sous-algorithme (procédure et fonction)
- Tableaux à une dimension

1 ALGORIGRAMME ET ORGANIGRAMME : DÉFINITIONS ET SYMBOLES

Situation d'accroche. À la gare routière de Mvan à Yaoundé, un contrôleur applique toujours la même procédure : vérifier le ticket, contrôler les bagages, autoriser l'embarquement, et recommencer pour chaque passager. Un jour, on lui demande d'expliquer sa méthode à un nouvel employé à l'aide d'un simple dessin fait de losanges et de rectangles reliés par des flèches. Ce dessin, c'est un **algorithme** : il permet de visualiser et de vérifier un raisonnement avant même de l'écrire en langage algorithmique. Mais comment choisir les bons symboles ? Comment suivre pas à pas ce que fait un algorithme pour détecter ses erreurs ? Et comment traiter le cas étonnant d'une fonction qui s'appelle elle-même ? C'est à toutes ces questions que cette leçon va répondre, en commençant par la base : les définitions et les symboles.

1.1 Pourquoi dessiner un algorithme ?

Dans les leçons précédentes, tu as appris à écrire des algorithmes en pseudo-code : une suite de lignes de texte (Lire, Affecter, Si... Alors...). Ce texte est précis, mais il a un défaut : quand l'algorithme devient long, avec des tests imbriqués et des boucles, il devient difficile de voir d'un coup d'œil le chemin que suivent les données.

L'être humain comprend plus vite un schéma qu'un texte. C'est le même principe que le plan d'une ville : on peut décrire l'itinéraire de la gare de Mvan au marché Mokolo par une longue phrase, mais une carte avec des flèches est incomparablement plus claire. L'algorithme est la « carte » d'un algorithme.

Dessiner un algorithme sert à trois choses :

- **Concevoir** : avant d'écrire le moindre pseudo-code, on dessine la logique du raisonnement ; les oublis (un cas non traité, une boucle sans sortie) sautent aux yeux.
- **Communiquer** : deux personnes qui ne parlent pas le même langage de programmation peuvent se comprendre grâce à un dessin normalisé, comme le contrôleur de Mvan explique sa procédure au nouvel employé.
- **Vérifier** : en suivant les flèches avec un crayon, on peut « exécuter à la main » l'algorithme et détecter ses erreurs avant de le programmer.

1.2 Définitions : algorithme et organigramme

DÉFINITION Algorithme

Un **algorithme** (ou organigramme de programmation) est la représentation graphique normalisée d'un algorithme, à l'aide de symboles de formes précises (ovale, rectangle, parallélogramme, losange) reliés par des flèches orientées qui indiquent l'ordre d'exécution des instructions.

DÉFINITION Organigramme

Un **organigramme** est un schéma qui représente, au sens large, l'organisation, les étapes ou les flux d'un processus quelconque : procédure administrative, circuit de validation d'un document dans un ministère, organisation hiérarchique d'une entreprise, déroulement d'une fabrication.

La distinction est subtile mais importante pour le Probatoire :

Critère	Algorithme	Organigramme (sens large)
Objet représenté	Un algorithme (suite d'instructions exécutables par une machine)	Un processus quelconque (administratif, industriel, organisationnel)
Symboles	Normalisés et stricts (norme ISO 5807)	Libres ou propres à un métier
Notion de test	Losange à exactement deux sorties (Oui/Non)	Pas nécessairement de logique de décision
But	Concevoir, vérifier, traduire en programme	Décrire, documenter, communiquer

À RETENIR Algorithme ou organigramme ?

Tout algorithme est un organigramme, mais tout organigramme n'est pas un algorithme. L'algorithme est le cas particulier où le processus représenté est un algorithme, avec des symboles strictement normalisés. En classe de Première TI, quand on parle d'algorithme, on utilise le mot **algorithme**.

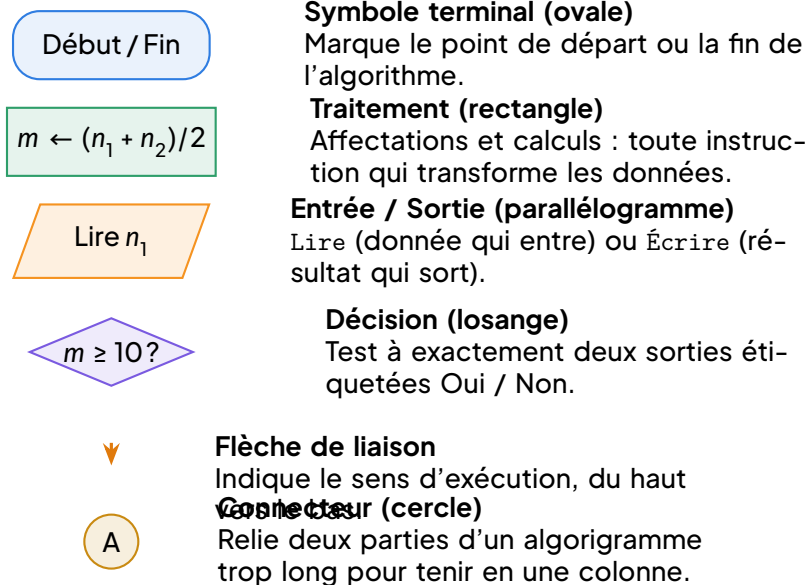
LESAVAIS-TU ? Une norme internationale : ISO 5807

Les symboles des algorithmes ne sont pas une invention de ton professeur : ils sont normalisés par la norme internationale ISO 5807 (publiée en 1985), qui reprend l'ancienne norme américaine ANSI X3.5 des années 1960. Grâce à cette normalisation, un algorithme dessiné à Yaoundé est lisible tel quel par un ingénieur logiciel à Tokyo, à Berlin ou à Montréal. Dans l'industrie du logiciel, ces schémas (appelés flowcharts en anglais) servent encore aujourd'hui à documenter les programmes critiques, par exemple dans l'aéronautique ou la banque.

1.3 Les symboles normalisés de l'algorithme

Un algorithme est construit avec un petit nombre de symboles, chacun ayant une **forme imposée** et un **rôle précis**. Retenir la forme, c'est retenir le rôle : l'examineur du Probatoire te

demandera souvent d'identifier un symbole ou de corriger un symbole mal choisi.



Les six symboles normalisés de l'algorithme et leur rôle.

Détaillons chaque symbole.

1. Le symbole terminal (ovale).

Il contient le mot **Début** ou **Fin**. Tout algorithme commence par un ovale **Début** et se termine par au moins un ovale **Fin**. On n'écrit jamais autre chose dans un ovale.

2. Le symbole de traitement (rectangle).

Il contient les instructions qui calculent ou modifient les données : affectations et opérations arithmétiques. Exemples : $m \leftarrow (n_1 + n_2)/2$, $c \leftarrow c + 1$. Un rectangle possède **une entrée et une sortie**.

3. Le symbole d'entrée/sortie (parallélogramme).

Il contient les échanges avec l'utilisateur : **Lire** n_1 (l'utilisateur fournit une donnée) ou **Écrire** m (le programme affiche un résultat). Lui aussi possède une entrée et une sortie.

4. Le symbole de décision (losange).

Il contient une **condition** qui ne peut être que vraie ou fausse, par exemple $m \geq 10$. C'est le seul symbole qui possède **deux sorties**, obligatoirement étiquetées (par exemple Oui et Non). C'est lui qui traduit la structure alternative **Si... Alors... Sinon**.

5. Les flèches de liaison.

Elles relient les symboles et indiquent l'ordre d'exécution. Le sens de lecture général est **du haut vers le bas** ; quand une flèche remonte (cas des boucles, vus à la section suivante), la pointe de la flèche est alors indispensable.

6. Le connecteur (cercle).

Quand un algorithme est trop long pour tenir sur une colonne, on le coupe : on place un cercle

contenant une lettre (A, B...) à l'endroit de la coupure, et un cercle identique à l'endroit où le tracé reprend. Cela évite les flèches qui traversent toute la page.

ATTENTION Erreurs fréquentes avec les symboles

- Losange à une seule sortie : une décision possède toujours exactement deux sorties étiquetées (Oui/Non). Un losange avec une seule flèche sortante est une erreur de construction : que se passe-t-il si la condition est fausse ?
- Traitement dans un parallélogramme : le parallélogramme est réservé à Lire et Écrire. Écrire $m \leftarrow (n_1 + n_2)/2$ dans un parallélogramme est faux : un calcul va dans un rectangle.
- Ovale contenant un traitement : l'ovale ne contient que Début ou Fin, jamais une instruction.
- Flèches sans pointe : sans orientation, on ne sait plus dans quel sens s'exécute l'algorithme.

1.4 Règles de construction d'un algorithme correct**PROPRIÉTÉ Règles de validité d'un algorithme**

Un algorithme est correctement construit si et seulement si :

1. il possède un seul symbole Début ;
2. il possède au moins un symbole Fin, et tout chemin partant du Début aboutit à une Fin ;
3. tout symbole est relié au reste du schéma par au moins une flèche (pas de symbole « orphelin ») ;
4. chaque losange possède exactement deux sorties étiquetées (Oui/Non) ;
5. chaque rectangle et chaque parallélogramme possède une entrée et une sortie ;
6. le sens général de lecture va du haut vers le bas.

MÉTHODE Construire un algorithme en 4 étapes

Pour transformer un énoncé de problème en algorithme correct :

1. Identifier les entrées et les sorties. Quelles données l'utilisateur fournit-il (parallélogrammes Lire) ? Quels résultats le programme affiche-t-il (parallélogrammes Écrire) ? Encadrer le tout par Début et Fin.
2. Ordonner les traitements. Placer dans des rectangles, dans l'ordre, les calculs néces-

saies pour passer des entrées aux sorties.

3. Placer les tests. Pour chaque choix à faire, dessiner un losange contenant la condition, et faire partir deux flèches étiquetées Oui et Non vers les deux suites d'actions correspondantes.
4. Relier et vérifier. Tracer toutes les flèches, puis contrôler les six règles de validité : un seul Début, au moins une Fin, aucun symbole isolé, deux sorties par losange.

1.5 Exemples complets d'algorithmes

EXEMPLE Prix TTC d'un article au marché

Problème. Un commerçant du marché Central veut un petit programme qui calcule le prix TTC d'un article à partir de son prix hors taxe, sachant que la TVA au Cameroun est de 19,25 %.

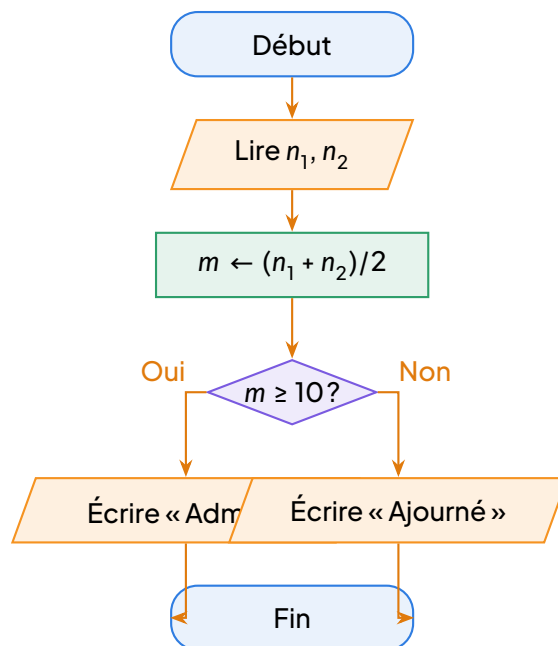
Analyse. Entrée : le prix HT (parallélogramme Lire). Traitement : $TTC \leftarrow HT \times 1,1925$ (rectangle). Sortie : affichage du prix TTC (parallélogramme Écrire).

Algorithme correspondant :

```
Algorithme PrixTTC
Variable HT, TTC : Reel
Debut
    Lire(HT)
    TTC  $\leftarrow$  HT * 1.1925
    Ecrire("Prix TTC = ", TTC, " FCFA")
Fin
```

L'algorithme est la traduction dessinée de ces quatre lignes : ovale Début, flèche vers un parallélogramme Lire HT, flèche vers un rectangle $TTC \leftarrow HT \times 1,1925$, flèche vers un parallélogramme Écrire TTC, flèche vers l'ovale Fin. Pour un article à 10000 FCFA hors taxe, le rectangle calcule $10000 \times 1,1925 = 11925$ FCFA.

Voici maintenant un algorithme complet comportant une **décision**, celui du calcul de la moyenne de deux notes avec verdict d'admission — un classique de la vie scolaire camerounaise.



Algorithme du calcul de la moyenne de deux notes : « Admis » si $m \geq 10$, « Ajourné » sinon. Observer les deux sorties étiquetées du losange et la convergence des deux branches vers l'unique Fin.

Lisons ce schéma comme le contrôleur de Mvan lit sa procédure : on démarre, on lit les deux notes (entrée), on calcule la moyenne (traitement), puis on teste : si la moyenne est supérieure ou égale à 10, on affiche « Admis », sinon « Ajourné ». Les deux branches se rejoignent ensuite vers la Fin. Remarque que le losange possède bien **deux sorties étiquetées** et que les deux chemins aboutissent à une Fin : les règles de construction sont respectées.

EXERCICE RÉSOLU 1

Identifier les symboles d'un algorithme

Énoncé. Un élève dessine l'algorithme suivant (décrit en texte) : un ovale « Début », puis un parallélogramme « Lire a », puis un rectangle « $d \leftarrow 2 \times a$ », puis un losange « $d > 50$? » dont la sortie Oui mène à un parallélogramme « Écrire d » puis à « Fin », et dont la sortie Non remonte vers le rectangle « $d \leftarrow 2 \times a$ ».

1. Citer chaque symbole utilisé et son rôle.
2. Cet algorithme respecte-t-il les règles de construction ?

SOLUTION

Solution.

1. Ovale « Début » / « Fin » : symboles terminaux, points de départ et d'arrêt. Parallélogrammes « Lire a » et « Écrire d » : symboles d'entrée/sortie. Rectangle « $d \leftarrow 2 \times a$ » : symbole de traitement (affectation et calcul). Losange « $d > 50$? » : symbole de

décision, avec deux sorties étiquetées Oui/Non.

2. Vérifions les règles de syntaxe : un seul Début (oui) ; au moins une Fin (oui) ; tout symbole est relié (oui) ; le losange a exactement deux sorties étiquetées (oui). La sortie Non qui remonte vers le rectangle forme une boucle : tant que $d \leq 50$, on double encore d . Si la valeur initiale a est strictement positive, d augmente à chaque passage et la condition $d > 50$ finit par être vraie, permettant d'atteindre la Fin. L'algorithme est donc correctement construit (structurellement valide) ; son exécution termine pour toute entrée $a > 0$.

EXERCICE 1 À toi de jouer

Le cybercafé « Le Bon Coin » à Bafoussam facture la connexion Internet à 300 FCFA l'heure, avec une réduction : toute consommation dépassant 5 heures donne droit à une remise de 10 % sur le montant total.

1. Identifie les entrées, les traitements, le test et les sorties du problème.
2. Construis l'algorithme complet en respectant les symboles normalisés et les règles de construction.
3. Vérifie ton schéma avec les six règles de validité.

CORRIGÉ Exercice 1

1. Entrée : le nombre d'heures h (parallélogramme Lire). Traitement : $m \leftarrow 300 \times h$ (rectangle). Test : $h > 5$? (losange, deux sorties). Si Oui : rectangle $m \leftarrow m \times 0,9$. Si Non : rien (branche directe). Sortie : parallélogramme Écrire m , puis ovale Fin.
2. Algorithme : Début $\rightarrow$ Lire $h \rightarrow m \leftarrow 300 \times h \rightarrow$ losange $h > 5$? ; branche Oui $\rightarrow$ rectangle $m \leftarrow m \times 0,9$; branche Non $\rightarrow$ rejoint directement la suite ; les deux branches convergent $\rightarrow$ Écrire $m \rightarrow$ Fin.
3. Vérification : un seul Début, une Fin atteinte par les deux branches, aucun symbole isolé, le losange a deux sorties étiquetées : le schéma est valide. Contrôle numérique : pour $h = 8$, $m = 2400$ puis $m = 2400 \times 0,9 = 2160$ FCFA ; pour $h = 3$, $m = 900$ FCFA sans remise.

À RETENIR L'essentiel de la section

- Un **algorithme** est la représentation graphique normalisée d'un algorithme : sym-

boles + flèches orientées, lus du haut vers le bas.

- Un **organigramme** au sens large décrit n'importe quel processus ; l'algorithme en est le cas particulier algorithmique.
- Les symboles : ovale (Début/Fin), rectangle (traitement), parallélogramme (Lire/Écrire), losange (décision à deux sorties), flèches (liaison), cercle (connecteur).
- Règles d'or : un seul Début, au moins une Fin, tout symbole relié, chaque losange a exactement deux sorties étiquetées.

Maintenant que les symboles n'ont plus de secret pour toi, la section suivante te montrera comment traduire en algorithmes les trois grandes structures de contrôle : la séquence, l'alternative et les boucles.

2 STRUCTURES DE CONTRÔLE EN ALGORITHMES

Dans la section précédente, nous avons découvert les symboles de base de l'algorithme : l'ovale de début et de fin, le rectangle de traitement, le parallélogramme d'entrée-sortie, le losange de décision et les flèches qui relient le tout. Mais un symbole isolé ne résout rien : c'est leur **agencement** qui donne naissance à un raisonnement. Tout comme en grammaire on assemble des mots selon des règles pour former des phrases, en algorithmique on assemble les symboles selon des schémas précis appelés **structures de contrôle**. Cette section te présente chacune d'elles, son pseudo-code et sa traduction graphique, afin que tu saches lire et construire n'importe quel algorithme.

DÉFINITION Algorithme vs Organigramme

Au sens strict du programme, l'**algorithme** (ou diagramme algorithmique) est la représentation graphique normalisée d'un algorithme (traitement de données, calculs, structures de contrôle). L'**organigramme** est un terme plus large, souvent utilisé en gestion pour décrire le fonctionnement d'un service ou d'une entreprise (organigramme administratif). Dans le cadre de ce cours, nous utiliserons exclusivement le terme algorithme et la norme algorithmique (losange pour le test, parallélogramme pour les E/S, rectangle pour les traitements).

PROPRIÉTÉ Théorème de Böhm-Jacopini (admis)

Tout algorithme, aussi complexe soit-il, peut être écrit en utilisant uniquement trois structures de contrôle :

- la **séquence** (instructions exécutées les unes après les autres) ;

- l'**alternative** (choix entre plusieurs traitements selon une condition);
- la **répétition** (ré-exécution d'un traitement tant qu'une condition est vérifiée).

Ce résultat, démontré en 1966 par les informaticiens italiens Corrado Böhm et Giuseppe Jacopini, est admis à notre niveau. Conséquence pratique : si tu maîtrises ces trois schémas graphiques, tu peux construire l'algorithme de n'importe quel problème.

LESAVAIS-TU ? Naissance du diagramme de flux

Les premiers « flowcharts » (diagrammes de flux) ont été utilisés en 1947 par Herman Goldstine et John von Neumann pour planifier les programmes de l'ordinateur ENIAC. La norme ISO 5807 (1985) a ensuite standardisé les symboles que nous utilisons encore aujourd'hui : ovale, rectangle, losange, parallélogramme. Au Cameroun, l'enseignement de l'algorithme suit cette norme internationale, gage de lisibilité partagée entre développeurs du monde entier.

2.1 La structure séquentielle

Intuition. La plupart des tâches de la vie se déroulent dans un ordre fixe : pour préparer le ndolé, on lave les feuilles, puis on les cuit, puis on ajoute les arachides. Inverser les étapes donnerait un résultat absurde. En algorithmique, c'est la même chose : certaines instructions doivent s'exécuter dans un ordre précis, sans choix ni répétition.

DÉFINITION Structure séquentielle

La **structure séquentielle** est l'enchaînement d'instructions exécutées les unes après les autres, dans l'ordre où elles sont écrites, du haut vers le bas de l'algorithme. Graphiquement, ce sont des rectangles (traitements) ou parallélogrammes (entrées/sorties) empilés verticalement et reliés par des flèches orientées vers le bas.

EXEMPLE Séquence : calcul de la moyenne de deux notes

Pseudo-code :

```
Algorithme Moyenne
Variables n1, n2, m : Reel
Debut
  Ecrire("Premiere note : ")
  Lire(n1)
  Ecrire("Deuxieme note : ")
  Lire(n2)
  m <- (n1 + n2) / 2
```

```
Ecrire("Moyenne = ", m)
Fin
```

Dans l'algorithme, chaque instruction occupe un symbole, et les flèches descendent sans jamais remonter : aucune instruction n'est sautée, aucune n'est répétée.

2.2 La structure alternative

Intuition. Un proviseur qui consulte une moyenne ne réagit pas toujours pareil : si la moyenne est supérieure ou égale à 10, l'élève est admis ; sinon, il est ajourné. L'algorithme doit donc choisir un traitement selon une condition. C'est le rôle du **losange**, le seul symbole d'où partent deux flèches distinctes.

DÉFINITION Alternative simple : Si...Alors

L'**alternative simple** exécute un traitement uniquement si une condition est vraie. Si la condition est fausse, on ne fait rien et on continue vers la suite. Dans l'algorithme, le losange possède une branche « Vrai » qui contient le traitement, et une branche « Faux » qui rejoint directement le flux principal sans traitement intermédiaire ; les deux branches se rejoignent ensuite en un point unique.

```
Si age >= 18 Alors
    Ecrire("Vous êtes majeur")
FinSi
```

DÉFINITION Alternative complète : Si...Alors...Sinon

L'**alternative complète** propose deux traitements exclusifs : l'un s'exécute si la condition est vraie, l'autre si elle est fausse. Dans l'algorithme, les deux branches du losange contiennent chacune un traitement (ou un bloc d'instructions) et se rejoignent obligatoirement en un point unique avant la suite du programme.

```
Si moyenne >= 10 Alors
    Ecrire("Admis")
Sinon
    Ecrire("Ajourné")
FinSi
```

ATTENTION Les deux branches doivent se rejoindre

Erreur fréquente des débutants : faire « sortir » une branche du losange sans la faire revenir vers le flux principal (flèche qui part dans le vide ou vers un autre losange sans passer par le point de jonction). Un algorithme n'a qu'un seul flux de contrôle : après le choix, tout le monde reprend le même chemin. De même, n'oublie jamais d'étiqueter les branches (Vrai/Faux ou Oui/Non) : un losange sans étiquette est illisible et sera pénalisé à l'examen.

DÉFINITION Alternative à choix multiples : Selon...Cas

Lorsqu'une variable peut prendre plusieurs valeurs précises et que chaque valeur déclenche un traitement différent, on utilise la structure **Selon...Cas** (aussi notée **Selon...Faire**). Dans l'algorithme, on la représente par une cascade de losanges : chaque losange teste une valeur (ex. `var = val`), la branche « Vrai » mène au traitement correspondant, la branche « Faux » mène au losange suivant. Un cas **Sinon** (par défaut) termine la cascade.

Selon mention Faire

Cas 1 : Ecrire("Passable")

Cas 2 : Ecrire("Assez bien")

Cas 3 : Ecrire("Bien")

Sinon : Ecrire("Mention inconnue")

FinSelon

2.3 Les structures répétitives (boucles)

Intuition. Saisir les notes de 60 élèves d'une classe de Première TI exige de répéter 60 fois le même traitement. Écrire 60 fois les mêmes instructions serait absurde : on utilise une **boucle**. Il existe trois formes de boucles, qui diffèrent par la position du test et la façon de compter.

DÉFINITION Boucle TantQue

La boucle **TantQue** place le test avant le corps de boucle : on vérifie d'abord la condition ; si elle est vraie, on exécute le corps, puis on remonte tester à nouveau. La boucle s'arrête dès que la condition devient fausse. Conséquence essentielle : si la condition est fausse dès le départ, le corps ne s'exécute jamais (zéro itération possible). Dans l'algorithme, le losange est placé au-dessus du corps, et une flèche remonte du corps vers le losange.

`i <- 1`

TantQue `i <= 5` Faire

 Ecrire("Bonjour")

`i <- i + 1`

FinTantQue

DÉFINITION Boucle Répéter...Jusqu'à

La boucle **Répéter...Jusqu'à** place le test après le corps : on exécute d'abord le corps, puis on teste la condition d'arrêt. Conséquence essentielle : le corps s'exécute au moins une fois, même si la condition est vraie dès le départ. Dans l'algorithme, le corps est placé au-dessus du losange, et la flèche de la branche « Faux » (condition non réalisée) remonte vers le corps.

Repeter

Ecrire("Entrez une note (0 à 20) : ")

Lire(note)

Jusqu'à (note >= 0) ET (note <= 20)

EXEMPLE Saisie contrôlée d'une note entre 0 et 20

Un professeur saisit les notes au clavier et peut se tromper (taper 25 ou -3). On veut obliger la saisie d'une valeur valide. Comme l'utilisateur doit saisir au moins une fois, la boucle **Répéter...Jusqu'à** est le bon choix. L'algorithme est le suivant : ovale « Début », parallélogramme « Saisir note », losange « $0 \leq \text{note} \leq 20$? » dont la branche « Non » (Faux) remonte vers la saisie et la branche « Oui » (Vrai) descend vers l'ovale « Fin ». Tant que la note est hors de l'intervalle [0 ; 20], la saisie est redemandée.

MÉTHODE Choisir entre TantQue et Répéter

Pose-toi la question : « Le corps de la boucle doit-il pouvoir ne jamais s'exécuter ? »

1. Oui (le traitement peut être inutile dès le départ, par exemple parcourir une liste qui pourrait être vide, ou traiter des fichiers s'il n'y en a pas) : utilise un **TantQue**, le test protège l'entrée.
2. Non (le traitement doit s'exécuter au moins une fois, par exemple une saisie clavier, un menu affiché à l'utilisateur) : utilise un **Répéter...Jusqu'à**.
3. Vérifie ensuite la logique de la condition : le **TantQue** continue tant que la condition est vraie ; le **Répéter** s'arrête quand la condition devient vraie. Les deux conditions sont donc contraires l'une de l'autre.

DÉFINITION Boucle Pour

La boucle **Pour** s'utilise lorsqu'on connaît à l'avance le nombre d'itérations (ou qu'on peut le calculer avant d'entrer dans la boucle). Elle regroupe en une seule ligne trois éléments :

l'initialisation de la variable de contrôle, le test de fin et l'incréméntation (ou décrémentation) automatique. La syntaxe complète admet un pas optionnel : Pour i allant de debut a fin [par pas de p] Faire.

```
Pour  $i$  allant de 1 a  $n$  Faire
   $S \leftarrow S + i$ 
FinPour
```

PROPRIÉTÉ Équivalence Pour / TantQue

Toute boucle Pour peut se réécrire avec un TantQue en rendant explicites les trois éléments. Cela prouve que le Pour n'ajoute pas de puissance expressive (conforme à Böhm-Jacopini), mais il évite les erreurs d'oubli d'initialisation ou d'incréméntation.

```
 $i \leftarrow 1$            // Initialisation
TantQue  $i \leq n$  Faire // Test de fin
   $S \leftarrow S + i$ 
   $i \leftarrow i + 1$  // Incréméntation
FinTantQue
```

Dans l'algorithme, une boucle Pour se dessine donc exactement comme un TantQue : un rectangle d'initialisation, un losange de test, un corps, un rectangle d'incréméntation, et une flèche qui remonte.

ATTENTION La boucle infinie : le piège mortel

L'erreur la plus classique : oublier de modifier la variable de contrôle dans le corps de la boucle (ou la modifier dans le mauvais sens). Si la condition porte sur i et que i ne change jamais, la condition reste vraie éternellement : le programme « tourne » sans fin (à l'exécution, l'ordinateur semble bloqué, CPU à 100%). Vérifie systématiquement que chaque passage dans la boucle rapproche la condition de son changement de valeur : c'est ce qu'on appelle la condition de sortie (ou variant de boucle).

EXPÉRIENCE Dessin d'algorithme : Maximum de trois nombres

Objectif. Construire l'algorithme complet d'un algorithme qui lit trois entiers et affiche le plus grand. Matériel. Papier quadrillé, crayon, gomme (ou logiciel libre [Diagram.net](https://diagram.net/) / [Draw.io](https://draw.io) avec la bibliothèque « Flowchart »). Manipulation.

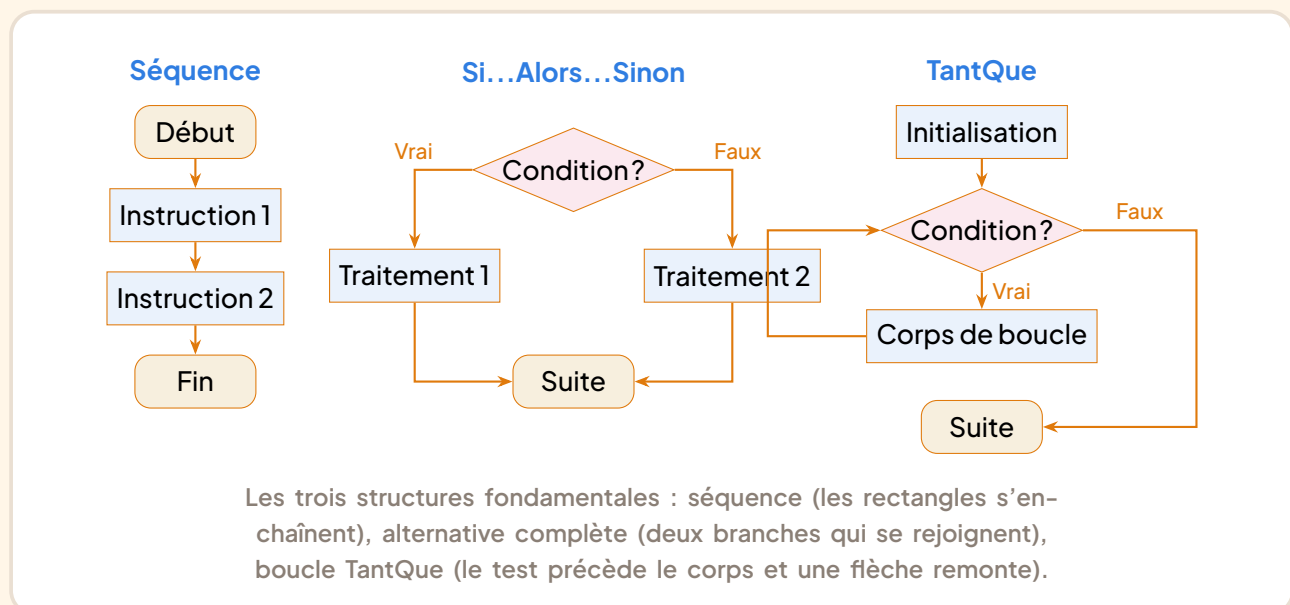
1. Dessiner l'ovale Début.
2. Trois parallélogrammes Lire a , Lire b , Lire c (séquence).

3. Rectangle_{max} ← a (initialisation).
4. Losange $b > \text{max}$? : branche Vrai → rectangle_{max} ← b ; branche Faux → jonction.
5. Losange $c > \text{max}$? : branche Vrai → rectangle_{max} ← c ; branche Faux → jonction.
6. Parallélogramme Écrire max.
7. Ovale Fin.
8. Relier tous les symboles par des flèches continues. Étiqueter Oui/Non sur les losanges.

Observation. On a utilisé deux alternatives simples imbriquées (séquence de deux Si...Alors). Aucune boucle n'était nécessaire. Interprétation. Ce schéma montre qu'un problème à données fixes (3 nombres) se résout par une structure séquentielle et alternative, sans répétition.

2.4 Correspondance systématique pseudo-code / algorithme

La figure suivante place côte à côte les trois structures fondamentales du théorème de Böhm-Jacopini : observe bien la position du losange et le sens des flèches dans chacune.



Le tableau ci-dessous résume la correspondance à connaître par cœur pour le Probatoire.

Structure	Pseudo-code	Signature graphique
Séquence	I1 ; I2 ; I3	Rectangles empilés, flèches descendantes
Si...Alors	Si C Alors T FinSi	Losange, branche « Faux » vers jonction directe
Si...Sinon	Si C Alors T1 Sinon T2 FinSi	Losange, deux branches qui se rejoignent
Selon...Cas	Selon v Faire Cas...	Cascade de losanges
TantQue	TantQue C Faire T FinTantQue	Losange avant le corps; 0 itération possible
Répéter	Repeter T Jusqu'a C	Losange après le corps; au moins 1 itération
Pour	Pour i de 1 a n Faire T FinPour	Initialisation + losange + incrémentation

EXERCICE RÉSOLU 2 Somme des n premiers entiers : de l'énoncé à l'algorithme

Énoncé. Construire l'algorithme d'un algorithme qui lit un entier $n \geq 1$ et calcule, à l'aide d'une boucle **Pour**, la somme $S = 1 + 2 + \dots + n$.

SOLUTION

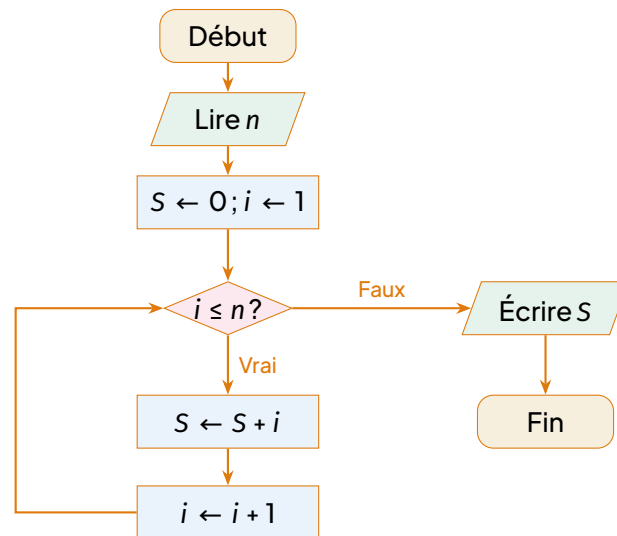
Solution. On connaît le nombre d'itérations (n tours), donc la boucle **Pour** est adaptée. Il faut deux variables : S (l'accumulateur, initialisé à 0) et i (le compteur). Le pseudo-code :

```

Algorithme SommeN
Variables n, i, S : Entier
Debut
    Lire(n)
    S ← 0
    Pour i allant de 1 a n Faire
        S ← S + i
    FinPour
    Ecrire("Somme = ", S)
Fin

```

Traduction en algorithme : après la lecture de n et l'initialisation $S \leftarrow 0$ et $i \leftarrow 1$, le losange teste $i \leq n$. Branche « Vrai » : on ajoute i à S , on incrémente i , puis on remonte au test. Branche « Faux » : on affiche S et on termine. Vérification mentale pour $n = 4$: S prend successivement les valeurs 1, 3, 6, 10 ; quand i vaut 5, le test échoue et on affiche 10, ce qui est correct.



Algorithme de la boucle Pour calculant $S = 1 + 2 + \dots + n$: l'initialisation, le test et l'incrément sont rendus explicites, comme dans un TantQue.

EXERCICE 2 À toi de jouer

1. Dessine l'algorithme d'un algorithme qui lit l'âge d'une personne et affiche « Mineur » ou « Majeur ».
2. Dessine, avec un Répéter... Jusqu'à, l'algorithme d'une saisie contrôlée d'un mot de passe : on redemande tant que la saisie est différente de « ti2024 ».
3. Réécris la boucle Pour i allant de 2 à 10 par pas de 2 sous forme de TantQue, puis indique combien d'itérations elle effectue.

CORRIGÉ Exercice

1. Ovalé « Début », parallélogramme « Lire âge », losange « âge ≥ 18 ? » : branche « Vrai » vers le parallélogramme « Afficher Majeur », branche « Faux » vers « Afficher Mineur » ; les deux branches se rejoignent vers l'ovalé « Fin ».
2. Ovalé « Début », parallélogramme « Saisir mot », losange « mot = "ti2024" ? » dont la branche « Faux » (Non) remonte vers la saisie et la branche « Vrai » (Oui) descend vers « Fin ». Le test est après le corps : la saisie a lieu au moins une fois.
3. Équivalent TantQue :

```

i ← -2
TantQue i ≤ 10 Faire
    // corps de la boucle

```

```
i <- i + 2  
FinTantQue
```

La variable i prend les valeurs 2, 4, 6, 8, 10 : la boucle effectue donc 5 itérations.

À RETENIR L'essentiel de la section

- Trois structures suffisent à tout construire (Böhm-Jacopini) : **séquence**, **alternative**, **répétition**.
- Le TantQue teste avant : le corps peut ne jamais s'exécuter. Le Répéter... Jusqu'à teste après : le corps s'exécute au moins une fois.
- Le Pour = initialisation + test + incrémentation ; il se dessine comme un TantQue.
- Toute boucle doit contenir une instruction qui fait progresser la variable de contrôle, sinon : boucle infinie.
- Un losange a deux branches étiquetées (Vrai/Faux) qui doivent se rejoindre.

3 DÉROULEMENT D'UN ALGORIGRAMME ET TABLE DE TRACE

Dans la section précédente, nous avons appris à représenter les structures de contrôle (séquence, alternative, boucles) à l'aide des symboles normalisés de l'algorigramme. Mais un dessin, aussi correct soit-il, ne prouve rien à lui seul : comment être certain que l'algorigramme que l'on vient de tracer calcule bien ce qu'on attend de lui ? Comment convaincre un examinateur — ou soi-même — que la boucle s'arrête au bon moment, que le test aiguille les données dans la bonne branche, qu'aucune variable ne garde une valeur fantaisiste ? La réponse tient en une technique simple, redoutablement efficace et exigée au Probatoire : le **déroulement** de l'algorigramme, consigné dans une **table de trace**. C'est l'art d'exécuter un algorithme « à la main », comme le ferait l'ordinateur, mais au ralenti et sur le papier.

3.1 Dérouler un algorigramme : jouer à être l'ordinateur

DÉFINITION Déroulement d'un algorigramme

Dérouler un algorigramme (on dit aussi exécuter à la main ou simuler l'exécution), c'est suivre le chemin des flèches depuis le symbole Début jusqu'au symbole Fin, en choisissant des **valeurs concrètes** pour les données d'entrée et en appliquant mentalement chaque instruction rencontrée : à chaque affectation, on calcule et on note la nouvelle

valeur de la variable ; à chaque test (losange), on évalue la condition avec les valeurs courantes et on emprunte la branche Oui ou la branche Non.

L'intuition est la suivante : l'ordinateur n'est qu'un exécutant très rapide mais sans intelligence. Il fait exactement ce que dit l'algorithme, une instruction à la fois, sans jamais anticiper. En déroulant l'algorithme, tu te mets à sa place : tu deviens le processeur. Si, avec tes valeurs concrètes, le résultat final est correct, ton algorithme est probablement bon ; s'il est faux, tu viens de débusquer une erreur avant même d'avoir écrit la moindre ligne de programme.

MÉTHODE Dérouler un algorithme en cinq étapes

1. Choisir un jeu d'essai : fixer des valeurs concrètes pour toutes les données d'entrée (par exemple $a = 7$, $b = 12$, $c = 5$).
2. Numéroter les étapes : parcourir l'algorithme en suivant les flèches et attribuer un numéro d'étape à chaque symbole traversé (lecture, affectation, test).
3. Noter chaque affectation : dès qu'une variable reçoit une valeur, calculer le membre de droite avec les valeurs actuelles des variables, puis inscrire la nouvelle valeur. Une variable ne change que lorsqu'elle est affectée.
4. Vérifier chaque test : à chaque losange, remplacer les variables par leurs valeurs courantes, évaluer la condition (Vrai ou Faux) et noter la branche empruntée. Ne jamais deviner la branche « au feeling ».
5. Conclure : à l'arrivée au symbole Fin, comparer le résultat affiché au résultat attendu. S'ils coïncident, le jeu d'essai est validé ; sinon, remonter la trace pour localiser l'étape fautive.

3.2 La table de trace : mémoire écrite de l'exécution

Dérouler « dans sa tête » est vite insuffisant : dès qu'une boucle tourne cinq ou six fois, on perd le fil des valeurs. D'où l'outil central de cette leçon.

DÉFINITION Table de trace

Une **table de trace** (ou table d'exécution) est un tableau dont :

- les colonnes correspondent aux **variables** de l'algorithme (plus éventuellement une colonne pour le numéro d'étape, une pour la condition évaluée et une pour les affichages) ;
- les lignes correspondent aux **étapes d'exécution** : chaque fois qu'une instruction modifie l'état (affectation, lecture, test), on ajoute une ligne portant les nouvelles valeurs.

On n'écrit dans une case que la valeur qui change ; les cases laissées vides signifient « valeur inchangée depuis la ligne précédente ».

PROPRIÉTÉ Règles de construction d'une table de trace

- Une ligne = un événement (affectation, lecture ou évaluation d'un test). On ne regroupe jamais deux affectations sur la même ligne.
- Pour une **boucle**, on consacre une ligne par passage dans le corps de la boucle, et l'on suit explicitement le compteur et la condition d'arrêt.
- La dernière ligne doit permettre de lire le résultat final et de le comparer au résultat attendu.

3.3 Exemple complet : trace du maximum de trois nombres

Reprenons l'algorithme classique du maximum de trois nombres, qui enchaîne deux alternatives. Voici d'abord l'algorithme en pseudo-code :

```
Algorithme MaxTrois
Variables a, b, c, max : Entier
Debut
  Lire(a, b, c)
  max ← a
  Si b > max Alors
    max ← b
  FinSi
  Si c > max Alors
    max ← c
  FinSi
  Ecrire("Le maximum est : ", max)
Fin
```

Déroulons-le avec le jeu d'essai $(a, b, c) = (7, 12, 5)$. Le résultat attendu est évidemment 12.

Étape	Instruction	a	b	c	max	Test
1	Lire(a, b, c)	7	12	5	—	
2	max ← a				7	
3	b > max?					12 > 7 : Vrai
4	max ← b				12	
5	c > max?					5 > 12 : Faux
6	Ecrire(max)					Affiche 12

Table de trace de l'algorithme `MaxTrois` pour le jeu d'essai (7,12,5). Les cases vides signifient « valeur inchangée ». Le résultat affiché (12) est bien le maximum attendu : le jeu d'essai est validé.

Lisons cette trace ligne par ligne. À l'étape 1, la lecture remplit les trois variables d'entrée. À l'étape 2, `max` reçoit 7 : c'est notre candidat provisoire. À l'étape 3, le test `b > max` s'écrit concrètement `12 > 7`, ce qui est **vrai** : on entre dans la branche « Alors » et, à l'étape 4, `max` devient 12. À l'étape 5, le second test `c > max` s'écrit `5 > 12`, ce qui est **faux** : on saute la branche « Alors », `max` ne change pas. L'affichage final donne 12 : correct.

ATTENTION Erreur fréquente : évaluer un test avec d'anciennes valeurs

Le piège classique consiste, à l'étape 5, à comparer `c` avec la valeur initiale de `max` (7) au lieu de sa valeur courante (12). On écrirait alors `5 > 7 : Faux` — le résultat reste bon ici par chance, mais avec le jeu d'essai (7, 12, 9), ce raisonnement fautif donnerait `9 > 7 : Vrai` et l'on conclurait à tort que `max` devient 9... alors que c'est justement le bon comportement pour de mauvaises raisons ! En traçant, on utilise toujours la dernière valeur écrite dans la colonne de la variable, jamais une valeur mémorisée de tête.

3.4 Tracer une boucle : un passage, une ligne

Avec une boucle, la table de trace prend tout son sens : c'est elle qui permet de voir la répétition. Considérons l'algorithme qui calcule 2^n par multiplications successives :

```

Algorithme PuissanceDeux
Variables n, i, p : Entier
Debut
  Lire(n)
  p ← 1
  Pour i Allant de 1 à n Faire
    p ← p * 2
  FinPour
  Ecrire("2^", n, " = ", p)
Fin

```

Traçons-le pour $n = 4$. Le résultat attendu est $2^4 = 16$.

Étape	Instruction	n	i	p	Condition
1	Lire(n); $p \leftarrow 1$	4	—	1	
2	Entrée dans le Pour		1		$i \leq 4$: Vrai
3	$p \leftarrow p \times 2$			2	
4	Tour suivant		2		$i \leq 4$: Vrai
5	$p \leftarrow p \times 2$			4	
6	Tour suivant		3		$i \leq 4$: Vrai
7	$p \leftarrow p \times 2$			8	
8	Tour suivant		4		$i \leq 4$: Vrai
9	$p \leftarrow p \times 2$			16	
10	Incrémentation		5		$i \leq 4$: Faux, sortie
11	Ecrire(p)				Affiche 16

Table de trace de la boucle calculant 2^n pour $n = 4$. On suit explicitement le compteur i et la condition $i \leq n$ à chaque passage : la boucle s'exécute exactement 4 fois et p vaut successivement 2, 4, 8, 16.

Observe la mécanique : le compteur i prend les valeurs 1, 2, 3, 4, puis 5 à l'incrément finale ; c'est à ce moment-là seulement que la condition $i \leq n$ devient fausse et que la boucle s'arrête. La variable p double à chaque passage : $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$. La trace rend cette évolution visible et vérifiable, ligne après ligne.

ATTENTION Erreur fréquente : oublier de mettre à jour le compteur dans la trace

L'erreur la plus fréquente — chez les élèves comme dans les programmes réels — est d'oublier de mettre à jour le compteur dans la trace (ou l'instruction d'incrément dans une boucle `Tant que`). Conséquence immédiate : la condition d'arrêt, évaluée avec un compteur figé, reste éternellement vraie, et ta trace ne se termine jamais... exactement comme la **boucle infinie** que subirait l'ordinateur. Si, en traçant, tu remarques que les lignes se répètent à l'identique sans que le compteur ne bouge, tu viens de détecter une boucle infinie : c'est précisément à cela que sert la table de trace ! Vérifie donc à chaque passage : « qu'est-ce qui fait progresser la condition d'arrêt ? »

3.5 Exemple chiffré : somme des éléments d'un tableau

EXEMPLE Trace complète : somme de 4 notes

Un enseignant du lycée de Nkongsamba veut calculer la somme des 4 notes d'un élève, stockées dans un tableau T d'indices 1 à 4. Voici l'algorithme :

```

Algorithme SommeNotes
Variables i, S : Entier
Tableau T[1..4] : Entier
Debut
    // On suppose le tableau déjà rempli : T = (12, 8, 15, 9)
    S ← 0
    Pour i Allant de 1 à 4 Faire
        S ← S + T[i]
    FinPour
    Ecrire("Somme = ", S)
Fin

```

Trace complète pour $T = (12, 8, 15, 9)$:

Étape	Instruction	i	S	Détail
1	$S \leftarrow 0$	—	0	Initialisation
2	Tour 1	1		$i \leq 4$: Vrai
3	$S \leftarrow S + T[1]$		12	$0 + 12$
4	Tour 2	2		$i \leq 4$: Vrai
5	$S \leftarrow S + T[2]$		20	$12 + 8$
6	Tour 3	3		$i \leq 4$: Vrai
7	$S \leftarrow S + T[3]$		35	$20 + 15$
8	Tour 4	4		$i \leq 4$: Vrai
9	$S \leftarrow S + T[4]$		44	$35 + 9$
10	Sortie	5		$i \leq 4$: Faux
11	Ecrire(S)			Affiche 44

Vérification : $12 + 8 + 15 + 9 = 44$. La trace confirme le résultat et montre le rôle d'accumulateur joué par S : initialisée à 0 (étape 1 — indispensable!), elle absorbe une note à chaque passage.

3.6 La trace, outil de vérification et de détection d'erreurs

La table de trace n'est pas un exercice scolaire artificiel : c'est un véritable **outil de diagnostic**.

À RETENIR

Ce que la table de trace permet de détecter

- Une valeur fausse : si le résultat final ne correspond pas au résultat attendu, on remonte la trace ligne par ligne jusqu'à la première valeur incorrecte ; l'instruction fautive est celle de cette ligne (mauvaise formule, variable mal initialisée, test inversé).
- Une boucle infinie : si les lignes se répètent sans que la condition d'arrêt ne progresse

(compteur oublié, condition mal écrite comme $i \geq 0$ pour un compteur qui croît), la trace ne se termine pas : l'algorithme est faux.

- Un débordement de borne : si une boucle sur un tableau de 4 cases tente d'accéder à $T[5]$, la trace le révèle immédiatement.

EXERCICE RÉSOLU 3 Traquer l'erreur dans une moyenne de notes

Énoncé. Voici un algorithme censé calculer la moyenne de 3 notes saisies :

```

Algorithme MoyenneFautive
Variables i, note, S : Entier
Debut
  Pour i Allant de 1 a 3 Faire
    Lire(note)
    S ← S + note
  FinPour
  Ecrire("Moyenne = ", S / 3)
Fin
  
```

Dérouler cet algorithme avec les notes (10, 12, 14) et identifier l'erreur.

SOLUTION

Solution détaillée. Construisons la table de trace en toute rigueur :

Étape	Instruction	i	note	S
1	(avant la boucle)	—	—	???
2	Tour 1 : Lire(note)	1	10	
3	$S \leftarrow S + \text{note}$			??? + 10

Dès l'étape 3, le drame : la variable S n'a jamais été initialisée. Sa valeur est indéterminée (en mémoire, elle contient ce qui traînait là avant : une « valeur poubelle »). La somme S est donc fausse, et la moyenne affichée sera imprévisible — parfois juste par miracle, le plus souvent aberrante.

Correction : ajouter $S \leftarrow 0$ juste avant le **Pour**. La trace corrigée donne alors $S = 10$, puis 22, puis 36, et la moyenne affichée est $36/3 = 12$, ce qui est bien la moyenne de (10, 12, 14). Morale : toute variable utilisée dans une expression doit avoir reçu une valeur au préalable — la table de trace est impitoyable avec les initialisations oubliées, car elle oblige à écrire « ??? » dans la case.

3.7 Choisir de bons jeux d'essai

Un seul déroulement ne suffit pas à valider un algorithme : le jeu d'essai (7, 12, 5) a validé `MaxTrois`, mais qu'en est-il si le maximum est le premier nombre ? Ou si deux nombres sont égaux ? Le

choix des valeurs de test est un savoir-faire à part entière.

MÉTHODE Constituer un jeu d'essai pertinent

Pour tester sérieusement un algorithme, prévoir au minimum :

1. Des valeurs ordinaires : un cas « normal », sans piège (par exemple (7, 12, 5) pour le maximum).
2. Des valeurs limites : les valeurs aux frontières du domaine — 0, la borne exacte d'une boucle ($n = 1$, $n = 4$ pour une boucle de 1 à 4), le seuil d'un test (une note exactement égale à 10 pour un test « admis si note ≥ 10 »).
3. Des cas particuliers : valeurs égales entre elles ((8, 8, 8)), maximum en première position ((12, 7, 5)), tableau déjà trié, entrée nulle ou vide quand elle est autorisée.

Pour chaque jeu d'essai, on détermine d'abord le résultat attendu à la main, puis on trace : c'est la comparaison des deux qui conclut.

EXEMPLE Jeux d'essai pour `MaxTrois`

Type	Jeu d'essai	Résultat attendu	Ce qu'on vérifie
Ordinaire	(7, 12, 5)	12	Branche « $b > \max$ » prise
Cas particulier	(12, 7, 5)	12	max dès la 1 ^{re} valeur : aucune branche prise
Cas particulier	(8, 8, 8)	8	Égalités : tests stricts jamais vrais, résultat correct
Valeur limite	(0, 0, 0)	0	Fonctionnement avec des zéros

Chacun de ces jeux d'essai traverse un chemin différent de l'algorithme : à eux quatre, ils couvrent toutes les branches. C'est la notion de **couverture des chemins**.

LESAVAIS-TU ? Le « dry run » des testeurs professionnels

Ce que tu viens d'apprendre porte un nom dans l'industrie : les testeurs et développeurs professionnels appellent cette technique le dry run (« exécution à sec ») ou l'**exécution symbolique** quand on la fait avec des outils automatiques. Avant même de lancer un programme, un bon développeur le « déroule » mentalement avec quelques valeurs, exactement comme tu l'as fait avec ta table de trace. Dans les grandes entreprises du logiciel, des outils d'analyse statique simulent ainsi des milliers d'exécutions pour traquer les boucles infinies et les variables non initialisées. Les équipes qui développent des applications mobiles à Douala ou à Yaoundé utilisent quotidiennement cette démarche : une heure de trace

sur papier évite souvent des journées entières de débogage. Tu pratiques donc, dès la Première TI, un geste professionnel authentique.

EXERCICE 3 À toi de tracer

On considère l'algorithme suivant :

```

Algorithme ComptePairs
Variables i, nb : Entier
Tableau T[1..5] : Entier
Debut
    // T = (3, 8, 7, 4, 10)
    nb ← 0
    Pour i Allant de 1 à 5 Faire
        Si T[i] mod 2 = 0 Alors
            nb ← nb + 1
        FinSi
    FinPour
    Ecrire(nb)
Fin
  
```

1. Construire la table de trace complète de cet algorithme.
2. Quel est le résultat affiché ? Vérifier par un calcul direct.
3. Proposer un jeu d'essai « limite » pour ce tableau et tracer à nouveau.

CORRIGÉ Exercice 1

1. Table de trace (colonnes : étape, i, T[i], test, nb) :

Étape	i	T[i]	Test $T[i] \bmod 2 = 0$	nb
Init	—	—		0
Tour 1	1	3	Faux	
Tour 2	2	8	Vrai	1
Tour 3	3	7	Faux	
Tour 4	4	4	Vrai	2
Tour 5	5	10	Vrai	3
Sortie	6	—		Affiche 3

2. Le résultat affiché est 3. Vérification directe : dans (3, 8, 7, 4, 10), les nombres pairs sont 8, 4 et 10, soit exactement 3 valeurs. La trace est conforme.
3. Jeu d'essai limite : $T = (1, 3, 5, 7, 9)$ (aucun pair). La trace montre que nb reste à 0 pendant toute la boucle et que l'algorithme affiche 0 : il gère correctement le cas extrême, grâce

à l'initialisation `nb <- 0`. Sans elle, l'affichage aurait été une valeur indéterminée.

À RETENIR L'essentiel de la section

- **Dérouler** un algorithme, c'est suivre les flèches avec des valeurs concrètes en appliquant chaque instruction comme le ferait l'ordinateur.
- La **table de trace** consigne cette exécution : les colonnes sont les variables, les lignes les étapes ; une boucle occupe une ligne par passage, avec suivi explicite du compteur et de la condition d'arrêt.
- On utilise toujours la dernière valeur écrite d'une variable pour évaluer un test.
- La trace permet de **vérifier** une solution (résultat final = résultat attendu) et de **détecter** les erreurs : variable non initialisée, valeur fausse, boucle infinie.
- Un bon **jeu d'essai** combine valeurs ordinaires, valeurs limites (0, bornes exactes) et cas particuliers (égalités, positions extrêmes), de façon à couvrir tous les chemins de l'algorithme.

Tu sais désormais exécuter un algorithme à la main et prouver — ou réfuter — son exactitude. Dans la section suivante, nous franchirons un nouveau cap : transformer un algorithme en algorithme écrit en pseudo-code, et apprendre à compter ses instructions pour mesurer son coût.

4 DE L'ALGORIGRAMME À L'ALGORITHME ; COMPTAGE D'INSTRUCTIONS

Dans la section précédente, tu as appris à dérouler un algorithme à la main, case par case, grâce à la table de trace : tu as suivi le fil d'exécution comme le ferait l'ordinateur. Mais un algorithme, aussi pratique soit-il pour visualiser un raisonnement, reste un dessin. Or, pour programmer réellement — en C, en JavaScript —, il faut un texte : le pseudo-code, puis le programme. Il faut aussi savoir répondre à une question que tout informaticien se pose : « mon algorithme fait-il beaucoup de travail ? ». Cette section t'apprend donc deux gestes professionnels complémentaires : **traduire** un algorithme en algorithme écrit, et **compter** les instructions d'un algorithme pour évaluer son coût.

4.1 Transformer un algorithme en pseudo-code

Pourquoi traduire ?

Imagine un technicien d'un cybercafé de Bafoussam qui a dessiné sur papier l'algorithme d'un programme de calcul de facturation. Pour le confier à un stagiaire qui doit le coder, le dessin ne suffit pas : il faut un texte structuré, sans ambiguïté, que le stagiaire pourra recopier presque mot pour mot en langage C. La traduction d'un algorithme en pseudo-code est ce passage

du dessin au texte.

DÉFINITION Traduction d'un algorithme

Traduire un algorithme en algorithmme consiste à parcourir le schéma **du haut vers le bas**, en suivant les flèches, et à remplacer chaque symbole graphique par l'instruction de pseudo-code qui lui correspond, en respectant l'**indentation** (les décalages vers la droite) qui matérialise l'imbrication des structures.

Le dictionnaire de correspondance est le suivant :

Symbole de l'algorithme	Traduction en pseudo-code
Ovale Début / Fin	Début / Fin
Parallélogramme (entrée/sortie)	Lire(variable) ou Écrire(expression)
Rectangle (traitement)	Affectation simple : <code>variable ← expression</code> Affectation tableau : <code>T[i] ← val</code> Appel de procédure : <code>Appeler Proc(args)</code>
Losange (test)	Si condition Alors ... Sinon ... FinSi
Boucle (retour de flèche vers un test)	TantQue condition Faire ... FinTantQue OU Pour ... FinPour

MÉTHODE Traduire un algorithme en algorithmme, pas à pas

1. Repérer le point de départ : l'ovale « Début ». Écrire l'en-tête de l'algorithme (nom, déclaration des variables).
2. Suivre les flèches de haut en bas et traduire chaque symbole rencontré, dans l'ordre, en utilisant le dictionnaire de correspondance.
3. À chaque losange, identifier les deux branches : la branche « oui » devient le bloc `Alors`, la branche « non » devient le bloc `Sinon`. Traduire d'abord tout le contenu d'une branche avant de passer à la suite du schéma (là où les deux branches se rejoignent).
4. À chaque boucle, repérer le test qui conditionne le retour en arrière : s'il s'agit d'un compteur connu à l'avance, utiliser `Pour` ; sinon, utiliser `TantQue`. Tout le corps de la boucle est traduit avant l'instruction qui suit la boucle.
5. Indenter systématiquement : le contenu d'un `Alors`, d'un `Sinon`, d'un `Pour` ou d'un `TantQue` est décalé d'un cran vers la droite. C'est l'indentation qui remplace visuellement les flèches du dessin.
6. Vérifier : relire le pseudo-code obtenu et le dérouler mentalement sur un exemple ; comparer avec la table de trace de l'algorithme. Les deux doivent donner le même résultat.

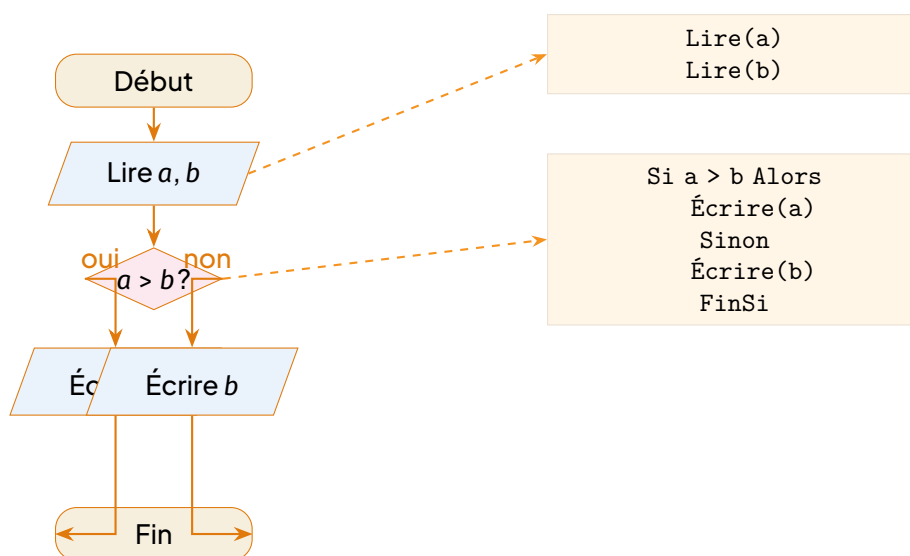
ATTENTION Pièges de traduction

Trois erreurs reviennent sans cesse chez les élèves :

- Inverser les branches du losange : la branche « oui » doit correspondre au **Alors**. Relis toujours la condition écrite dans le losange.
- Oublier le **Sinon** quand le losange n'a qu'une branche utile : dans ce cas, on écrit simplement **Si ... Alors ... FinSi**, sans **Sinon**. Ne jamais écrire un **Sinon** vide.
- Négliger l'indentation : un pseudo-code sans décalages est illisible et cache des erreurs de structure (par exemple une instruction placée hors de la boucle alors qu'elle devait y être).

4.2 Exemple guidé complet : le plus grand de deux nombres

Reprenons l'algorithme classique : lire deux nombres a et b , puis afficher le plus grand des deux. La figure ci-dessous montre, en regard, l'algorithme et sa traduction, avec les flèches de correspondance.



Correspondance symbole par symbole : à gauche l'algorithme, à droite le pseudo-code. Le losange devient un **Si...Alors...Sinon...FinSi** ; les deux parallélogrammes de sortie deviennent les deux **Écrire** indentés.

L'algorithme complet s'écrit donc :

```

Algorithme PlusGrand
Variable a, b : Reel
Debut
    Lire(a)
    Lire(b)
    Si a > b Alors
  
```

```

    Ecrire("Le plus grand est : ", a)
Sinon
    Ecrire("Le plus grand est : ", b)
FinSi
Fin

```

Observe bien : l'indentation des deux `Ecrire` montre clairement qu'ils appartiennent chacun à une branche du test, exactement comme les deux parallélogrammes pendaient chacun à une branche du losange.

4.3 Compter les instructions d'un algorithme

Pourquoi compter ?

Deux algorithmes peuvent résoudre le même problème, mais l'un peut être beaucoup plus coûteux que l'autre : il effectue plus d'opérations, donc il prend plus de temps. Pour comparer objectivement deux solutions, on commence par un outil simple et rigoureux : le **comptage d'instructions**.

DÉFINITION Instruction élémentaire

On appelle **instruction** toute opération élémentaire d'un algorithme : une **affectation**, une **lecture** (`Lire`), une **écriture** (`Ecrire`) ou l'**évaluation d'un test** (condition d'un `Si`, d'un `TantQue` ou d'un `Pour`). Chacune compte pour une instruction.

Cas 1 : une séquence simple.

Dans une suite d'instructions sans test ni boucle, le nombre total d'instructions est simplement la somme des instructions. Par exemple :

```

Algorithme Perimetre
Variable L, l, p : Reel
Debut
    Lire(L)           // instruction 1
    Lire(l)           // instruction 2
    p <- 2 * (L + l)   // instruction 3 (une affectation)
    Ecrire(p)         // instruction 4
Fin

```

Cet algorithme exécute exactement 4 instructions, quelles que soient les valeurs saisies.

Cas 2 : une boucle Pour de n itérations.

C'est là que le comptage devient intéressant. Considérons une boucle :

```

Pour i allant de 1 a n Faire
    <k instructions du corps>
FinPour

```

Le corps de la boucle est répété n fois : il contribue donc pour $n \times k$ instructions. Mais il ne faut pas oublier le test de la boucle, ni l'initialisation du compteur.

PROPRIÉTÉ Comptage dans une boucle Pour

Dans une boucle `Pour i allant de 1 à n` dont le corps contient k instructions :

- le corps s'exécute exactement $n \times k$ instructions ;
- le test « $i \leq n$? » est évalué $n + 1$ fois (n fois vrai, puis une dernière fois faux pour sortir) ;
- l'initialisation $i \leftarrow 1$ compte pour 1 instruction.

Le coût total de la boucle est donc : $1 + (n + 1) + n \times k$ instructions.

Esquisse de démonstration (par récurrence sur n). Notons $C(n)$ le nombre d'exécutions du corps pour une boucle de n tours.

- Initialisation : si $n = 0$, la boucle ne s'exécute jamais, donc $C(0) = 0 = 0 \times k$. La propriété est vraie au rang 0.
- Hérédité : supposons que pour une boucle de n tours, le corps s'exécute $n \times k$ instructions. Une boucle de $n + 1$ tours fait un tour de plus, c'est-à-dire k instructions supplémentaires :

$$C(n + 1) = C(n) + k = n \times k + k = (n + 1) \times k.$$

La propriété est donc vraie au rang $n + 1$.

- Conclusion : par récurrence, pour tout $n \in \mathbb{N}$, le corps s'exécute exactement $n \times k$ instructions. ■

ATTENTION Le test de boucle est évalué $n + 1$ fois, pas une seule !

L'erreur la plus fréquente au Probatoire est de compter le test d'une boucle une seule fois, ou n fois. En réalité, pour une boucle de n tours, la condition est évaluée $n + 1$ fois : n évaluations « vraies » (qui déclenchent chacune un tour) puis une dernière évaluation « fausse » qui fait sortir de la boucle. Cette dernière évaluation existe toujours, même si elle ne déclenche rien. De même pour un `TantQue` : si le corps s'exécute n fois, le test a été évalué $n + 1$ fois.

EXEMPLE Comptage exact : moyenne de n notes

Un professeur de Première TI au lycée de Nkongsamba veut calculer la moyenne des notes de ses élèves. Voici l'algorithme, annoté avec le nombre d'exécutions de chaque ligne :

```
Algorithme MoyenneNotes
Variable n, i : Entier
Variable note, somme, moyenne : Reel
Debut
    Lire(n)                // 1 instruction
    somme ← 0                // 1 instruction
    Pour i allant de 1 à n Faire // init : 1 ; test : n+1
```

```

Lire(note)           // n instructions
somme ← somme + note // n instructions
FinPour
moyenne ← somme / n   // 1 instruction
Ecrire(moyenne)      // 1 instruction
Fin

```

Total des instructions :

$$T(n) = \underbrace{1}_{\text{Lire}(n)} + \underbrace{1}_{\text{somme} \leftarrow 0} + \underbrace{1}_{\text{init } i} + \underbrace{(n+1)}_{\text{tests}} + \underbrace{2n}_{\text{corps}} + \underbrace{1}_{\text{moyenne}} + \underbrace{1}_{\text{Ecrire}}$$

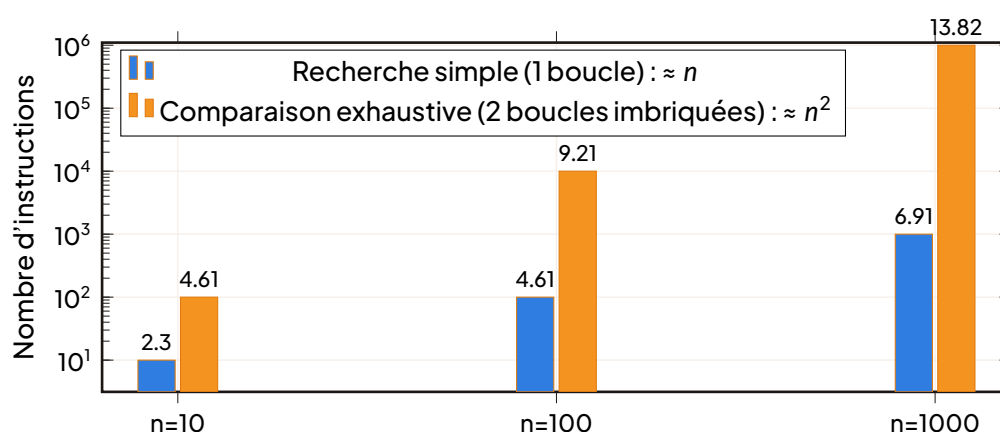
$$= 3n + 6.$$

Pour $n = 50$ élèves, l'algorithme exécute $3 \times 50 + 6 = 156$ instructions. On remarque que le nombre d'instructions croît proportionnellement à n : on dit que le coût est linéaire.

4.4 Notion intuitive de coût d'un algorithme

Le comptage exact est utile, mais ce qui intéresse surtout l'informaticien, c'est la façon dont le nombre d'instructions augmente quand la taille n des données grandit. C'est la notion de **coût** (ou complexité) d'un algorithme — un complément précieux pour le Bac et la Terminale.

- Un algorithme avec **une seule boucle** de n tours effectue un nombre d'instructions de l'ordre de n (coût linéaire) : si n est multiplié par 10, le travail est multiplié par 10.
- Un algorithme avec **deux boucles imbriquées** de n tours chacune effectue de l'ordre de $n \times n = n^2$ instructions (coût quadratique) : si n est multiplié par 10, le travail est multiplié par 100 ! Pour $n = 1000$, cela fait environ un million d'instructions au lieu de mille.



Comparaison du nombre d'instructions de deux algorithmes de recherche dans un tableau de n éléments (échelle logarithmique) : la recherche simple parcourt le tableau une fois ($\approx n$ instructions), tandis que l'algorithme qui compare chaque élément à tous les autres (deux boucles imbriquées) explose : $\approx n^2$ instructions.

LESAVAIS-TU ? Pourquoi ce décompte compte vraiment

Un processeur moderne exécute plusieurs milliards d'instructions par seconde : pour $n = 1000$, un algorithme en n^2 (un million d'instructions) reste instantané. Mais les applications réelles manipulent des millions de données : pour $n = 10^6$, un algorithme en n^2 demande 10^{12} instructions, soit plusieurs minutes, alors qu'un algorithme en n reste immédiat. C'est pour cela que les ingénieurs — y compris ceux qui gèrent les grands systèmes de facturation de la CAMTEL ou les fichiers d'élèves du MINESEC — choisissent leurs algorithmes avec soin. Tu approfondiras cette idée en Terminale avec la notion de complexité.

EXERCICE RÉSOLU 4 Traduction puis comptage : la somme des entiers pairs

Énoncé. Voici l'algorithme d'un algorithme : Début $\rightarrow$ Lire $n \rightarrow s \leftarrow 0 \rightarrow$ Pour i de 1 à n : Si i est pair Alors $s \leftarrow s + i$ FinSi FinPour $\rightarrow$ Écrire $s \rightarrow$ Fin.

1. Traduire cet algorithme en pseudo-code.
2. Compter le nombre total d'instructions exécutées en fonction de n .

SOLUTION

Solution. 1. Traduction. On lit le schéma de haut en bas ; le losange « i pair ? » devient un Si...Alors...FinSi (pas de Sinon, car la branche « non » ne fait rien) :

```

Algorithme SommePairs
Variable n, i, s : Entier
Debut
  Lire(n)
  s  $\leftarrow$  0
  Pour i allant de 1 à n Faire
    Si i MOD 2 = 0 Alors
      s  $\leftarrow$  s + i
    FinSi
  FinPour
  Ecrire(s)
Fin
  
```

2. Comptage.

- Avant la boucle : Lire(n) (1) + $s \leftarrow 0$ (1) = 2 instructions.
- Boucle Pour : initialisation de i (1) + test évalué $n + 1$ fois.
- Corps de la boucle, à chaque tour : le test $i \text{ MOD } 2 = 0$ (1 instruction, évaluée à chaque tour) + l'affectation $s \leftarrow s + i$ (exécutée uniquement si i est pair, soit $\lfloor n/2 \rfloor$ fois).
- Après la boucle : Ecrire(s) (1).

Comptage exact (pour n pair, $n = 2p$) :

$$T(n) = 2 + 1 + (n + 1) + n \times 1 + \frac{n}{2} \times 1 + 1 = 2,5n + 5.$$

En majorant (complexité au pire cas), on borne le corps par 2 instructions par tour (test + affectation) :

$$T_{\text{majoré}}(n) = 2 + 1 + (n + 1) + 2n + 1 = 3n + 5.$$

Pour $n = 100$, au plus 305 instructions. Le coût est linéaire : il reste proportionnel à n .

EXERCICE 4 À toi de jouer

On considère l'algorithme suivant, qui affiche la table de multiplication d'un nombre a :

```
Algorithme TableMultiplication
Variable a, i, p : Entier
Debut
  Lire(a)
  Pour i allant de 1 à 10 Faire
    p ← a * i
    Ecrire(a, " x ", i, " = ", p)
  FinPour
Fin
```

1. Dessiner l'algorithme correspondant à cet algorithme.
2. Compter le nombre exact d'instructions exécutées (la boucle fait ici un nombre fixe de tours).
3. Combien d'instructions exécuterait une version où la table irait de 1 à n , avec n lu au clavier ?

CORRIGÉ Exercice : Table de multiplication

1. Algorithme : Début → parallélogramme « Lire a » → rectangle « $i \leftarrow 1$ » → losange « $i \leq 10 ?$ » ; branche « oui » : rectangle « $p \leftarrow a \times i$ » → parallélogramme « Écrire p » → rectangle « $i \leftarrow i + 1$ » → retour au losange ; branche « non » → Fin. (Représentation équivalente TantQue de la boucle Pour).
2. Comptage : Lire(a) : 1 ; initialisation $i \leftarrow 1$: 1 ; test évalué $10 + 1 = 11$ fois ; corps (2 instructions : affectation + écriture) exécuté 10 fois : 20. Total : $1 + 1 + 11 + 20 = 33$ instructions.
3. Version de 1 à n : $1 + 1 + (n + 1) + 2n = 3n + 3$ instructions. Le coût est linéaire en n .

À RETENIR L'essentiel de la section

- Traduire un algorithme en algorithme se fait de haut en bas, symbole par symbole : parallélogramme → Lire/Ecrire, rectangle → affectation (simple, tableau, ou appel procédure), losange → Si...Alors...Sinon...FinSi, boucle → Pour ou TantQue.

L'indentation remplace les flèches.

- Une instruction = une affectation, une lecture, une écriture ou l'évaluation d'un test.
- Séquence simple : on additionne les instructions. Boucle de n tours avec k instructions dans le corps : $n \times k$ instructions, plus l'initialisation (1) et le test évalué $n + 1$ fois.
- Le coût d'un algorithme décrit la croissance du nombre d'instructions avec la taille n des données : une boucle donne un coût linéaire ($\approx n$), deux boucles imbriquées un coût quadratique ($\approx n^2$), bien plus élevé.

5 EXÉCUTION PAS À PAS : STRUCTURES DE DONNÉES ET SOUS-ALGORITHMES

Dans la section précédente, nous avons appris à transformer un algorithme en algorithme et à compter les instructions. Nous savons donc déjà lire et dérouler des algorithmes simples : une suite d'affectations, un test, une boucle. Mais les problèmes réels — gérer les notes d'une classe de Première TI, les produits d'une boutique, les clients d'un cybercafé — exigent des outils plus puissants : les **tableaux** (pour stocker plusieurs valeurs) et les **sous-algorithmes** (pour découper un gros problème en petits morceaux réutilisables). Cette section te donne la méthode pour exécuter à la main, ligne par ligne, de tels algorithmes. C'est une compétence décisive au Probatoire : le correcteur te demandera souvent de fournir la **table de trace** complète d'un algorithme donné.

5.1 Exécution manuelle d'un algorithme avec tableau

Pourquoi c'est délicat. Avec des variables simples, la trace est facile : une colonne par variable suffit. Avec un tableau, deux choses évoluent en même temps : l'**indice** (la position dans le tableau) et le **contenu** des cases. Toute la difficulté — et toutes les erreurs d'examen — viennent de la confusion entre les deux.

DÉFINITION Tableau et indice

Un **tableau** est une structure de données qui regroupe, sous un même nom, plusieurs valeurs de même type, rangées dans des cases numérotées. Le numéro d'une case s'appelle l'**indice**. Si T est un tableau de n cases numérotées de 1 à n , alors $T[i]$ désigne la valeur contenue dans la case numéro i . L'indice i est une position ; $T[i]$ est un contenu.

ATTENTION Indice ou contenu ? L'erreur classique

L'erreur la plus fréquente consiste à confondre l'**indice** d'une case et la **valeur** qu'elle

contient. Si $T = [15, 8, 12, 3]$:

- $T[2]$ vaut 8 (le contenu de la case 2), et non 2 ;
- écrire $T[T[2]]$ revient à écrire $T[8]$, ce qui est hors limites pour un tableau de 4 cases : bug garanti ;
- dans une boucle Pour i de 1 à n , c'est i qui varie ; la case visitée est $T[i]$. Ne jamais écrire $T[i] \leftarrow i$ en croyant « remplir le tableau avec ses valeurs » : on y rangerait les positions 1, 2, 3, ... et non les données.

Dans ta table de trace, prévois toujours une colonne pour l'indice i et une colonne (ou une ligne) pour le contenu des cases concernées.

Méthode de parcours case par case. Pour exécuter à la main un algorithme qui parcourt un tableau, on suit systématiquement trois repères à chaque tour de boucle : (1) la valeur courante de l'indice i ; (2) la valeur lue $T[i]$; (3) les variables de travail (compteur, somme, maximum...) mises à jour en conséquence. On s'arrête dès que la condition de boucle n'est plus satisfaite, et on vérifie que le nombre de tours est bien celui attendu (pour Pour i de 1 à n : exactement n tours).

5.2 Trace d'une recherche dans un tableau

EXEMPLE Recherche de la note maximale dans un tableau de 5 notes

Le professeur d'informatique a saisi les 5 notes d'un devoir : $Notes = [12, 17, 9, 15, 14]$. On cherche la note maximale.

```

Algorithme Maximum
Variables
  Notes : tableau[1..5] de Reel
  i, max : Entier
Debut
  max ← Notes[1]      // on suppose le 1er element maximal
  Pour i de 2 à 5 Faire
    Si Notes[i] > max Alors
      max ← Notes[i]
    FinSi
  FinPour
  Ecrire("Note maximale : ", max)
Fin
  
```

Table de trace complète :

Tour	i	Notes[i]	max (après le test)
initialisation	—	—	12
1	2	17	17 (car 17 > 12)
2	3	9	17 (car 9 < 17)
3	4	15	17 (car 15 < 17)
4	5	14	17 (car 14 < 17)

L'algorithme affiche : Note maximale : 17. On compte 1 affectation initiale + 4 tours de boucle (chacun : 1 comparaison, parfois 1 affectation).

ATTENTION Initialiser le maximum

N'initialise jamais `max` à 0 « par réflexe » : si toutes les valeurs du tableau sont négatives (par exemple des températures), le résultat serait faux. La bonne pratique est `max ← T[1]`, puis on démarre la boucle à l'indice 2. Même remarque pour la recherche d'une valeur : initialise un booléen `trouve ← Faux` avant la boucle, et non après.

5.3 Trace d'un tri simple : le tri par sélection

LESAVAIS-TU ? Complément utile pour le Bac

Le tri par sélection dépasse légèrement le strict minimum du programme de Première, mais il revient régulièrement au Probatoire et au Baccalauréat TI sous forme de trace à compléter. Le maîtriser maintenant, c'est prendre de l'avance — et c'est un excellent entraînement à la manipulation des indices.

Idée du tri par sélection. On parcourt le tableau de gauche à droite. À l'étape i , on cherche le plus petit élément de la zone non encore triée (cases i à n) et on l'échange avec l'élément de la case i . Ainsi, après l'étape i , les i premières cases contiennent les i plus petites valeurs, dans l'ordre.

Algorithme TriSelection

Variables

T : tableau[1..4] de Entier

i, j, posMin, tmp : Entier

Debut

Pour i de 1 à 3 Faire

posMin ← i

Pour j de i+1 à 4 Faire

Si T[j] < T[posMin] Alors

posMin ← j

FinSi

```

    FinPour
    // échange de T[i] et T[posMin]
    tmp ← T[i]
    T[i] ← T[posMin]
    T[posMin] ← tmp
  FinPour
Fin

```

Étape	Case 1	Case 2	Case 3	Case 4	Action réalisée
Départ	15	8	12	3	tableau initial
$i = 1$	3	8	12	15	min de [15,8,12,3] = 3 (case 4); échange cases 1 et 4
$i = 2$	3	8	12	15	min de [8,12,15] = 8 (case 2); échange case 2 avec elle-même
$i = 3$	3	8	12	15	min de [12,15] = 12 (case 3); échange case 3 avec elle-même
Fin	3	8	12	15	tableau trié

Table de trace du tri par sélection appliqué à [15, 8, 12, 3] : état du tableau après chaque étape (les cases échangées sont en gras).

Observe deux points essentiels : la boucle externe s'arrête à $n-1$ (la dernière case est forcément en place quand toutes les autres le sont), et l'échange exige une variable temporaire `tmp` : écrire directement `T[i] ← T[posMin]` puis `T[posMin] ← T[i]` écraserait la première valeur avant de l'avoir sauvegardée.

5.4 Exécution d'un algorithme avec sous-algorithme

Le problème à résoudre. Imagine un algorithme de gestion de notes qui calcule des moyennes, des carrés d'écart, des maximums à plusieurs endroits. Recopier le même bloc d'instructions à chaque fois est long et dangereux (une correction oubliée à un endroit, et le programme devient faux). La solution : écrire le bloc une seule fois sous forme de **sous-algorithme** (fonction ou procédure), puis l'appeler chaque fois qu'on en a besoin.

DÉFINITION Fonction, paramètres, valeur retournée

Une **fonction** est un sous-algorithme nommé qui reçoit des valeurs en entrée (les **paramètres**), exécute un bloc d'instructions, et retourne un résultat au programme appelant grâce au mot-clé `Retourner`. Les variables déclarées à l'intérieur de la fonction sont dites **locales** : elles n'existent que pendant l'exécution de la fonction et sont invisibles depuis le programme principal.

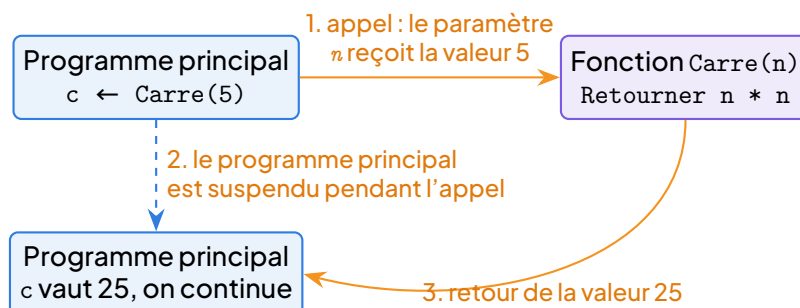
DÉFINITION Procédure

Une **procédure** est un sous-algorithme nommé qui reçoit des paramètres, exécute un bloc d'instructions, mais ne retourne pas de valeur (pas de `Retourner`). Elle produit un effet de bord (affichage, modification de paramètres passés par référence, écriture dans un fichier). En Première TI, on privilégie les fonctions pour la clarté de la trace.

MÉTHODE Tracer un appel de fonction en trois temps

Pour exécuter à la main un programme qui appelle une fonction :

1. Recopier les paramètres. Au moment de l'appel, chaque paramètre formel de la fonction reçoit la valeur de l'argument correspondant, dans l'ordre (le 1er argument va dans le 1er paramètre, etc.).
2. Exécuter le corps à part. On ouvre une zone de trace séparée pour la fonction : on y déroule ses instructions avec ses variables locales, comme s'il s'agissait d'un petit algorithme indépendant, jusqu'au `Retourner`.
3. Reporter la valeur retournée. On revient au programme principal, à l'endroit exact de l'appel : l'expression d'appel est remplacée par la valeur retournée, et l'exécution reprend son cours.



Mécanisme d'appel d'une fonction : le programme principal transmet les arguments, la fonction s'exécute avec ses variables locales, puis la valeur retournée reprend la place de l'appel.

EXERCICE RÉSOLU 5 Trace complète d'un programme appelant une fonction Maximum

Énoncé. On donne la fonction et le programme principal suivants :

Fonction Max2(a : Entier, b : Entier) : Entier
Variables

```

    m : Entier          // variable LOCALE a la fonction
Debut
    Si a > b Alors
        m ← a
    Sinon
        m ← b
    FinSi
    Retourner m
Fin

Algorithme Principal
Variables
    x, y, grand : Entier // variables du programme principal
Debut
    x ← 7
    y ← 12
    grand ← Max2(x, y)
    grand ← Max2(grand, 20)
    Ecrire("Le plus grand est : ", grand)
Fin

```

Donner la table de trace complète et l’affichage obtenu.

SOLUTION

Solution détaillée. On trace d’abord le programme principal, en ouvrant une sous-trace à chaque appel.

Programme principal :

Ligne	Action	État des variables (x, y, grand)
1	$x \leftarrow 7$	7, ?, ?
2	$y \leftarrow 12$	7, 12, ?
3	appel $\text{Max2}(7, 12)$	voir sous-trace 1
4	$\text{grand} \leftarrow 12$	7, 12, 12
5	appel $\text{Max2}(12, 20)$	voir sous-trace 2
6	$\text{grand} \leftarrow 20$	7, 12, 20
7	affichage	Le plus grand est : 20

Sous-trace 1 — appel $\text{Max2}(7, 12)$: les paramètres reçoivent les valeurs dans l’ordre : $a = 7$, $b = 12$. Test : $7 > 12$? Faux, donc $m \leftarrow 12$. La fonction retourne 12. La variable locale m disparaît.

Sous-trace 2 — appel $\text{Max2}(12, 20)$: $a = 12$, $b = 20$. Test : $12 > 20$? Faux, donc $m \leftarrow 20$. La fonction retourne 20.

Affichage final : Le plus grand est : 20.

ATTENTION Variables locales et variables du programme principal

Dans une trace, ne mélange jamais les deux mondes :

- la variable locale `m` de `Max2` n'existe pas dans le programme principal : écrire `Ecrire(m)` dans `Principal` est une erreur ;
- réciproquement, la fonction ne voit pas `x`, `y`, `grand` : elle ne travaille que sur ses paramètres `a`, `b` et ses locales ;
- deux variables peuvent porter le même nom dans les deux mondes sans être la même variable : dans ta table de trace, fais deux zones bien séparées ;
- enfin, l'ordre des arguments compte : `Max2(12, 7)` et `Max2(7, 12)` donnent ici le même résultat par chance, mais pour une fonction `Puissance(base, exposant)`, inverser les arguments change tout.

5.5 Fonctions récursives simples

LE SAVAIS-TU ? La récursivité, une idée puissante

La récursivité consiste, pour une fonction, à s'appeler elle-même. C'est l'outil naturel pour résoudre des problèmes qui se décomposent en sous-problèmes de même nature (factorielle, parcours d'arbre, recherche dichotomique). Au Probatoire, on te demandera souvent de tracer l'exécution d'une fonction récursive simple (factorielle, puissance, somme des n premiers entiers) pour vérifier que tu comprends le mécanisme d'empilement des appels (la **pile d'exécution**).

DÉFINITION Fonction récursive, cas de base, pas récursif

Une **fonction récursive** est une fonction qui s'appelle elle-même dans son propre corps. Elle doit obligatoirement comporter deux parties :

1. Un **cas de base** (ou condition d'arrêt) : une condition simple pour laquelle la fonction retourne un résultat direct sans s'appeler elle-même. Sans cas de base, la fonction s'appelle indéfiniment : c'est une récursion infinie (erreur fatale : `Stack Overflow`).
2. Un **pas récursif** : l'appel à la fonction elle-même avec un argument « plus simple » (plus petit, plus court), qui rapproche du cas de base.

EXEMPLE Factorielle : $n! = n \times (n - 1)!$

La factorielle est l'exemple classique : $0! = 1$ (cas de base) et $n! = n \times (n - 1)!$ pour $n > 0$ (pas

récuratif).

```

Fonction Factorielle(n : Entier) : Entier
Debut
    Si n = 0 Alors          // CAS DE BASE
        Retourner 1
    Sinon                  // PAS RECURSIF
        Retourner n * Factorielle(n - 1)
    FinSi
Fin

```

Trace de `Factorielle(3)` : On utilise une pile d'appels (représentée verticalement, le dernier arrivé en haut). Chaque ligne correspond à un appel suspendu en attente du résultat de l'appel suivant.

Étape	Action	Pile d'appels (en attente) / Résultat
1	Appel <code>Factorielle(3)</code>	<code>Factorielle(3)</code> attend $3 \times \dots$
2	Appel <code>Factorielle(2)</code>	<code>Factorielle(2)</code> attend $2 \times \dots$
3	Appel <code>Factorielle(1)</code>	<code>Factorielle(1)</code> attend $1 \times \dots$
4	Appel <code>Factorielle(0)</code>	Cas de base atteint : retourne 1
5	Retour vers <code>Factorielle(1)</code>	Calcule $1 \times 1 = 1$, retourne 1
6	Retour vers <code>Factorielle(2)</code>	Calcule $2 \times 1 = 2$, retourne 2
7	Retour vers <code>Factorielle(3)</code>	Calcule $3 \times 2 = 6$, retourne 6

Résultat final : 6. On voit bien le mécanisme « aller » (empilement jusqu'au cas de base) puis « retour » (dépilement et calculs).

MÉTHODE Tracer une fonction réursive à la main

1. Phase « Aller » (Empilement). Écris les appels successifs en descendant : $F(n)$, $F(n-1)$, $F(n-2)$, ... jusqu'au cas de base. Note à côté de chaque appel l'opération « en attente » (ex : $\times n$, $\times(n-1)$).
2. Cas de base. Écris la valeur retournée immédiatement (ex : 1 pour factorielle).
3. Phase « Retour » (Dépilement). Remonte d'une ligne : remplace l'appel par le calcul de l'opération en attente avec la valeur venue du bas. Continue jusqu'en haut.
4. La valeur tout en haut est le résultat final.

ATTENTION Pièges classiques de la récursivité

- Oublier le cas de base ou écrire une condition qui ne devient jamais vraie (ex : Si $n < 0$ au lieu de Si $n = 0$ pour une factorielle appelée avec $n \geq 0$) → boucle infinie, plantage mémoire (Stack Overflow).
- Ne pas réduire le problème dans l'appel récursif (ex : Factorielle(n) au lieu de Factorielle($n-1$)) → boucle infinie.
- Confondre la variable n de chaque niveau. Chaque appel a sa propre copie de n (variable locale). Sur la pile, n vaut 3, puis 2, puis 1, puis 0 simultanément dans des cadres différents.
- Compter les instructions. Pour Factorielle(n), il y a $n + 1$ appels, n multiplications, n tests. La complexité en temps est linéaire $O(n)$.

EXERCICE RÉSOLU 6 Trace de la fonction Puissance

Énoncé. Tracer l'exécution de Puissance(2, 3) avec la fonction ci-dessous (calcule x^y pour $y \geq 0$).

```

Fonction Puissance(x : Reel, y : Entier) : Reel
Debut
    Si y = 0 Alors
        Retourner 1
    Sinon
        Retourner x * Puissance(x, y - 1)
    FinSi
Fin

```

SOLUTION

Solution.

Étape	Appel	Opération en attente / Résultat
1	Puissance(2, 3)	attend $2 \times \dots$
2	Puissance(2, 2)	attend $2 \times \dots$
3	Puissance(2, 1)	attend $2 \times \dots$
4	Puissance(2, 0)	Cas de base : retourne 1
5	Retour vers (2, 1)	$2 \times 1 = 2$
6	Retour vers (2, 2)	$2 \times 2 = 4$
7	Retour vers (2, 3)	$2 \times 4 = 8$

Résultat : 8 (soit 2^3).

LESAVAIS-TU ? Le débogage à la main, une tradition d'excellence

Avant même que les ordinateurs soient accessibles, les pionniers de l'informatique — comme Alan Turing ou les programmeuses de l'ENIAC — exécutaient leurs algorithmes sur papier, ligne par ligne, pour en vérifier la correction. Cette pratique, appelée trace d'exécution ou dry run en anglais, reste aujourd'hui l'une des méthodes les plus efficaces pour déboguer les erreurs : la plupart des bugs (mauvaise initialisation, boucle qui s'arrête un tour trop tôt, indice hors limites, récursion infinie) sautent aux yeux dès qu'on déroule l'algorithme à la main. Les grands ateliers de développement, y compris ceux qui fleurissent à Douala et Yaoundé, continuent d'enseigner cette discipline : un bon programmeur teste d'abord dans sa tête et sur sa feuille, ensuite seulement sur la machine.

EXERCICE 5 À toi de jouer

Soit le tableau $T = [4, 11, 6, 9, 2]$ et l'algorithme suivant :

```

Algorithme Comptage
Variables
  T : tableau[1..5] de Entier
  i, c : Entier
Debut
  c ← 0
  Pour i de 1 à 5 Faire
    Si T[i] > 5 Alors
      c ← c + 1
    FinSi
  FinPour
  Ecrire(c)
Fin

```

1. Construis la table de trace complète (colonnes : tour, i, T[i], c).
2. Que calcule cet algorithme ? Que vaut c à la fin ?
3. Écris une fonction `EstGrand(v : Entier) : Booleen` qui retourne Vrai si $v > 5$, puis ré-écris la boucle en l'utilisant.

EXERCICE 6 Récursivité : Somme des entiers

On considère la fonction récursive suivante :

```

Fonction Somme(n : Entier) : Entier
Debut
  Si n = 1 Alors
    Retourner 1
  Sinon
    Retourner n + Somme(n - 1)
  FinSi

```

Fin

1. Quel est le rôle de cette fonction ? (Donne la formule mathématique).
2. Trace l'exécution de `Somme(4)` en utilisant la méthode de la pile (tableau aller/retour).
3. Combien d'appels à `Somme` sont effectués pour `Somme(4)` ? Combien d'additions ?
4. Que se passe-t-il si on appelle `Somme(0)` ? Pourquoi ? Comment corriger le cas de base pour accepter $n = 0$?

CORRIGÉ Exercice : Comptage

1. Table de trace :

Tour	i	T[i]	c
—	—	—	0
1	1	4	0
2	2	11	1
3	3	6	2
4	4	9	3
5	5	2	3

2. L'algorithme compte les éléments strictement supérieurs à 5. Il affiche 3 (les valeurs 11, 6 et 9). 3.

```

Fonction EstGrand(v : Entier) : Booleen
Debut
    Retourner (v > 5)
Fin
// boucle reecrite :
Pour i de 1 a 5 Faire
    Si EstGrand(T[i]) Alors
        c ← c + 1
    FinSi
FinPour

```

CORRIGÉ Exercice : Récursivité – Somme des entiers

1. La fonction calcule la somme des n premiers entiers naturels non nuls : $S(n) = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$.
2. Trace de `Somme(4)` :

Étape	Appel	Opération en attente / Résultat
1	Somme(4)	attend 4 + ...
2	Somme(3)	attend 3 + ...
3	Somme(2)	attend 2 + ...
4	Somme(1)	Cas de base : retourne 1
5	Retour vers Somme(2)	$2 + 1 = 3$
6	Retour vers Somme(3)	$3 + 3 = 6$
7	Retour vers Somme(4)	$4 + 6 = 10$

Résultat : 10 (vérification : $4 \times 5/2 = 10$). 3. 4 appels (Somme(4), Somme(3), Somme(2), Somme(1)). 3 additions (lors du retour). 4. Si on appelle Somme(0) : la condition $n = 1$ est fausse, on passe au Sinon : $0 + \text{Somme}(-1)$, puis $-1 + \text{Somme}(-2)$, etc. Récursion infinie (débordement de pile). Pour corriger : changer le cas de base en Si $n \leq 0$ Alors Retourner 0 (ou Si $n = 0$ Alors Retourner 0 si on garantit $n \geq 0$).

À RETENIR L'essentiel de la section

- Dans un tableau, l'**indice** est une position, $T[i]$ est un contenu : la trace doit suivre les deux séparément.
- Toute recherche (maximum, comptage) s'initialise correctement avant la boucle, puis se déroule case par case.
- Le tri par sélection échange, à chaque étape, le minimum de la zone non triée avec la première case de cette zone ; l'échange exige une variable temporaire.
- Un appel de fonction se trace en trois temps : recopier les paramètres, exécuter le corps à part avec ses variables locales, reporter la valeur retournée.
- Variables locales et variables du programme principal vivent dans deux mondes distincts : ta table de trace doit les séparer clairement.
- Une fonction récursive nécessite un **cas de base** (arrêt) et un **pas récursif** (réduction du problème). On la trace par empilement (phase aller) puis dépilement (phase retour).
- Attention aux récursions infinies (cas de base manquant ou argument non réduit) : erreur Stack Overflow.

6 FONCTIONS RÉCURSIVES SIMPLES

Dans la section précédente, nous avons appris à exécuter pas à pas des algorithmes qui font appel à des sous-algorithmes : une fonction `Carre(x)` appelée depuis un programme principal, par exemple. Nous avons vu que chaque appel « met en pause » le programme appelant, exécute le sous-algorithme, puis revient avec un résultat. Mais que se passerait-il si une fonction s'appelait... elle-même ? Cette idée, qui semble étrange au premier abord, est en réalité l'une des plus puissantes de toute l'informatique. C'est l'objet de cette dernière section : les **fonctions récursives**.

6.1 Intuition : le problème qui se résout en se réduisant

Imagine la scène suivante dans un lycée de Yaoundé : le proviseur veut connaître le nombre total d'élèves présents dans une file d'attente à la cantine. Plutôt que de tous les compter lui-même, il demande au premier élève de la file : « Combien êtes-vous à partir de toi ? ». Cet élève ne sait pas répondre directement, alors il se retourne et pose la même question à l'élève derrière lui, et ainsi de suite jusqu'au dernier élève de la file. Celui-ci, n'ayant personne derrière lui, répond : « 1 ». L'élève devant lui ajoute 1 et répond « 2 », puis « 3 », et la réponse remonte ainsi jusqu'au premier élève, qui annonce le total au proviseur.

Cette petite histoire contient toute la **récursivité** :

- chaque élève applique la même méthode que le précédent (demander au suivant et ajouter 1) : c'est l'**appel récursif** ;
- le dernier élève répond immédiatement sans interroger personne : c'est le **cas de base** ;
- les réponses remontent ensuite de proche en proche : c'est la **remontée des résultats**.

De nombreux problèmes mathématiques et informatiques se définissent naturellement ainsi : un problème « grand » se ramène au même problème « plus petit », jusqu'à un cas tellement petit qu'on en connaît directement la réponse.

6.2 Définition et ingrédients obligatoires

DÉFINITION Fonction récursive

Une **fonction récursive** est une fonction qui, dans son propre corps, contient un appel à elle-même. Toute fonction récursive correcte possède obligatoirement deux ingrédients :

1. le **cas de base** (ou condition d'arrêt) : un cas particulier pour lequel la fonction retourne directement un résultat sans s'appeler elle-même ;
2. l'**appel récursif** : un appel de la fonction à elle-même, portant sur un problème strictement plus petit, qui se rapproche du cas de base à chaque étape.

À RETENIR La règle d'or de la récursivité

Une fonction récursive correcte répond toujours à deux questions :

- « Quand est-ce que je m'arrête ? » → le cas de base ;
- « Comment je me rapproche de l'arrêt ? » → l'appel récursif sur une donnée plus petite.

Si l'une de ces deux réponses manque, la fonction ne s'arrêtera jamais.

6.3 Exemple fondateur : la factorielle

En mathématiques, la **factorielle** d'un entier naturel n , notée $n!$, est le produit de tous les entiers de 1 à n :

$$n! = 1 \times 2 \times 3 \times \dots \times n \quad \text{avec, par convention, } 0! = 1.$$

Par exemple, $4! = 1 \times 2 \times 3 \times 4 = 24$.

Observons maintenant une remarque capitale : $4! = 4 \times 3!$, et $3! = 3 \times 2!$, et $2! = 2 \times 1!$, et $1! = 1 \times 0!$. Autrement dit, la factorielle de n se définit à partir de la factorielle de $n - 1$:

$$\text{fact}(n) = n \times \text{fact}(n - 1) \quad \text{et} \quad \text{fact}(0) = 1.$$

Nous tenons nos deux ingrédients : le cas de base est $n = 0$ (on retourne 1 directement), et l'appel récursif porte sur $n - 1$, qui est plus petit que n et rapproche donc de 0. Voici l'algorithme correspondant :

```

Fonction fact(n : Entier) : Entier
Debut
  Si n = 0 Alors
    Retourner 1          // cas de base : 0! = 1
  Sinon
    Retourner n * fact(n - 1) // appel récursif
FinSi
Fin
  
```

PROPRIÉTÉ Correction de la fonction factorielle

Pour tout entier $n \geq 0$, la fonction `fact` ci-dessus retourne bien $n!$. Démonstration guidée par récurrence sur n :

- Initialisation : pour $n = 0$, la fonction entre dans le cas de base et retourne 1. Or $0! = 1$ par convention. La propriété est vraie au rang 0.
- Hérédité : supposons la propriété vraie au rang $n - 1$, c'est-à-dire $\text{fact}(n - 1) = (n - 1)!$ (c'est l'hypothèse de récurrence). Alors, pour $n \geq 1$, la fonction retourne :

$$\text{fact}(n) = n \times \text{fact}(n - 1) = n \times (n - 1)! = n!$$

La propriété est donc vraie au rang n .

- Conclusion : par récurrence, $\text{fact}(n) = n!$ pour tout $n \in \mathbb{N}$.

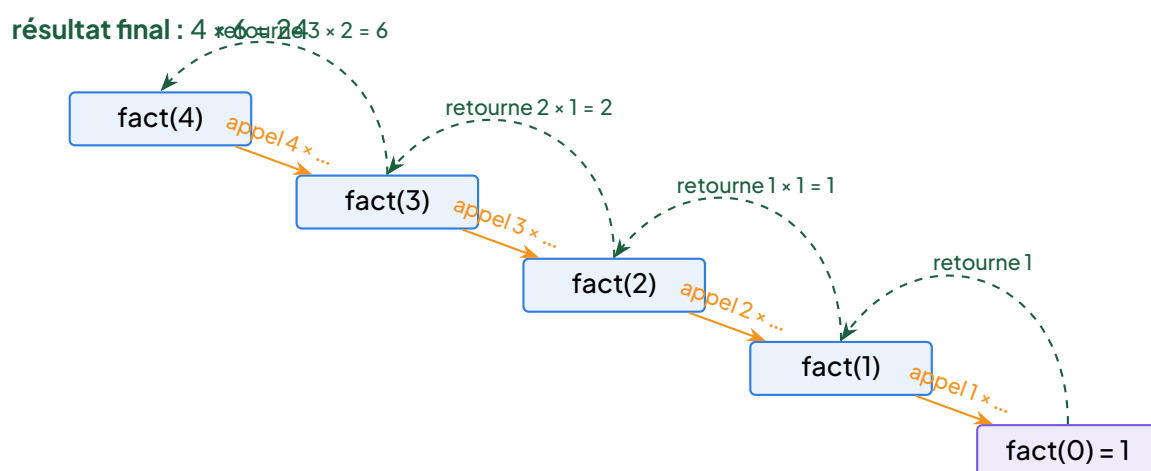
Cette méthode de preuve est la méthode générale pour vérifier qu'une fonction récursive est correcte : on vérifie le cas de base, puis on montre que l'appel récursif, supposé correct sur le problème plus petit, produit le bon résultat sur le problème courant.

6.4 Exécution à la main de $\text{fact}(4)$: descente et remontée

Comment l'ordinateur exécute-t-il réellement $\text{fact}(4)$? Le mécanisme repose sur la **pile d'appels** : à chaque appel de fonction, l'ordinateur empile le contexte de l'appel (la valeur du paramètre, l'endroit où reprendre ensuite) ; quand une fonction retourne un résultat, son contexte est dépilé et l'exécution reprend dans l'appelant.

L'exécution se déroule donc en deux phases :

1. **Descente des appels** : $\text{fact}(4)$ doit calculer $4 \times \text{fact}(3)$; pour cela il appelle $\text{fact}(3)$, qui appelle $\text{fact}(2)$, qui appelle $\text{fact}(1)$, qui appelle $\text{fact}(0)$. Les appels s'empilent.
2. **Cas de base atteint** : $\text{fact}(0)$ retourne 1 sans nouvel appel. La descente est terminée.
3. **Remontée des résultats** : $\text{fact}(1)$ retourne $1 \times 1 = 1$; $\text{fact}(2)$ retourne $2 \times 1 = 2$; $\text{fact}(3)$ retourne $3 \times 2 = 6$; $\text{fact}(4)$ retourne $4 \times 6 = 24$. Les contextes se dépilent un à un.



Descente des appels (flèches pleines orange) puis remontée des résultats (flèches vertes pointillées) pour $\text{fact}(4)$.

MÉTHODE Exécuter une fonction récursive à la main

Pour dérouler correctement une fonction récursive sur papier (compétence exigée au Probatoire), procède toujours ainsi :

1. **Empiler** : écris les appels les uns sous les autres (ou de gauche à droite en escalier), en notant pour chacun la valeur du paramètre et le calcul en attente, jusqu'à atteindre le

cas de base.

2. Vérifier le cas de base : note la valeur retournée par le cas de base.
3. Dépiler : reprends les appels en sens inverse (du dernier empilé au premier), en remplaçant chaque appel récursif par la valeur qu'il a retournée, et effectue le calcul en attente.
4. Conclure : la valeur obtenue au premier appel est le résultat final.

Rédige la trace sous forme de deux colonnes « descente » / « remontée » : c'est la présentation la plus claire et la plus sûre.

6.5 Deuxième exemple : la somme des n premiers entiers

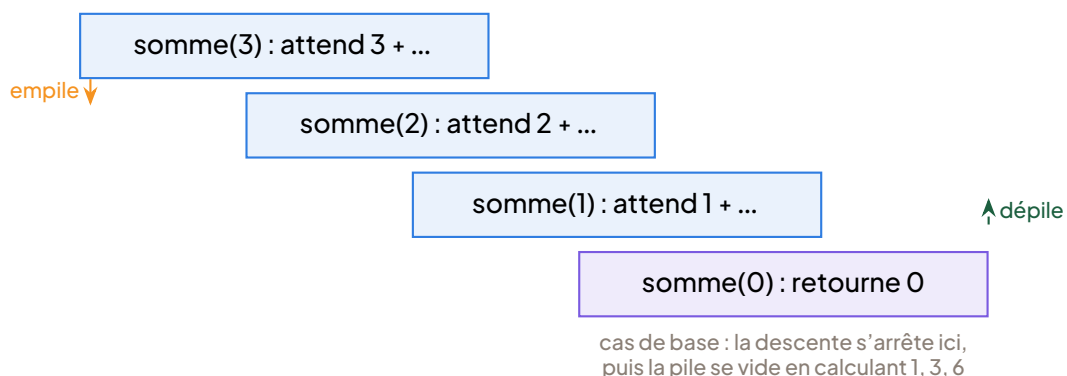
On veut calculer $\text{somme}(n) = 1 + 2 + \dots + n$. Le raisonnement récursif est immédiat : la somme des n premiers entiers, c'est n plus la somme des $n - 1$ premiers ; et la somme des 0 premiers entiers vaut 0.

```

Fonction somme(n : Entier) : Entier
Debut
  Si n = 0 Alors
    Retourner 0          // cas de base
  Sinon
    Retourner n + somme(n - 1) // appel récursif
FinSi
Fin
  
```

Traçons $\text{somme}(3)$. Descente : $\text{somme}(3) = 3 + \text{somme}(2)$, puis $\text{somme}(2) = 2 + \text{somme}(1)$, puis $\text{somme}(1) = 1 + \text{somme}(0)$. Cas de base : $\text{somme}(0) = 0$. Remontée : $\text{somme}(1) = 1 + 0 = 1$; $\text{somme}(2) = 2 + 1 = 3$; $\text{somme}(3) = 3 + 3 = 6$. Résultat : 6, ce qui correspond bien à $1 + 2 + 3$.

État de la pile d'appels (sommet à droite)



Pile d'appels au moment le plus profond de l'exécution de $\text{somme}(3)$: quatre contextes empilés, le cas de base au sommet.

6.6 Troisième exemple : la puissance

Pour calculer x^n (avec $n \geq 0$), on utilise le même schéma : $x^n = x \times x^{n-1}$, et $x^0 = 1$.

```

Fonction puis(x : Reel, n : Entier) : Reel
Debut
  Si n = 0 Alors
    Retourner 1 // cas de base : x^0 = 1
  Sinon
    Retourner x * puis(x, n - 1) // appel récursif
FinSi
Fin

```

EXEMPLE Trace complète de puis(2, 5)

Descente (on empile) :

$\text{puis}(2,5) = 2 \times \text{puis}(2,4)$
 $\text{puis}(2,4) = 2 \times \text{puis}(2,3)$
 $\text{puis}(2,3) = 2 \times \text{puis}(2,2)$
 $\text{puis}(2,2) = 2 \times \text{puis}(2,1)$
 $\text{puis}(2,1) = 2 \times \text{puis}(2,0)$

Cas de base : $\text{puis}(2,0) = 1$.

Remontée (on dépile) : $\text{puis}(2,1) = 2 \times 1 = 2$; $\text{puis}(2,2) = 2 \times 2 = 4$; $\text{puis}(2,3) = 2 \times 4 = 8$;
 $\text{puis}(2,4) = 2 \times 8 = 16$; $\text{puis}(2,5) = 2 \times 16 = 32$.

Résultat final : $\text{puis}(2,5) = 32$, ce qui est bien 2^5 .

6.7 Récursif ou itératif ?

Tout ce qu'on peut écrire en récursif peut aussi s'écrire avec une boucle (version itérative). Voici les deux versions de la factorielle côte à côte :

```

// Version recursive
Fonction factRec(n : Entier) : Entier
Debut
  Si n = 0 Alors
    Retourner 1
  Sinon
    Retourner n * factRec(n - 1)
FinSi
Fin

// Version iterative
Fonction factIter(n : Entier) : Entier
Variable i, res : Entier
Debut
  res <- 1
  Pour i De 1 A n Faire
    res <- res * i
  FinPour

```

Retourner res
Fin

Critère	Version récursive	Version itérative
Lisibilité	Très proche de la définition mathématique	Parfois moins naturelle
Mémoire	Un contexte empilé par appel (n contextes pour $\text{fact}(n)$)	Un nombre fixe de variables
Risque	Récursion infinie si cas de base absent ou mal conçu	Boucle infinie si condition mal écrite

La version récursive est souvent plus élégante et plus facile à prouver (par récurrence), mais chaque appel consomme de la mémoire sur la pile. En Première TI, l'essentiel est de savoir **lire et exécuter** les deux versions, et de reconnaître les situations où la définition récursive est naturelle.

ATTENTION Récursion infinie : l'erreur classique

Les deux erreurs suivantes provoquent une **récursion infinie** : la fonction s'appelle elle-même sans jamais s'arrêter, la pile d'appels déborde et le programme plante (en C ou en JavaScript, on obtient une erreur de type `stack overflow`, « débordement de pile »).

1. Oublier le cas de base : Retourner $n * \text{fact}(n - 1)$ sans le Si $n = 0$: les appels descendent vers $-1, -2, -3, \dots$ sans fin.
2. Ne pas rapprocher l'appel du cas de base : écrire $\text{fact}(n + 1)$ ou $\text{fact}(n)$ au lieu de $\text{fact}(n - 1)$. Le paramètre ne diminue pas, le cas de base n'est jamais atteint.

Avant de valider une fonction récursive, pose-toi toujours les deux questions de la règle d'or : quand est-ce que je m'arrête ? et est-ce que chaque appel me rapproche de l'arrêt ?

EXERCICE RÉSOLU 7 Trace d'une fonction récursive

Énoncé. On considère la fonction suivante :

```

Fonction mystere(n : Entier) : Entier
Debut
    Si n = 1 Alors
        Retourner 1
    Sinon
        Retourner n * n + mystere(n - 1)
    FinSi
Fin

```

1. Identifier le cas de base et l'appel récursif.
2. Exécuter à la main `mystere(4)`.
3. Que calcule cette fonction en général ?

SOLUTION

1. Le cas de base est $n = 1$ (la fonction retourne 1 directement). L'appel récursif est `mystere(n - 1)` : le paramètre diminue de 1 à chaque appel, donc il atteindra bien 1.

2. Descente des appels :

$$\text{mystere}(4) = 4^2 + \text{mystere}(3) = 16 + \text{mystere}(3)$$

$$\text{mystere}(3) = 3^2 + \text{mystere}(2) = 9 + \text{mystere}(2)$$

$$\text{mystere}(2) = 2^2 + \text{mystere}(1) = 4 + \text{mystere}(1)$$

Cas de base : `mystere(1) = 1`. Remontée des résultats :

$$\text{mystere}(2) = 4 + 1 = 5$$

$$\text{mystere}(3) = 9 + 5 = 14$$

$$\text{mystere}(4) = 16 + 14 = 30$$

Le résultat retourné est donc 30.

3. En dépliant la définition, on voit que $\text{mystere}(n) = n^2 + (n - 1)^2 + \dots + 2^2 + 1^2$: la fonction calcule la somme des carrés des n premiers entiers. Vérification : $1 + 4 + 9 + 16 = 30$. ✓

EXERCICE 7 À toi de jouer

1. Écris une fonction récursive `produit(a, b)` qui calcule le produit de deux entiers positifs a et b en n'utilisant que l'addition (indice : $a \times b = a + a \times (b - 1)$). Précise le cas de base.
2. Exécute à la main `produit(3, 4)` en présentant la descente puis la remontée.
3. Que se passe-t-il si l'on appelle `fact(-2)` avec la fonction factorielle du cours ? Explique pourquoi et propose une protection.

CORRIGÉ Exercice : produit récursif

1. Le cas de base est $b = 0$, car $a \times 0 = 0$:

```
Fonction produit(a : Entier, b : Entier) : Entier
Debut
  Si b = 0 Alors
    Retourner 0
  Sinon
    Retourner a + produit(a, b - 1)
FinSi
Fin
```

2. Descente : `produit(3,4) = 3 + produit(3,3)`; `produit(3,3) = 3 + produit(3,2)`; `produit(3,2) = 3 + produit(3,1)`; `produit(3,1) = 3 + produit(3,0)`. Cas de base : `produit(3,0) = 0`. Remontée :

$3 + 0 = 3$; $3 + 3 = 6$; $3 + 6 = 9$; $3 + 9 = 12$. Résultat : $12 = 3 \times 4$. ✓

3. Avec $n = -2$, le test $n = 0$ n'est jamais vérifié et les appels descendent vers $-3, -4, \dots$: récursion infinie. Protection : ajouter au début un test `Si  $n < 0$  Alors` qui signale une erreur (ou retourne une valeur conventionnelle), car la factorielle n'est définie que pour $n \geq 0$.

LE SAVAIS-TU ? La récursivité partout autour de toi

La récursivité n'est pas qu'un exercice de style mathématique : elle est au cœur de nombreux outils que tu utilises. L'explorateur de fichiers de Windows parcourt un dossier en parcourant récursivement ses sous-dossiers ; les moteurs de recherche explorent le Web de lien en lien ; les algorithmes de tri rapide (quicksort), que tu rencontreras dans ta scolarité, trient un tableau en triant récursivement ses deux moitiés. Même les fractales — ces figures géométriques qui se répètent à toutes les échelles, comme le flocon de Koch — sont des objets récursifs. Maîtriser la récursivité, c'est acquérir une nouvelle façon de penser : réduire un grand problème à un petit problème identique.

À RETENIR L'essentiel de la section

- Une **fonction récursive** s'appelle elle-même ; elle exige un cas de base (arrêt) et un appel récursif sur un problème plus petit.
- L'exécution se fait en deux phases : descente des appels (on empile) jusqu'au cas de base, puis remontée des résultats (on dépile en calculant).
- Modèles à connaître par cœur : $\text{fact}(n) = n \times \text{fact}(n - 1)$ avec $\text{fact}(0) = 1$; $\text{somme}(n) = n + \text{somme}(n - 1)$; $\text{puis}(x, n) = x \times \text{puis}(x, n - 1)$.
- Sans cas de base, ou si l'appel ne s'en rapproche pas : récursion infinie et débordement de pile.
- La correction d'une fonction récursive se prouve par récurrence : initialisation sur le cas de base, puis hérédité.

7 ACTIVITÉ D'INTÉGRATION

8 CONSTRUCTION D'ALGORIGRAMMES ET EXÉCUTION DES ALGORITHMES À LA MAIN

8.1 Objectifs de la leçon

À l'issue de cette leçon, tu seras capable de :

- distinguer **algorithme** et **organigramme** et identifier les **symboles normalisés** (terminaux, entrée/sortie, traitement, décision, flux);
- représenter graphiquement les trois **structures de contrôle** fondamentales : séquence, alternative (simple, imbriquée, multiple), boucle (Pour, Tant que);
- dérouler** un algorithme pas à pas sur un jeu d'essai et consigner l'évolution des variables dans une **table de trace** rigoureuse;
- transformer un algorithme en **pseudo-code** (écriture algorithmique classique) et **compiler les instructions élémentaires** exécutées (lectures, écritures, affectations, tests);
- exécuter à la main un algorithme faisant appel à une **structure de données** (tableau, enregistrement) ou à un **sous-algorithme** (procédure, fonction);
- écrire et exécuter une **fonction récursive** simple en identifiant le **cas de base** et le **cas général**, et en simulant la pile d'appels.

8.2 Situation professionnelle : La caisse du lycée

Le secrétariat du **Lycée Bilingue de Bonabéri** (Douala) gère chaque année l'attribution des **aides à l'excellence** financées par la coopérative du personnel. Le règlement intérieur fixe le barème suivant, calculé à partir de la moyenne annuelle m de chaque élève (moyenne sur 20, arrondie au centième) :

Condition sur la moyenne m	Aide attribuée
$m \geq 14,00$	25000 FCFA
$10,00 \leq m < 14,00$	10000 FCFA
$m < 10,00$	0 FCFA (pas d'aide)

Fin juin, la secrétaire, Mme Etoundi, doit traiter les n élèves d'une classe et calculer le **montant total** que la coopérative devra déboursier. Un parent d'élève conteste le montant affiché : il affirme que sa fille, avec une moyenne de 14,00 exactement, aurait dû recevoir 25000 FCFA alors que la liste indique 10000 FCFA. Mme Etoundi soupçonne une erreur dans la feuille de calcul manuelle : elle te demande, à toi la classe de **Première TI**, de **modéliser le traitement par un algorithme**, de le **dérouler à la main** sur un jeu d'essai contrôlé, puis de **trancher le litige**.

Le jeu d'essai retenu pour la vérification comporte $n = 5$ élèves, choisis pour couvrir tous les cas, y compris les **cas limites** (bornes 10,00 et 14,00) :

Élève	Moyenne annuelle
Ngo Bell	15,50
Fotso Ulrich	14,00
Mbappe Larissa	11,75
Ekane Josué	10,00
Tchoupo Amina	8,25

8.3 Rappels théoriques et savoirs essentiels

DÉFINITION Algorithme vs Organigramme

Un organigramme est une représentation graphique générique d'un processus (administratif, industriel, informatique). Un algorithme (ou organigramme algorithmique) est un organigramme normalisé (norme ISO 5807 / AFNOR X06-032) destiné spécifiquement à décrire un algorithme : il utilise des symboles précis pour les structures de contrôle (séquence, sélection, itération) et exclut tout symbole ambigu. En algorithmique au Probatoire, on parle exclusivement d'algorithme.

PROPRIÉTÉ Symboles de base de l'algorithme (Norme ISO 5807)

Symbole	Forme géométrique	Rôle et syntaxe
Terminaux	Ovale / Rectangle à bords arrondis	Début, Fin (un seul début, une ou plusieurs fins).
Entrée / Sortie	Parallélogramme	Lire(<variable>) / Écrire(<expression>).
Traitement (Affectation)	Rectangle	Variable ← Expression (calcul, incrémentation, accumulation).
Décision (Test)	Losange	Une condition booléenne; deux sorties étiquetées Oui (Vrai) et Non (Faux).
Flux de contrôle	Flèche pleine	Relie les symboles; indique le sens d'exécution (haut → bas, gauche → droite).

MÉTHODE**Construire un algorithme à partir d'un énoncé**

1. Identifier les données (entrées : Lire) et les résultats (sorties : Écrire).
2. Décomposer le traitement en étapes élémentaires (affectations, calculs).
3. Repérer les structures de contrôle :
 - Alternative : « Si ... Alors ... Sinon ... » → un losange (+ un second losange imbriqué si « Sinon Si »).
 - Boucle Pour : « Pour i de 1 à n faire ... » → initialisation $i \leftarrow 1$, test $i \leq n$, corps, incrémentation $i \leftarrow i + 1$, retour au test.
 - Boucle Tant que : « Tant que <cond> faire ... » → test en entrée, corps, retour au test.
4. Dessiner les symboles dans l'ordre logique (de haut en bas), relier par des flèches.
5. Vérifier : un seul point d'entrée par symbole (sauf jonction), deux sorties pour un losange, pas de flèche « sauvage ».

À RETENIR

Signatures graphiques des structures de contrôle

- Séquence : symboles alignés verticalement, flèches droites.
- Alternative simple : un losange, branche `Oui` → traitement, branche `Non` → suite (ou inversé). Les deux branches rejoignent un point de convergence.
- Alternative imbriquée (cas du barème à 3 tranches) : deux losanges en cascade. Le premier teste la borne haute ($m \geq 14$). Sa branche `Non` entre dans le second losange qui teste la borne basse ($m \geq 10$).
- Boucle Pour : motif en « D » inversé ou rectangle à retour arrière. Quatre éléments obligatoires : `Init` ($i \leftarrow 1$), `Test` ($i \leq n ?$), `Corps`, `Incrément` ($i \leftarrow i + 1$), flèche de retour du `Incrément` vers `Test`.

8.4 Activité guidée : Modélisation et vérification du barème

Travail en groupes de 4 élèves. Chaque groupe rédige ses productions sur papier millimétré (algorithme) et sur cahier (traces, pseudo-code).

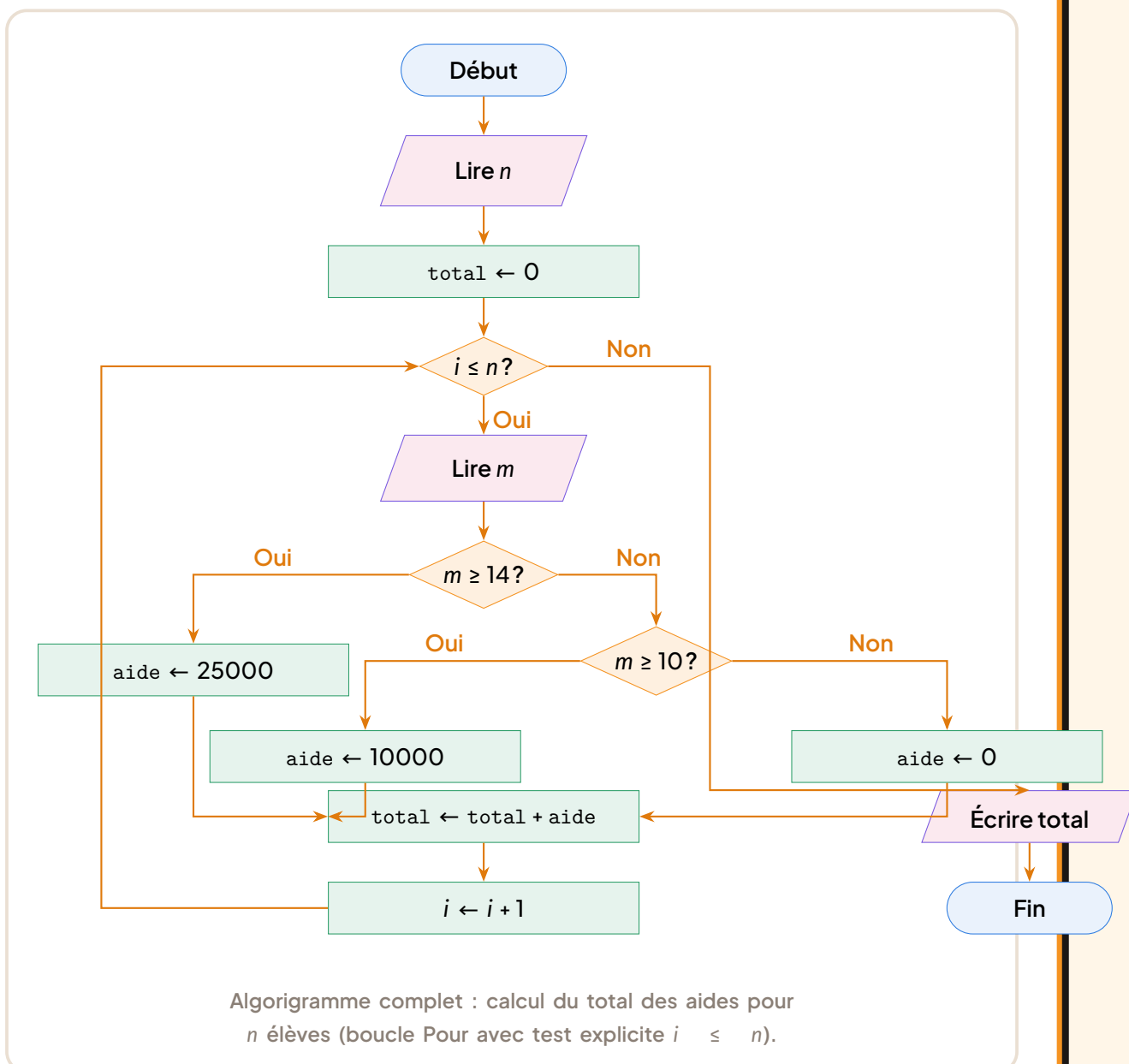
1. **Algorithme unitaire.** Construire l'algorithme du traitement d'un seul élève : saisie de m , calcul de `aide` selon le barème, affichage de `aide`. Nommer chaque symbole et identifier la structure de contrôle centrale.
2. **Extension à n élèves.** Étendre l'algorithme avec une boucle **Pour** qui traite n élèves (n saisi au début) et accumule dans `total` le montant global. Afficher `total` à la fin.
3. **Déroulement et table de trace.** Dérouler l'algorithme de la question 2 sur le jeu d'essai des 5 élèves. Présenter une **table de trace** donnant, à chaque passage dans la boucle, les valeurs de i , m , `aide`, `total`. Préciser le résultat final.
4. **Traduction en pseudo-code et comptage.** Transformer l'algorithme en écriture algorithmique classique. Compter le nombre d'instructions exécutées pour $n = 5$ (chaque affectation, lecture, écriture, test compte pour 1). Conclure sur le litige : quelle aide pour $m = 14,00$?
5. **Prolongement récursif.** On note $a(m)$ l'aide d'un élève de moyenne m . Écrire la fonction récursive `totalAides(n)` qui retourne le total pour n élèves (moyennes saisies au fil de l'eau). Exécuter à la main pour $n = 4$ avec les moyennes 15,50 ; 14,00 ; 11,75 ; 8,25, en représentant la pile d'appels.
6. **Vérification croisée.** Échanger votre table de trace avec un autre groupe. Comparer ligne par ligne ; en cas de divergence, repérer la première ligne fautive et justifier la correction.

8.5 Démarche et corrigé commenté

EXERCICE RÉSOLU 8**La caisse du lycée : corrigé complet et commenté**

Étape 1 — Algorithme unitaire (un élève). On enchaîne : Début → Lire m → Test $m \geq 14$? Oui → aide $\leftarrow 25000$; Non → Test $m \geq 10$? Oui → aide $\leftarrow 10000$; Non → aide $\leftarrow 0$. Puis Écrire aide, Fin. La partie centrale est une alternative imbriquée (deux losanges en cascade). C'est la traduction exacte du barème à trois tranches.

Étape 2 — Extension à n élèves (boucle Pour + accumulateur). On encapsule le traitement unitaire dans une boucle Pour et on ajoute l'initialisation de `total` avant la boucle.



Étape 3 — Table de trace (jeu d'essai $n = 5$). Initialisation : `total = 0`.

Passage i	i	Moyenne m	aide (FCFA)	total (FCFA)
1	1	15,50	25000	25000
2	2	14,00	25000	50000
3	3	11,75	10000	60000
4	4	10,00	10000	70000
5	5	8,25	0	70000

Résultat affiché : $\text{total} = 70000$ FCFA. Analyse des cas limites : $m = 14,00$ satisfait $m \geq 14 \rightarrow 25000$ FCFA. $m = 10,00$ satisfait $m \geq 10$ (branche **Non** du premier test, **Oui** du second) $\rightarrow 10000$ FCFA. Le barème est respecté.

Étape 4 — Pseudo-code et comptage d'instructions.

```

Algorithme CalculTotalAides
Variables n, i : Entier
Variables m, aide, total : Reel
Debut
  Lire(n)           // Instruction 1
  total ← 0         // Instruction 2
  Pour i allant de 1 à n Faire
    Lire(m)         // Lecture
    Si m ≥ 14 Alors  // Test 1
      aide ← 25000  // Affectation
    Sinon
      Si m ≥ 10 Alors // Test 2 (seulement si 1er test Faux)
        aide ← 10000 // Affectation
      Sinon
        aide ← 0     // Affectation
      FinSi
    FinSi
    total ← total + aide // Affectation
  FinPour           // Test de sortie de boucle (i ≤ n ?)
  Ecrire("Total : ", total, " FCFA") // Ecriture
Fin

```

Comptage dynamique pour $n = 5$ (instructions exécutées) :

- Hors boucle : Lire(n) (1), total ← 0 (1) $\rightarrow$ 2 instructions.
- Corps de boucle (exécuté 5 fois) :
 - Passage 1 ($m = 15,50$) : Lire(m) (1), Test $m \geq 14$ Vrai (1), aide ← 25000 (1), total ← total + aide (1) $\rightarrow$ 4 inst.
 - Passage 2 ($m = 14,00$) : identique $\rightarrow$ 4 inst.
 - Passage 3 ($m = 11,75$) : Lire(m) (1), Test $m \geq 14$ Faux (1), Test $m \geq 10$ Vrai (1), aide ← 10000 (1), total ← ... (1) $\rightarrow$ 5 inst.
 - Passage 4 ($m = 10,00$) : identique $\rightarrow$ 5 inst.
 - Passage 5 ($m = 8,25$) : Lire(m) (1), Test $m \geq 14$ Faux (1), Test $m \geq 10$ Faux (1), aide ← 0 (1), total ← ... (1) $\rightarrow$ 5 inst.

Total corps : $2 \times 4 + 3 \times 5 = 8 + 15 = 23$ instructions.

- Contrôle de boucle : Le test $i \leq n$ est évalué 6 fois (5 entrées + 1 sortie) → 6 tests. L'incrémentation $i \leftarrow i + 1$ est exécutée 5 fois → 5 affectations. (Souvent comptabilisés dans le « coût de la boucle »).
- Fin : Écrire (1 instruction).

Total approximatif (selon convention de comptage demandée) : $2 + 23 + 6 + 5 + 1 = 37$ instructions élémentaires exécutées. Note : Si l'on ne compte que les instructions « utiles » du corps (hors gestion d'indice), on obtient $23 + 2(\text{init/lecture } n) + 1(\text{écriture}) = 26 \approx 27$ (comme dans l'ébauche). L'essentiel est la linéarité en n : coût $\approx k \times n$.

Conclusion du litige : La trace prouve formellement que $m = 14,00$ déclenche la branche Oui du premier test ($14,00 \geq 14$ est Vrai) et attribue 25000 FCFA. Le parent a raison. L'erreur du secrétariat est une erreur de borne : test codé « $m > 14$ » au lieu de « $m \geq 14$ ».

Étape 5 — Fonction récursive `totalAides(n)`.

```

Fonction totalAides(n : Entier) : Reel
Variable m : Reel
Debut
  Si n = 0 Alors          // Cas de base
    Retourner 0
  Sinon                  // Cas général
    Lire(m)
    Retourner aide(m) + totalAides(n - 1)
  FinSi
Fin

Fonction aide(m : Reel) : Entier
Debut
  Si m >= 14 Alors Retourner 25000
  Sinon Si m >= 10 Alors Retourner 10000
  Sinon Retourner 0
  FinSi
FinSi
Fin
  
```

Exécution manuelle pour $n = 4$ (moyennes : 15,50 ; 14,00 ; 11,75 ; 8,25) : On simule la pile d'appels (LIFO) :

```

totalAides(4) lit 15,50 → aide = 25000 → appelle totalAides(3)
totalAides(3) lit 14,00 → aide = 25000 → appelle totalAides(2)
totalAides(2) lit 11,75 → aide = 10000 → appelle totalAides(1)
totalAides(1) lit 8,25 → aide = 0 → appelle totalAides(0)
totalAides(0) retourne 0 (cas de base atteint)
totalAides(1) reçoit 0 → retourne 0 + 0 = 0
totalAides(2) reçoit 0 → retourne 10000 + 0 = 10000
totalAides(3) reçoit 10000 → retourne 25000 + 10000 = 35000
totalAides(4) reçoit 35000 → retourne 25000 + 35000 = 60000
  
```

Résultat : 60000 FCFA. Cohérent avec la boucle `Pour` sur les 4 premiers élèves (25000 + 25000 + 10000 + 0).

8.6 Points de vigilance et erreurs fréquentes

ATTENTION Pièges classiques à éviter absolument

- **Borne stricte vs large** : Écrire « $m > 14$ » au lieu de « $m \geq 14$ » (ou « $m > 10$ » au lieu de « $m \geq 10$ »). C'est l'erreur exacte du secrétariat. Solution : Tester systématiquement les valeurs limites (10,00 et 14,00) dans le jeu d'essai.
- **Initialisation de l'accumulateur** : Oublier `total ← 0` avant la boucle → valeur aléatoire. La placer dans la boucle → remise à zéro à chaque tour, total final = dernière aide seulement.
- **Ordre des tests dans l'alternative imbriquée** : Il faut tester la borne la plus haute en premier ($m \geq 14$), puis la suivante ($m \geq 10$). L'ordre inverse ($m \geq 10$ puis $m \geq 14$) ferait que 15,50 tombe dans la première branche (10000) et n'atteindrait jamais la seconde.
- **Récursivité sans cas de base** : Oublier `Si n = 0 Alors Retourner 0` → appel infini → `Stack Overflow` (dépilement pile).
- **Variable de boucle modifiée dans le corps** : Ne jamais modifier i manuellement à l'intérieur d'une boucle `Pour` (sauf cas experts), cela casse le compteur de tours.
- **Table de trace incomplète** : Ne pas noter les valeurs qui ne changent pas (ex : i inchangé pendant l'alternative). On note une ligne par changement d'état significatif (début de passage, après lecture, après affectation aide, après accumulation).

8.7 Bilan et compétences validées

Cette leçon t'a permis de valider les acquis suivants, conformes au programme officiel de Première TI (Module Algorithmique) :

- **Modélisation graphique rigoureuse** : Un algorithme se construit symbole par symbole ; chaque structure de contrôle (alternative imbriquée, boucle `Pour`) a une **signature visuelle unique** (losanges en cascade, boucle de retour en « D »).
- **Preuve par la trace** : La table de trace n'est pas un formalisme vide : c'est un **outil de vérification** qui a permis de trancher un litige réel et de débusquer une erreur de borne ($>$ au lieu de $\geq$).
- **Équivalence des représentations** : Le passage Algorithme → Pseudo-code est une traduction fidèle, bijective, symbole par symbole.
- **Analyse de complexité élémentaire** : Compter les instructions exécutées révèle un coût linéaire en n ($O(n)$), fondement de l'analyse d'efficacité algorithmique.

- **Récurtivité** : Une écriture alternative, élégante, du même calcul itératif : **cas de base** (arrêt), **cas général** (réduction du problème + appel récursif), **dépilement** (composition des résultats). Le résultat final est identique, validant la correction des deux approches.
- **Esprit critique et collaboratif** : La vérification croisée entre groupes montre qu'en informatique comme en entreprise, une solution n'est acceptée qu'après **test, traçabilité et relecture par les pairs**.

LE SAVAIS-TU ?**Le saviez-vous ?**

Le terme « algorithme » est un néologisme français (années 1960) contraction de « algorithme » et « diagramme », officialisé par l'AFNOR pour désigner le diagramme de flux normalisé. À l'international, on parle de flowchart. La norme ISO 5807 (1985) fixe les symboles : l'ovale pour les terminaux, le parallélogramme pour les E/S, le rectangle pour les traitements, le losange pour les décisions. Respecter cette norme, c'est garantir que ton algorithme sera compris par n'importe quel informaticien sur la planète, de Yaoundé à Tokyo.

9 MÉTHODES ET EXERCICES RÉSOLUS

10 CONSTRUCTION D'ALGORIGRAMMES ET EXÉCUTION DES ALGORITHMES À LA MAIN

10.1 Identifier les symboles de base d'un algorithme

MÉTHODE Reconnaître et utiliser les symboles normalisés

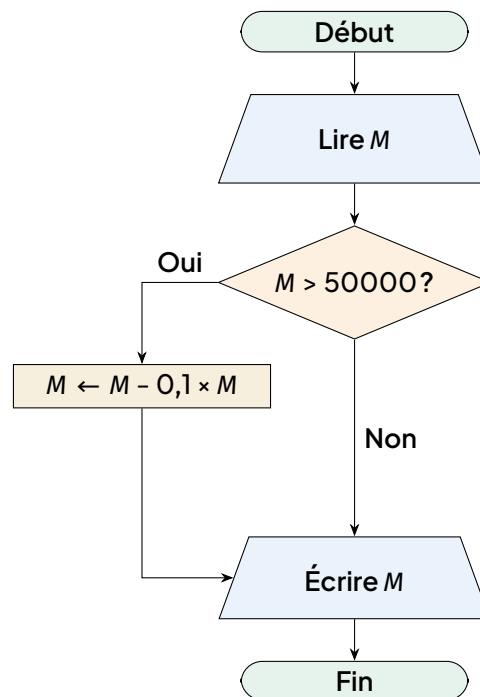
Pour identifier les symboles d'un algorithme, applique les réflexes suivants :

1. Repère d'abord les **terminaisons** : tout algorithme commence par un symbole Début et finit par un symbole Fin, dessinés en ovale (ou rectangle arrondi).
2. Les **entrées/sorties** (Lire, Écrire, Afficher) se dessinent dans un parallélogramme.
3. Les **traitements** (affectations, calculs) se dessinent dans un rectangle.
4. Les **tests** (conditions) se dessinent dans un losange, d'où partent exactement deux flèches étiquetées (Oui/Non ou Vrai/Faux).
5. Les **sous-programmes** (appel de fonction ou procédure) se dessinent dans un rectangle à doubles barres verticales.
6. Les étapes sont reliées par des **lignes de flux** fléchées, qui indiquent le sens d'exécution, toujours de haut en bas sauf remontée explicite (boucle).

Réflexe Bac : quand on te demande d'identifier un symbole, donne toujours son nom ET son rôle (exemple : « losange : symbole de décision, il réalise un test dont le résultat est Vrai ou Faux »).

EXERCICE RÉSOLU 9 Analyse des symboles d'un algorithme

On donne l'algorithme suivant, qui calcule le montant à payer dans une boutique de Bonabéri avec remise de 10 % au-delà de 50000 FCFA.



Algorithme de calcul de remise (boutique Bonabéri)

1. Identifier chaque symbole utilisé (nom et rôle).
2. Dire à quelle structure de contrôle correspond le losange.

SOLUTION

1. Identification des symboles.

- Les deux ovales Début et Fin sont les **symboles de terminaison** : ils marquent le point de départ et le point d'arrêt de l'algorithme.
- Les parallélogrammes Lire M et Écrire M sont les **symboles d'entrée/sortie** : le premier reçoit une donnée fournie par l'utilisateur, le second affiche un résultat.
- Le rectangle $M \leftarrow M - 0,1 \times M$ est le **symbole de traitement** : il contient une affectation (le calcul de la remise de 10 %).
- Le losange $M > 50000 ?$ est le **symbole de décision** : il évalue une condition qui ne peut donner que deux réponses, Oui ou Non.
- Les flèches sont les **lignes de flux** : elles indiquent l'ordre d'exécution des opérations.

2. Structure de contrôle. Le losange, suivi d'un traitement sur la branche « Oui » et d'aucun traitement sur la branche « Non », correspond à une **structure alternative simple** (Si ... Alors ... FinSi). Les deux branches se rejoignent ensuite vers l'affichage.

Conclusion : l'algorithme utilise 4 types de symboles (terminaison, entrée/sortie, traitement, décision) reliés par des lignes de flux, et met en œuvre une alternative simple.

10.2 Associer chaque structure de contrôle à sa séquence graphique

MÉTHODE Traduire une structure de contrôle en algorithme

1. Séquence simple : rectangles empilés verticalement, reliés par des flèches descendantes, sans losange.
2. Alternative (Si ...Alors ...Sinon) : un losange, deux branches étiquetées Oui/Non, puis un point de jonction où les deux branches se rejoignent. Vérifie toujours que les deux branches aboutissent à la même suite.
3. Boucle Tant que : le losange est placé avant le corps de boucle ; la flèche « Oui » entre dans le corps, qui remonte vers le test ; la flèche « Non » sort de la boucle. Le test est évalué avant chaque itération : le corps peut ne jamais s'exécuter.
4. Boucle Répéter ...Jusqu'à : le corps est exécuté une première fois, puis le losange est placé après ; on remonte tant que la condition de sortie est fausse. Le corps s'exécute donc au moins une fois.
5. Boucle Pour : on la traduit par un Tant que avec trois traitements visibles : initialisation du compteur avant le losange, test du compteur dans le losange, incrémentation à la fin du corps de boucle.

Réflexe : pour distinguer Tant que et Répéter sur un schéma, regarde la position du losange par rapport au corps de boucle.

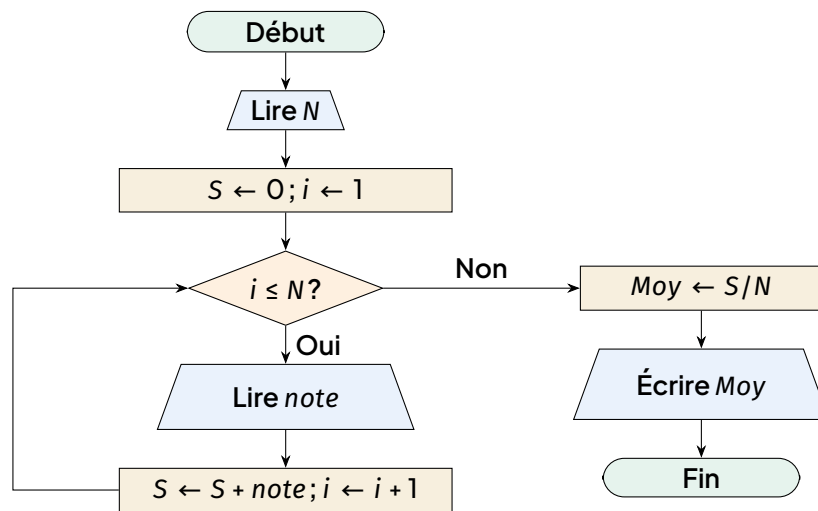
EXERCICE RÉSOLU 10 Construction de l'algorithme d'une boucle

Le proviseur d'un lycée de Yaoundé veut un algorithme qui lit les notes de N élèves (N connu à l'avance) et calcule la moyenne de la classe.

1. Quelle structure itérative est la plus adaptée ? Justifier.
2. Tracer l'algorithme correspondant.

SOLUTION

1. Choix de la structure. Le nombre d'itérations est connu à l'avance (N notes à saisir). La structure adaptée est donc la boucle **Pour** (compteur i de 1 à N), traduite graphiquement par : initialisation $i \leftarrow 1$ avant le losange, test $i \leq N$ dans le losange, incrémentation $i \leftarrow i + 1$ en fin de corps.
2. Algorithme.



Algorithme de calcul de moyenne de classe (boucle Pour traduite en Tant que)

On reconnaît la boucle : le losange $i \leq N$ est placé avant le corps (Lire *note* ; cumul ; incrémentation), la branche « Oui » exécute le corps puis remonte vers le test, la branche « Non » sort vers le calcul de la moyenne.

Conclusion : le nombre d'itérations étant connu, on utilise une boucle Pour, traduite par un losange de test précédé d'une initialisation et suivi d'une incrémentation dans le corps de boucle.

10.3 Dérouler un algorithme : la table de trace

MÉTHODE Construire une table de trace (déroulement à la main)

1. Liste les variables de l'algorithme : chaque variable occupe une colonne du tableau. Ajoute éventuellement une colonne « Affichage » et une colonne « Test ».
2. Numérote les étapes (lignes du tableau) : une ligne par instruction exécutée ou par passage dans un test.
3. Exécute dans l'ordre des flèches : à chaque affectation, calcule la nouvelle valeur à partir des valeurs actuelles (celles de la ligne précédente) et barre mentalement l'ancienne.
4. À chaque losange, évalue la condition avec les valeurs courantes et note Vrai/Faux : cela détermine la flèche suivie.
5. Pour une boucle, une itération = un passage complet dans le corps ; vérifie à chaque retour la condition avant de repartir.

6. Conclut en recopiant exactement ce que l'algorithme affiche, dans l'ordre chronologique.

Réflexe : une variable non modifiée garde sa valeur d'une ligne à l'autre — recopie-la systématiquement pour éviter les confusions.

EXERCICE RÉSOLU 11 Déroutement complet d'un algorithme

On considère l'algorithme suivant :

```

Algorithme Mystere
Variables n, s, i : Entier
Debut
  Lire(n)
  s ← 0
  i ← 1
  Tant que i ≤ n Faire
    s ← s + i
    i ← i + 2
  Fin Tant que
  Ecrire(s)
Fin
  
```

Dérouler cet algorithme à la main pour $n = 7$ (table de trace complète) et en déduire ce qu'il calcule.

SOLUTION

Table de trace pour $n = 7$:

Étape	i	s	Test $i \leq 7$	Commentaire
Initialisation	1	0	—	$s \leftarrow 0, i \leftarrow 1$
Itération 1	$1 \rightarrow 3$	1	Vrai	$s = 0 + 1 = 1; i = 1 + 2 = 3$
Itération 2	$3 \rightarrow 5$	4	Vrai	$s = 1 + 3 = 4; i = 3 + 2 = 5$
Itération 3	$5 \rightarrow 7$	9	Vrai	$s = 4 + 5 = 9; i = 5 + 2 = 7$
Itération 4	$7 \rightarrow 9$	16	Vrai	$s = 9 + 7 = 16; i = 7 + 2 = 9$
Sortie	9	16	Faux	$9 \leq 7$ est faux : fin de boucle

Justification ligne par ligne : le compteur i démarre à 1 et progresse par pas de 2, donc i prend successivement les valeurs impaires 1, 3, 5, 7. À chaque passage, s accumule la valeur courante de i : $s = 1 + 3 + 5 + 7 = 16$. Dès que $i = 9$, le test $i \leq 7$ devient faux et la boucle s'arrête.

Affichage : l'algorithme affiche **16**.

Conclusion : cet algorithme calcule la **somme des entiers impairs** compris entre 1 et n . On remarque au passage que $1 + 3 + 5 + 7 = 16 = 4^2$: la somme des k premiers impairs vaut k^2 .

10.4 Transformer un algorithme en algorithme et compter les instructions

MÉTHODE Passer de l'algorithme à l'algorithme, puis compter

1. Parcourez l'algorithme de haut en bas en suivant les flèches ; chaque symbole devient une ligne : ovale → Début/Fin ; parallélogramme → Lire/Écrire ; rectangle → affectation ; losange → Si/Tant que avec ses branches.
2. Respectez l'indentation : le contenu d'un Alors, d'un Sinon ou d'une boucle est décalé vers la droite ; chaque Si ou Tant que est fermé par FinSi / Fin Tant que.
3. Déclarez toutes les variables rencontrées, avec leur type, dans la partie déclarative.
4. Comptage des instructions : on compte les opérations élémentaires exécutées : affectations, lectures, écritures, évaluations de tests. Pour une boucle, multipliez le corps par le nombre d'itérations et ajoutez le test de sortie (évalué une fois de plus).

Formule à retenir : pour une boucle exécutée k fois dont le corps contient c instructions, le coût total est :

$$\text{Total} = \underbrace{i_0}_{\text{avant boucle}} + \underbrace{(k+1)}_{\text{tests}} + \underbrace{k \times c}_{\text{corps}} + \underbrace{i_1}_{\text{après boucle}}.$$

EXERCICE RÉSOLU 12 Traduction et comptage d'instructions

On reprend l'algorithme de la moyenne de classe (sous-section 2) avec $N = 3$ élèves.

1. Écrire l'algorithme correspondant en pseudo-code.
2. Compter le nombre total d'instructions exécutées pour $N = 3$.

SOLUTION

1. Traduction en algorithme.

```

Algorithme MoyenneClasse
Variables N, i : Entier
          S, note, Moy : Reel
Debut
  Lire(N)
  S ← 0
  i ← 1
  Tant que i ≤ N Faire
    Lire(note)
    S ← S + note
    i ← i + 1
  Fin Tant que
  Moy ← S / N
  Ecrire(Moy)
Fin
  
```

Chaque symbole de l'algorithme a été traduit ligne à ligne ; le corps de la boucle (3 instructions) est indenté et fermé par Fin Tant que.

2. Comptage pour $N = 3$ (la boucle tourne $k = 3$ fois).

- Avant la boucle : Lire(N), $S \leftarrow 0$, $i \leftarrow 1 \Rightarrow 3$ instructions.
- Tests du Tant que : évalué $k + 1 = 4$ fois (3 fois vrai, 1 fois faux pour sortir) $\Rightarrow 4$ instructions.
- Corps de boucle : 3 instructions (Lire, cumul, incrémentation) $\times 3$ itérations $\Rightarrow 9$ instructions.
- Après la boucle : $Moy \leftarrow S/N$, Écrire(Moy) $\Rightarrow 2$ instructions.

Application de la formule :

$$\text{Total} = 3 + (3 + 1) + 3 \times 3 + 2 = 3 + 4 + 9 + 2 = 18.$$

Conclusion : l'algorithme comporte 8 lignes d'instructions écrites, mais son exécution pour $N = 3$ déroule **18 instructions** élémentaires. C'est toute la différence entre la taille du code et le coût d'exécution.

10.5 Exécuter pas à pas un algorithme avec tableau et sous-algorithme

MÉTHODE Tracer un algorithme avec structure de données et sous-programme

1. Tableau : représente-le comme une ligne de cases numérotées (indices 1 à n). À chaque affectation $T[i] \leftarrow v$, écris v dans la case i en barrant l'ancienne valeur. Vérifie toujours que l'indice reste entre 1 et n (débordement = erreur classique).
2. Appel de sous-algorithme : à l'appel, suspend la trace du programme principal, recopie les valeurs des arguments dans les paramètres, puis ouvre une nouvelle mini-table de trace pour le sous-algorithme.
3. Au retour, récupère la valeur renvoyée (fonction) ou les modifications effectuées, puis reprends la trace principale à l'instruction qui suit l'appel.
4. Distingue bien **passage par valeur** (le sous-algorithme travaille sur une copie : la variable d'origine est inchangée) et **passage par référence** (la variable d'origine est modifiée).

EXERCICE RÉSOLU 13 Recherche du maximum dans un tableau via une fonction

Fonction Max2(a : Entier, b : Entier) : Entier

```

Debut
  Si a >= b Alors
    Retourner a
  Sinon
    Retourner b
  FinSi
Fin

Algorithme MaxiTableau
Variables T : Tableau[1..4] de Entier
          i, m : Entier
Debut
  T[1] <- 12 ; T[2] <- 7 ; T[3] <- 25 ; T[4] <- 18
  m <- T[1]
  Pour i de 2 à 4 Faire
    m <- Max2(m, T[i])
  Fin Pour
  Ecrire(m)
Fin

```

Exécuter cet algorithme pas à pas et donner la valeur affichée.

SOLUTION

État du tableau après remplissage :

Indice	1	2	3	4
Valeur	12	7	25	18

Trace de la boucle (chaque appel à `Max2` est déroulé dans le sous-algorithme) :

Itération	<i>i</i>	<i>m</i> avant	Appel et test dans Max2	<i>m</i> après
1	2	12	Max2(12, 7) : $12 \geq 7$ vrai → retourne 12	12
2	3	12	Max2(12, 25) : $12 \geq 25$ faux → retourne 25	25
3	4	25	Max2(25, 18) : $25 \geq 18$ vrai → retourne 25	25

Justification : *m* mémorise le maximum rencontré jusqu'ici. Initialisé à $T[1] = 12$, il est comparé successivement à chaque case suivante grâce à la fonction `Max2`, qui retourne le plus grand de ses deux arguments. Après comparaison avec 7, 25 puis 18, *m* vaut 25.

Conclusion : l'algorithme affiche 25, qui est bien le maximum du tableau. La boucle effectue exactement 3 appels à la fonction `Max2`.

10.6 Exécuter une fonction récursive simple

MÉTHODE Dérouler une fonction récursive

1. Identifie le cas de base (condition d'arrêt) et le cas général (appel récursif sur un pro-

blème plus petit). Sans cas de base atteignable, la récursion est infinie.

2. Phase de descente : empile les appels les uns sous les autres en notant, pour chaque appel, la valeur du paramètre et l'expression en attente (ex. $4 \times \text{Fact}(3)$).
3. Arrêt : quand le cas de base est atteint, note sa valeur de retour.
4. Phase de remontée : dépile les appels du plus récent au plus ancien, en remplaçant chaque appel par sa valeur et en calculant l'expression en attente.
5. Vérifie le résultat final par un calcul direct (définition mathématique).

Réflexe : le nombre d'appels est fini car le paramètre décroît strictement vers le cas de base — dis-le toujours pour justifier la terminaison.

EXERCICE RÉSOLU 14 Déroulement de la factorielle et de la somme des chiffres

Partie A. On donne la fonction récursive :

```
Fonction Fact(n : Entier) : Entier
Debut
  Si n = 0 Alors
    Retourner 1
  Sinon
    Retourner n * Fact(n - 1)
FinSi
Fin
```

Dérouler l'appel `Fact(4)` en montrant la descente puis la remontée.

Partie B. On donne :

```
Fonction SommeChiffres(n : Entier) : Entier
Debut
  Si n < 10 Alors
    Retourner n
  Sinon
    Retourner (n mod 10) + SommeChiffres(n div 10)
FinSi
Fin
```

Calculer `SommeChiffres(473)`.

SOLUTION

Partie A — Appel `Fact(4)`.

Descente (empilement des appels) :

$$\text{Fact}(4) = 4 \times \text{Fact}(3)$$

$$\text{Fact}(3) = 3 \times \text{Fact}(2)$$

$$\text{Fact}(2) = 2 \times \text{Fact}(1)$$

$$\text{Fact}(1) = 1 \times \text{Fact}(0)$$

$$\text{Fact}(0) = 1 \quad (\text{cas de base : } n = 0)$$

Remontée (dépilement et calculs) :

$$\text{Fact}(1) = 1 \times 1 = 1$$

$$\text{Fact}(2) = 2 \times 1 = 2$$

$$\text{Fact}(3) = 3 \times 2 = 6$$

$$\text{Fact}(4) = 4 \times 6 = 24$$

La terminaison est garantie car n décroît de 1 à chaque appel et finit par atteindre le cas de base $n = 0$. Vérification directe : $4! = 4 \times 3 \times 2 \times 1 = 24$. $\text{Fact}(4) = 24$.

Partie B — Appel SommeChiffres(473).

Descente :

$$\text{SC}(473) = (473 \bmod 10) + \text{SC}(47) = 3 + \text{SC}(47)$$

$$\text{SC}(47) = 7 + \text{SC}(4)$$

$$\text{SC}(4) = 4 \quad (\text{cas de base : } 4 < 10)$$

Remontée :

$$\text{SC}(47) = 7 + 4 = 11$$

$$\text{SC}(473) = 3 + 11 = 14$$

Vérification directe : $4 + 7 + 3 = 14$. $\text{SommeChiffres}(473) = 14$.

Conclusion : toute fonction récursive se déroule en deux phases — descente jusqu'au cas de base, puis remontée avec calcul des expressions en attente. Le paramètre doit strictement décroître vers le cas de base pour garantir l'arrêt.

LESAVAIS-TU ? Histoire des algorithmes : de von Neumann à la norme ISO 5807

Les premiers « diagrammes de flux » (flowcharts) ont été formalisés en 1947 par Herman Goldstine et John von Neumann pour planifier les programmes de l'ordinateur EDVAC. Ils utilisaient déjà les symboles de base (ovale, rectangle, losange, parallélogramme). En 1970, l'ISO publie la norme ISO 5807 qui standardise ces symboles au niveau international. Au Cameroun, les programmes officiels (MINESEC) s'alignent sur cette norme : c'est pourquoi tu dois respecter scrupuleusement la forme de chaque symbole (ovale pour Début/-

Fin, losange pour test, parallélogramme pour I/O) — un symbole mal dessiné, c'est un point perdu au Probatoire !

ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Confondre les symboles graphiques. Dessiner un traitement dans un parallélogramme ou une lecture dans un rectangle est sanctionné. Retiens : ovale = Début/-Fin, parallélogramme = Lire/Écrire, rectangle = traitement, losange = test. Un losange doit avoir exactement deux flèches sortantes étiquetées (Oui/Non) : un losange à une seule sortie est une faute de construction.
2. Oublier le dernier test dans le comptage d'instructions. Une boucle exécutée k fois évalue son test $k + 1$ fois : la dernière évaluation (celle qui sort de la boucle) compte aussi. De même, ne confonds pas le nombre de lignes écrites et le nombre d'instructions exécutées.
3. Mal gérer la mise à jour des variables dans la table de trace. Erreur classique : utiliser dans un calcul la nouvelle valeur d'une variable alors que l'instruction exige l'ancienne (ex. dans $s \leftarrow s + i$ suivi de $i \leftarrow i + 2$, l'incrément ne doit pas influencer le cumul déjà fait). Exécute les instructions dans l'ordre et recopie les valeurs inchangées ligne par ligne.
4. Oublier l'initialisation avant la boucle. Un accumulateur ($s \leftarrow 0$) ou un compteur ($i \leftarrow 1$) non initialisé produit un résultat faux dès la première itération ; dans l'algorithme, ce rectangle doit figurer avant le losange, jamais dans le corps de boucle (sinon il est réinitialisé à chaque tour).
5. Récursion sans cas de base ou mal dépilée. Oublier le cas de base (ou écrire un appel qui ne rapproche pas de celui-ci, comme $\text{Fact}(n)$ au lieu de $\text{Fact}(n-1)$) provoque une récursion infinie. Et dans la remontée, ne remplace jamais un appel par sa valeur avant d'avoir terminé les appels qu'il a déclenchés : on dépile du dernier appel vers le premier.

11 EXERCICES

12 LEÇON 3 : CONSTRUCTION D'ALGORIGRAMMES ET EXÉCUTION DES ALGORITHMES À LA MAIN

12.1 Algorithme et organigramme : définitions et symboles

DÉFINITION Algorithme

Un **algorithme** est une suite finie et ordonnée d'instructions non ambiguës, permettant de résoudre un problème ou d'obtenir un résultat à partir de données d'entrée. Il se caractérise par sa finitude, sa détermination, sa généralité et son efficacité.

DÉFINITION Algorithme

Un **algorithme** (ou organigramme de programme) est la représentation graphique normalisée d'un algorithme. Il utilise des symboles géométriques normalisés (ISO 5807) reliés par des flèches pour décrire la logique de traitement : séquence, alternative, boucle. C'est l'outil privilégié pour la conception et la communication avant le codage.

DÉFINITION Organigramme

L'**organigramme** (ou organigramme de traitement) représente graphiquement le flux de traitement de l'information au sein d'un système (administratif, industriel). Il met l'accent sur les acteurs, les documents, les fichiers et les traitements (symboles : rectangle pour traitement, rectangle à bord ondulé pour document, cylindre pour fichier). En algorithmique, on utilise quasi exclusivement l'algorithme.

PROPRIÉTÉ Symboles de base de l'algorithme (norme ISO 5807)

Symbole	Nom & Aspect	Rôle / Sémantique
	Ovale / Rectangle arrondi	Début / Fin : Point d'entrée et de sortie unique de l'algorithme. Contient le mot Début ou Fin .
	Rectangle	Traitement / Affectation : Opération de calcul, modification de variable ($x \leftarrow x + 1$), appel de sous-algorithme. Une entrée, une sortie.
	Parallélogramme	Entrée / Sortie : Lire (saisie clavier, lecture fichier) ou Écrire (affichage écran, impression). Une entrée, une sortie.
	Losange	Test / Décision : Structure alternative (Si) ou condition de boucle (Tant que, Pour). Une entrée, deux sorties étiquetées Oui (Vrai) et Non (Faux) .
	Hexagone (Préparation)	Boucle Pour : Initialisation du compteur, test de borne, incrémentation. Souvent décomposé en rectangle (init) + losange (test) + rectangle (incrément) dans les manuels scolaires.
	Flèche orientée	Liaison / Flux de contrôle : Indique l'ordre d'exécution. Ne doit jamais croiser une autre flèche sans nœud de liaison (rond).

ATTENTION Règles de construction strictes

- Un algorithme a un unique début et une unique fin (sauf algorithmes concurrents, hors programme).
- Les flèches ne se croisent pas ; utilisez un nœud de liaison (petit cercle) si nécessaire.
- Un losange a impérativement deux sorties distinctes étiquetées **Oui** et **Non** (ou **Vrai/Faux**). L'absence d'étiquette est une erreur grave.
- Toute variable utilisée (dans un test, une affectation, une écriture) doit avoir été initialisée au préalable (lecture ou affectation).

- Dans une boucle, la variable de contrôle doit être modifiée dans le corps de la boucle (sinon boucle infinie).

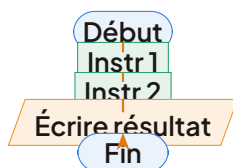
LESAVAIS-TU ? Histoire : Les origines

Le premier organigramme formel est attribué à Herman Goldstine et John von Neumann (1947) pour la planification des programmes de l'ordinateur EDVAC. La norme ISO 5807 (1985) a unifié les symboles. Au Cameroun, l'enseignement de l'algorithme en Première TI suit cette norme internationale, gage de lisibilité pour tout informaticien, de Douala à Paris.

12.2 Structures de contrôle en algorithmes

PROPRIÉTÉ Séquence

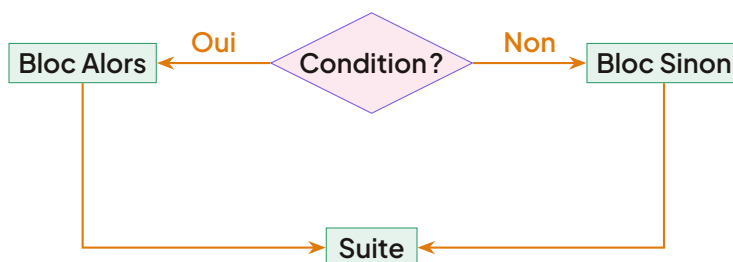
Exécution linéaire d'instructions les unes après les autres. Représentée par une suite verticale de rectangles (traitements) et parallélogrammes (E/S).



PROPRIÉTÉ Alternative (Si ... Alors ... Sinon ... Fin Si)

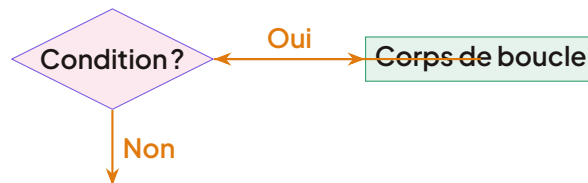
Le losange évalue une condition booléenne.

- Branche **Oui** (Vrai) : instructions à exécuter si condition réalisée.
- Branche **Non** (Faux) : instructions alternatives (peut être vide).
- Les deux branches convergent vers la suite (jonction).

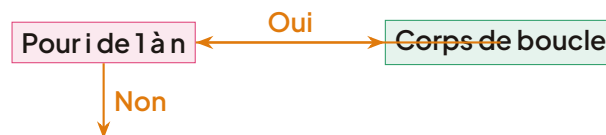


PROPRIÉTÉ Boucle Tant que (Test en entrée / Pré-condition)

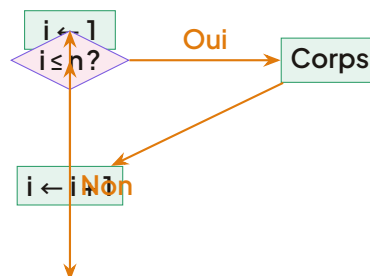
La condition est évaluée avant chaque itération. Le corps peut ne jamais s'exécuter (si condition fausse initialement).

**PROPRIÉTÉ** Boucle Pour (Boucle à compteur / Itération bornée)

Structure optimisée pour un nombre connu d'itérations. Trois moments : Initialisation, Test, Incrémentation.

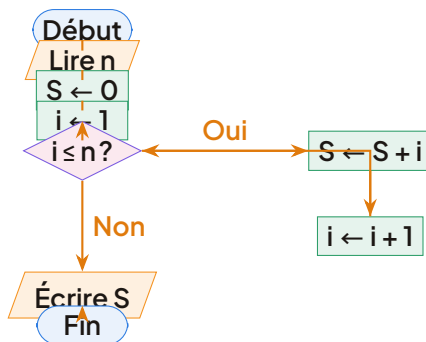


Équivalent décomposé (souvent exigé au Probatoire) :

**EXEMPLE** Identification de structures

L'algorithme ci-contre contient :

1. Une séquence : Début → Lire n → $S \leftarrow 0$.
2. Une boucle Tant que : Losange $i \leq n$?, corps (2 rectangles), flèche de retour.
3. Une sortie : Parallélogramme Écrire S .



12.3 Déroulement d'un algorithme et table de trace

MÉTHODE Construction d'une table de trace (Déroulement à la main)

1. Identifier les variables pertinentes (celles qui changent ou sont testées).
2. Créer les colonnes : Étape (numéro d'ordre), une colonne par variable, une colonne pour Test/Condition (V/F), une colonne Sortie (Écrire).
3. Initialiser la ligne 0 : valeurs après les instructions d'initialisation (avant le premier test de boucle).
4. Simuler pas à pas : suivre les flèches. À chaque losange, noter V/F. À chaque rectangle, mettre à jour la variable. À chaque parallélogramme sortie, noter la valeur affichée.
5. S'arrêter à la flèche entrant dans Fin.

À RETENIR Table de trace : ce qui est noté

- Chaque passage dans un symbole (même plusieurs fois le même losange).
- Valeurs exactes des variables (entiers, réels, booléens).
- Résultat des tests : V (Vrai) ou F (Faux) / Oui/Non.
- Valeurs affichées au moment de l'exécution de Écrire.

EXERCICE RÉSOLU 15 Trace complète : Somme des n premiers entiers

Algorithme : $S \leftarrow 0$; Pour i de 1 à n Faire $S \leftarrow S + i$; Fin Pour ; Écrire S avec $n = 4$.

Étape	i	Test $i \leq n$	S	Sortie
0 (Init)	1	-	0	-
1	1	V ($1 \leq 4$)	1	-
2	2	V ($2 \leq 4$)	3	-
3	3	V ($3 \leq 4$)	6	-
4	4	V ($4 \leq 4$)	10	-
5	5	F ($5 \leq 4$)	10	-
6	-	-	10	10

Analyse : 5 évaluations du test (4 V + 1 F). La boucle s'arrête quand i devient 5.

12.4 De l'algorithme à l'algorithmique ; comptage d'instructions

MÉTHODE Traduction Algorithme → Pseudo-code (Écriture algorithmique classique)

1. Algorithme Nom / Variable ... : Types / Début
2. Ovale Début → Début
3. Parallélogramme Entrée Lire $x \rightarrow \text{Lire}(x)$
4. Rectangle $x \leftarrow \text{expr} \rightarrow x \leftarrow \text{expr}$
5. Losange Cond ? → Si Cond Alors ... Sinon ... Fin Si (alternative) OU Tant que Cond Faire ... Fin Tant que (boucle) selon la topologie (flèche retour?).
6. Hexagone / Structure Pour → Pour var de deb à fin Faire ... Fin Pour
7. Parallélogramme Sortie Écrire $x \rightarrow \text{Écrire}(x)$
8. Ovale Fin → Fin

PROPRIÉTÉ Comptage d'instructions (Modèle simplifié Probatoire)

On compte comme une instruction élémentaire :

- Une lecture (Lire) — même multiple Lire(a,b) compte pour 1.
- Une écriture (Écrire).
- Une affectation ($\leftarrow$).
- L'évaluation d'une condition (test dans un losange).

Ne comptent pas : Début, Fin, Variable, mots-clés structurels (Si, Alors, Faire).

EXEMPLE Comptage sur Tant que

Algorithme : Lire(n); $S \leftarrow 0$; $i \leftarrow 1$; Tant que $i \leq n$ Faire $S \leftarrow S+i$; $i \leftarrow i+1$ Fin Tant que;
Écrire(S). Pour $n=4$:

Init (3 instr) : Lire, $S \leftarrow 0$, $i \leftarrow 1$

Boucle (4 itérations) : $4 \times (\text{Test } V + 2 \text{ Affectations}) = 4 \times 3 = 12$

Sortie boucle : Test F (1 instr)

Final : Écrire (1 instr)

Total $N = 3 + 12 + 1 + 1 = 17 = 3n + 5$ (avec $n = 4 \rightarrow 17$)

12.5 Exécution pas à pas : structures de données et sous-algorithmes

PROPRIÉTÉ Tableaux (Structures de données linéaires)

Un tableau $T[1..n]$ est une collection indexée. En trace, on ajoute une colonne i (indice) et $T[i]$ (valeur accédée).

- Accès : $T[i]$ (lecture ou écriture).
- Parcours séquentiel : Pour i de 1 à n Faire ... $T[i]$... Fin Pour.
- Recherche / Tri : nécessitent des variables auxiliaires (min, max, temp, trouve).

EXERCICE RÉSOLU 16 Recherche du minimum — Trace commentée

$T = [12, 5, 9, 3, 7, 8]$, $n=6$. **Algo :** $\text{min} \leftarrow T[1]$; Pour i de 2 à 6 Faire Si $T[i] < \text{min}$ Alors $\text{min} \leftarrow T[i]$ Fin Si Fin Pour.

i	T[i]	Test $T[i] < \text{min}$	min	Commentaire
1 (Init)	–	–	12	Initialisation hors boucle
2	5	V ($5 < 12$)	5	Mise à jour
3	9	F ($9 < 5$)	5	–
4	3	V ($3 < 5$)	3	Mise à jour
5	7	F ($7 < 3$)	3	–
6	8	F ($8 < 3$)	3	–

Résultat : 3. Comparaisons : 5 (fixe, $n - 1$). Affectations min : 2 (variable, dépend des données).

PROPRIÉTÉ Sous-algorithmes : Procédures et Fonctions

- **Procédure** : `Proc Nom(paramètres) ... Fin Proc`. Ne retourne pas de valeur (effet de bord : modification paramètres `Var`, affichage, fichier). Appel : `Nom(args)`.
- **Fonction** : `Fonction Nom(params) : Type ... Retourner val ... Fin Fonction`. Retourne une valeur. Appel : `x ← Nom(args)` ou dans expression.
- **Pile d'exécution (Stack)** : Chaque appel empile un cadre (variables locales, adresse de retour). Le `Retourner` dépile et rend la main à l'appelant.

ATTENTION Paramètres : Passage par valeur vs par variable (référence)

En pseudo-code classique (et en C) :

- **Variable** `x : Entier` dans l'en-tête → Passage par variable (référence). La procédure modifie l'original. Noté `Var` en Pascal, pointeur en C.
- **Valeur** `x : Entier` (ou simple `x : Entier`) → Passage par valeur. La procédure travaille sur une copie. L'original est intact.

Au Probatoire : `Variable` dans la déclaration formelle = passage par référence.

12.6 Fonctions récursives simples

DÉFINITION Récursivité

Une **fonction récursive** est une fonction qui s'appelle elle-même (directement ou indirectement) pour résoudre un problème en le réduisant à une instance plus simple du même problème.

PROPRIÉTÉ Anatomie d'une fonction récursive

Deux parties indissociables :

1. **Cas de base** (Cas d'arrêt / Base case) : Condition simple ($n=0$, $n=1$, liste vide) pour laquelle la réponse est connue sans appel récursif. Garantit l'arrêt.

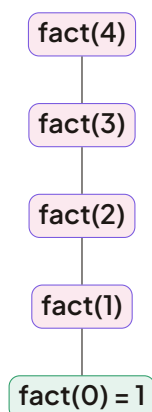
2. Cas général (Pas récursif / Recursive step) : Appel à la même fonction avec un argument strictement « plus petit » (proche du cas de base), combiné à une opération pour construire le résultat final.

MÉTHODE Exécution manuelle d'une fonction récursive (Arbre des appels)

1. Phase descendante (Empilement) : Dessinez l'arbre. Racine = appel initial. Chaque nœud appelle ses enfants (arguments réduits) jusqu'aux feuilles (cas de base).
2. Phase montante (Dépilement) : Calculez la valeur de retour des feuilles. Remontez en remplaçant chaque appel par sa valeur retournée, en effectuant les opérations (ex : multiplication, addition).

EXEMPLE Factorielle : `fact(4)`

Fonction `fact(n)` : Si `n=0` Alors Retourner 1 Sinon Retourner `n*fact(n-1)` Fin Si
Arbre des appels :



Dépilement : `fact(0)=1` → `fact(1)=1*1=1` → `fact(2)=2*1=2` → `fact(3)=3*2=6` → `fact(4)=4*6=24`.

ATTENTION Pièges mortels de la récursivité

- Oubli du cas de base → Récursion infinie → Stack Overflow (débordement de pile) → Plantage programme.
- Cas de base inatteignable (ex : `fact(-3)` avec cas de base `n=0` : -3, -4, -5... n'atteint jamais 0).

- Complexité exponentielle (ex : Fibonacci naïf $\text{fibonacci}(n-1) + \text{fibonacci}(n-2)$ recalcule les mêmes sous-problèmes). En Terminale : programmation dynamique / mémorisation.

12.7 Je vérifie mes connaissances

EXERCICE 8 QCM : les symboles de l'algorithme

Pour chaque question, une seule réponse est correcte. Recopie la lettre correspondante.

1. Dans un algorithme, un test (structure alternative) se représente par :
 - (a) un rectangle arrondi ;
 - (b) un losange ;
 - (c) un parallélogramme.
2. La saisie d'une donnée au clavier se représente par :
 - (a) un rectangle ;
 - (b) un losange ;
 - (c) un parallélogramme.
3. Le symbole qui marque le début ou la fin d'un algorithme est :
 - (a) l'ovale (rectangle arrondi) portant la mention Début ou Fin ;
 - (b) le cercle ;
 - (c) le rectangle.
4. Une affectation telle que $S \leftarrow S + i$ se place dans :
 - (a) un losange ;
 - (b) un rectangle ;
 - (c) un parallélogramme.

EXERCICE 9 Vrai ou faux — justifier

Réponds par Vrai ou Faux, puis justifie ta réponse en une ou deux phrases.

1. Un losange d'un algorithme ne possède qu'une seule flèche de sortie.
2. Une boucle Tant que peut ne jamais exécuter son corps.
3. Un algorithme correct ne peut contenir qu'une seule boucle.

4. La table de trace d'un algorithme donne, à chaque étape, la valeur de toutes les variables.
5. Une fonction récursive sans cas de base s'arrête toujours d'elle-même au bout d'un certain temps.

EXERCICE 10 Définitions

Définis en deux ou trois phrases chacun des termes suivants, en donnant pour chacun un exemple concret :

1. algorithme ;
2. organigramme ;
3. fonction récursive ;
4. table de trace (déroulement à la main).

EXERCICE 11 Comptage direct d'instructions

On considère l'algorithme suivant :

```
Algorithme Comptage
Variable n, i, S : Entier
Début
  Lire(n)
  S ← 0
  i ← 1
  Tant que i ≤ n Faire
    S ← S + i
    i ← i + 1
  Fin Tant que
  Écrire(S)
Fin
```

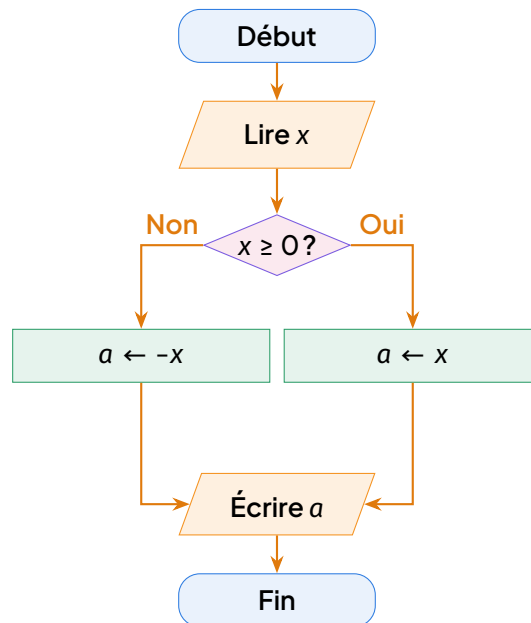
Pour $n = 4$:

1. Combien de fois le test $i \leq n$ est-il évalué ?
2. Combien d'affectations sont exécutées au total (initialisations comprises) ?
3. Combien d'instructions sont exécutées au total, en comptant chaque lecture, écriture, affectation et chaque évaluation du test comme une instruction ?

12.8 Je m'entraîne

EXERCICE 12 Identifier et traduire un algorithme

L'algorithme ci-dessous calcule la valeur absolue d'un nombre réel.



Algorithme : Calcul de la valeur absolue

1. Pour chacun des cinq symboles utilisés, donne son nom et son rôle.
2. Quelle structure de contrôle cet algorithme met-il en œuvre ?
3. Traduis cet algorithme en pseudo-code (écriture algorithmique classique).
4. Déroule-le à la main pour $x = -7$ puis pour $x = 3$.

EXERCICE 13 Table de trace d'une boucle

On considère l'algorithme suivant, qui calcule la somme des n premiers entiers impairs :

```

Algorithme SommeImpairs
Variable n, i, S : Entier
Début
  Lire(n)
  S ← 0
  Pour i de 1 à n Faire
    S ← S + (2 * i - 1)
  Fin Pour
  Écrire(S)
Fin
  
```

1. Construis la table de trace complète pour $n = 5$ (colonnes : numéro d'étape, i , valeur du test ou de la borne, S).
2. Quelle valeur est affichée? Que remarques-tu en comparant ce résultat à n^2 ?
3. Dessine l'algorithme correspondant à cet algorithme, en respectant les symboles normalisés (utilise l'hexagone pour la boucle **Pour** ou sa décomposition normalisée).

EXERCICE 14 Recherche du minimum dans un tableau

On considère l'algorithme suivant, appliqué au tableau $T = [12, 5, 9, 3, 7, 8]$ de 6 éléments :

```

Algorithme Minimum
Variable T : Tableau[1..6] de Entier
Variable i, min : Entier
Début
    min ← T[1]
    Pour i de 2 à 6 Faire
        Si T[i] < min Alors
            min ← T[i]
        Fin Si
    Fin Pour
    Écrire(min)
Fin
  
```

1. Exécute pas à pas cet algorithme en construisant la table de trace (colonnes : i , $T[i]$, résultat du test, $\min$).
2. Quelle valeur est affichée à la fin?
3. Combien de comparaisons $T[i] < \min$ sont effectuées au total? Ce nombre dépend-il du contenu du tableau? Justifie.
4. Combien d'affectations $\min \leftarrow T[i]$ sont exécutées pour ce tableau? Ce nombre dépend-il du contenu du tableau? Justifie.

EXERCICE 15 Fonction récursive : factorielle

On considère la fonction récursive suivante :

```

Fonction fact(n : Entier) : Entier
Début
    Si n = 0 Alors
        Retourner 1
    Sinon
        Retourner n * fact(n - 1)
    Fin Si
  
```

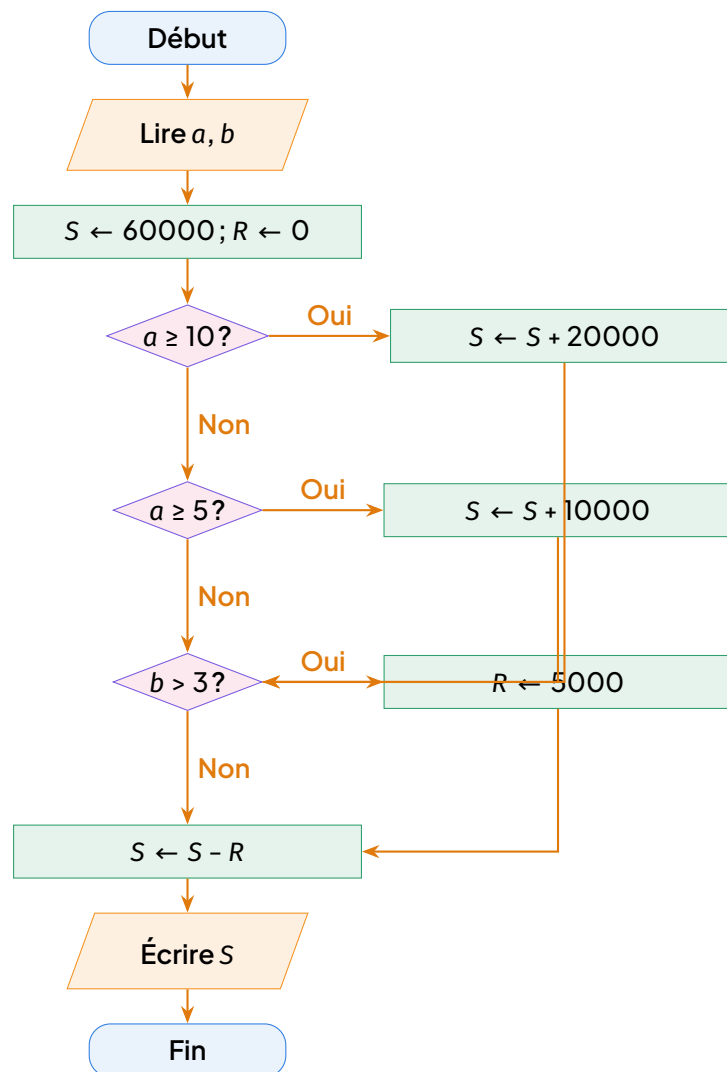
Fin

1. Identifie le cas de base et le cas général (appel récursif) de cette fonction.
2. Exécute à la main `fact(4)` en écrivant la chaîne complète des appels, puis les valeurs retournées à chaque niveau.
3. Que se passe-t-il si l'on appelle `fact(-3)` ? Explique pourquoi, et propose une correction de la fonction pour gérer les entiers négatifs.

12.9 Je me prépare au Probatoire

EXERCICE 16 Salaire net à la menuiserie « Le Bon Bois » de Bafoussam

La menuiserie « Le Bon Bois » calcule chaque mois le salaire net de ses ouvriers selon les règles suivantes : le salaire de base est de 60000 FCFA ; une prime d'ancienneté est accordée : 10000 FCFA si l'ouvrier a au moins 5 ans d'ancienneté, 20000 FCFA s'il a au moins 10 ans ; une retenue de 5000 FCFA est appliquée si l'ouvrier a eu plus de 3 absences dans le mois. Le salaire net est la somme du salaire de base et de la prime, diminuée de la retenue éventuelle. Voici l'algorithme rédigé par le comptable :



Algorithme : Calcul du salaire net (a = ancienneté, b = absences)

1. Recopie et complète le tableau suivant en identifiant la nature de chaque symbole de l'algorithme :

Symbole	Nom du symbole	Rôle
Ovale	...	...
Parallélogramme	...	...
Rectangle	...	...
Losange	...	...

2. Nomme les structures de contrôle présentes dans cet algorithme et cite les symboles concernés.

3. Déroule cet algorithme à la main pour les trois jeux d'essai suivants, en construisant pour chacun la table de trace complète (colonnes : étape, a , b , S , R) :
 - Jeu 1 : $a = 12, b = 2$;
 - Jeu 2 : $a = 7, b = 5$;
 - Jeu 3 : $a = 2, b = 0$.
4. Traduis cet algorithme en pseudo-code (écriture algorithmique classique avec Si ... Alors ... Sinon ... Fin Si).
5. Le comptable affirme : « un ouvrier de 12 ans d'ancienneté reçoit 30000 FCFA de prime ». En t'appuyant sur ton déroulement du jeu 1, explique pourquoi cette affirmation est fausse et précise la prime réellement versée.

EXERCICE 17 Produit par additions successives à la boutique de Mendong

Pour apprendre à son fils Appolinaire le principe de la multiplication, le gérant d'une boutique de Mendong (Yaoundé) lui propose l'algorithme suivant, qui calcule le produit $a \times b$ (avec $a \geq 0$ et $b \geq 0$) par additions successives :

```

Algorithme Produit
Variable a, b, i, P : Entier
Début
  Lire(a, b)
  P ← 0
  i ← 1
  Tant que i ≤ b Faire
    P ← P + a
    i ← i + 1
  Fin Tant que
  Écrire(P)
Fin
  
```

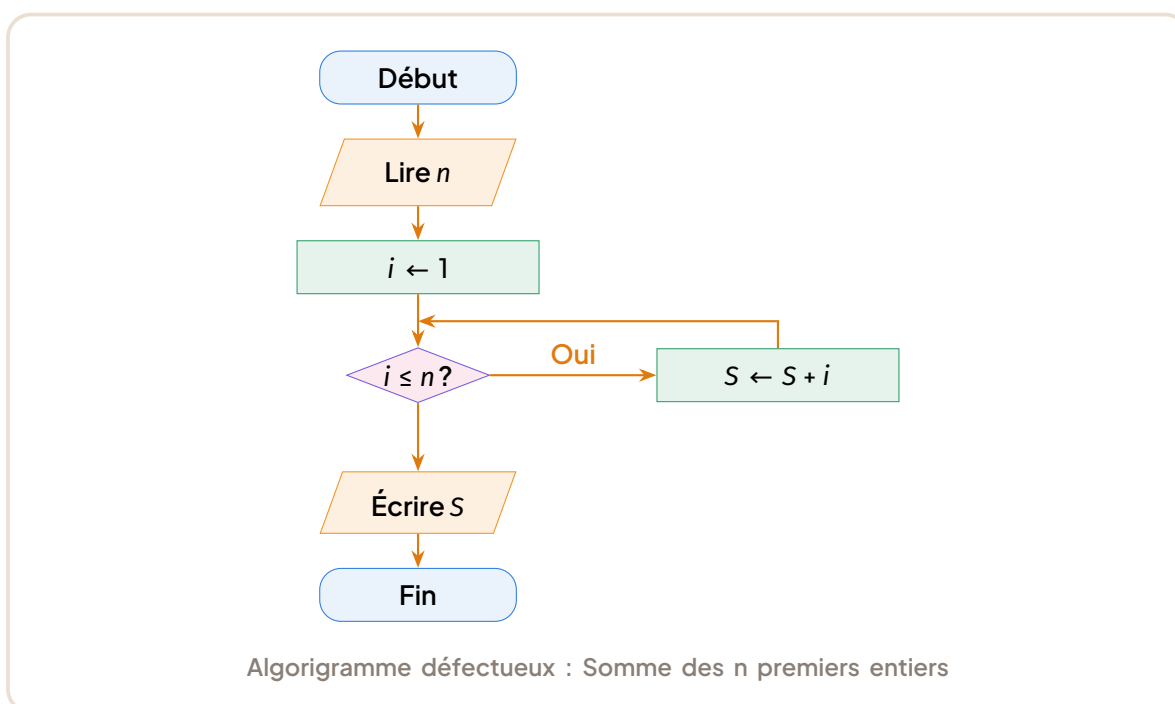
1. Construis la table de trace complète de cet algorithme pour $a = 4$ et $b = 3$, en comptant chaque étape.
2. On convient de compter comme une instruction : chaque lecture, chaque écriture, chaque affectation et chaque évaluation du test $i \leq b$. Montre que le nombre total d'instructions exécutées est $N = 3b + 5$.
3. Calcule N pour $a = 7$ et $b = 100$.
4. Appolinaire remarque qu'on peut choisir de boucler sur la plus petite des deux valeurs. Écris l'algorithme amélioré `ProduitMalin` qui commence par placer dans b le minimum de a et b (et dans a l'autre valeur), puis applique la même méthode.
5. Donne, en fonction de $m = \min(a, b)$, le nombre d'instructions N' de l'algorithme amélioré (on comptera les instructions supplémentaires de l'échange : 1 test et 3 affecta-

tions). Calcule N' pour $a = 7$ et $b = 100$, puis conclus sur le gain obtenu.

6. Dessine l'algorithme de l'algorithme `Produit` (question 1), en respectant les symboles normalisés.

EXERCICE 18 Un algorithme défectueux et la suite de Fibonacci

Le club informatique du Lycée de Nkongsamba organise un concours. Le candidat Rodrigue a rendu l'algorithme ci-dessous, censé lire un entier $n \geq 1$ et afficher la somme $1 + 2 + \dots + n$:



Partie A : chasse aux erreurs.

- En examinant cet algorithme, repère les trois défauts suivants et, pour chacun, cite le symbole concerné et explique le problème :
 - défaut 1 : une variable utilisée sans avoir été initialisée ;
 - défaut 2 : une boucle infinie (une variable de contrôle n'est jamais modifiée) ;
 - défaut 3 : un losange qui ne possède qu'une seule branche étiquetée (le cas « Non » n'est pas explicité correctement).
- Redessine l'algorithme corrigé, en respectant les symboles normalisés et en garantissant l'arrêt de la boucle.
- Traduis l'algorithme corrigé en pseudo-code.
- Déroule ton pseudo-code pour $n = 4$ et donne la valeur affichée.

Partie B : fonction récursive. Le jury propose ensuite la fonction récursive suivante, qui calcule le n -ième terme de la suite de Fibonacci :

```
Fonction fibo( $n$  : Entier) : Entier
Début
  Si  $n \leq 1$  Alors
    Retourner  $n$ 
  Sinon
    Retourner  $\text{fibo}(n - 1) + \text{fibo}(n - 2)$ 
  Fin Si
Fin
```

5. Identifie le (ou les) cas de base et le cas général de cette fonction.
6. Exécute `fibo(5)` à la main en dessinant l'arbre complet des appels (chaque nœud est un appel, relié aux deux appels qu'il déclenche), puis donne la valeur retournée.
7. Combien de fois au total la fonction `fibo` est-elle appelée lors du calcul de `fibo(5)` (l'appel initial compris) ?
8. Explique, en deux ou trois phrases, pourquoi le cas de base est indispensable dans toute fonction récursive, et ce qui se produirait si l'on remplaçait le test $n \leq 1$ par $n = 5$.

13 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : les symboles de l'algorithme

1. Réponse b : le losange. C'est le symbole de la décision : il évalue une condition booléenne et possède deux sorties étiquetées *Oui* et *Non*.
2. Réponse c : le parallélogramme. C'est le symbole des entrées/sorties : *Lire* (saisie) comme *Écrire* (affichage).
3. Réponse a : l'ovale (rectangle arrondi) portant la mention *Début* ou *Fin*. Il marque le point d'entrée et le point de sortie uniques de l'algorithme.
4. Réponse b : le rectangle. C'est le symbole du traitement : calculs, affectations, appels de sous-algorithmes.

Point de vigilance : ne pas confondre parallélogramme (entrées/sorties) et rectangle (traitements) ; une affectation n'est jamais une entrée/sortie.

CORRIGÉ Exercice 2 — Vrai ou faux

1. Faux. Un losange possède une entrée mais deux sorties distinctes, étiquetées *Oui* et *Non* (ou *Vrai/Faux*), correspondant aux deux valeurs possibles de la condition testée.
2. Vrai. Dans une boucle *Tant que*, le test est évalué avant la première itération : si la condition est fausse dès le départ (par exemple $i \leftarrow 5$ puis *Tant que* $i \leq 3$), le corps de la boucle n'est jamais exécuté.
3. Faux. Un algorithme peut contenir autant de boucles que nécessaire, y compris des boucles imbriquées (par exemple le parcours d'un tableau à deux dimensions). Seules comptent la correction et la lisibilité.
4. Vrai. Par définition, la table de trace donne, à chaque étape du déroulement, la valeur de toutes les variables pertinentes, le résultat des tests et les valeurs affichées : c'est précisément ce qui permet de vérifier l'algorithme à la main.
5. Faux. Sans cas de base, la fonction s'appelle elle-même indéfiniment : la pile d'exécution déborde (stack overflow) et le programme plante. L'arrêt n'est jamais « automatique ».

CORRIGÉ Exercice 3 — Définitions

1. Algorithme : représentation graphique normalisée (ISO 5807) d'un algorithme, composée de symboles géométriques (ovale, rectangle, parallélogramme, losange) reliés par des flèches orientées. Exemple : l'algorithme du calcul de la valeur absolue d'un nombre.
2. Organigramme : représentation graphique du flux de traitement de l'information dans un système (administration, entreprise), mettant en évidence les acteurs, les documents et les fichiers. Exemple : l'organigramme du circuit de validation d'un bulletin de notes dans un lycée (proviseur → secrétariat → imprimé).
3. Fonction récursive : fonction qui s'appelle elle-même pour résoudre un problème en le ramenant à un cas plus simple ; elle comporte obligatoirement un cas de base (arrêt) et un cas général (appel récursif). Exemple : $\text{fact}(n) = n * \text{fact}(n-1)$ avec $\text{fact}(0) = 1$.
4. Table de trace : tableau construit en exécutant un algorithme « à la main », donnant à chaque étape la valeur de chaque variable, le résultat de chaque test et chaque affichage. Exemple : la trace de la somme des 4 premiers entiers, qui montre S passant de 0 à 1, 3, 6 puis 10.

CORRIGÉ Exercice 4 — Comptage direct d'instructions

Pour $n = 4$, la boucle s'exécute pour $i = 1, 2, 3, 4$, puis le test devient faux pour $i = 5$.

1. Évaluations du test $i \leq n$: 4 fois vrai (pour $i = 1, 2, 3, 4$) + 1 fois faux (pour $i = 5$) = 5 évaluations, soit $n + 1$.
2. Affectations : 2 initialisations ($S \leftarrow 0, i \leftarrow 1$) + 2 affectations par itération $\times$ 4 itérations = $2 + 8 = 10$.
3. Total d'instructions :

$$N = \underbrace{1}_{\text{Lire}} + \underbrace{10}_{\text{affectations}} + \underbrace{5}_{\text{tests}} + \underbrace{1}_{\text{Écrire}} = 17$$

On vérifie la formule générale $N = 3n + 5$: pour $n = 4$, $3 \times 4 + 5 = 17$. ✓

Point de vigilance : le test qui fait sortir de la boucle (le dernier, faux) est bien exécuté et doit être compté ; c'est l'oubli le plus fréquent.

CORRIGÉ Exercice 5 — Identifier et traduire un algorithme (valeur absolue)

1. Symboles utilisés :

Symbole	Nom	Rôle
Ovale	Début / Fin	Point d'entrée et de sortie de l'algorithme
Parallélogramme	Entrée / Sortie	Lire x (saisie) et Écrire a (affichage)
Losange	Test / Décision	Évalue la condition $x \geq 0$; deux sorties Oui/Non
Rectangle	Traitement	Affectations $a \leftarrow -x$ et $a \leftarrow x$
Flèche	Liaison	Indique l'ordre d'exécution (flux de contrôle)

2. Structure de contrôle : une alternative (Si ... Alors ... Sinon ... Fin Si) : le losange $x \geq 0$? oriente vers l'un des deux rectangles, puis les deux branches convergent vers l'affichage.

3. Traduction en pseudo-code :

```

Algorithme ValeurAbsolue
Variable x, a : Réel
Début
    Lire(x)
    Si x >= 0 Alors
        a <- x
    Sinon
        a <- -x
    Fin Si
    Écrire(a)
Fin
  
```

4. Déroulement à la main :

Étape	x	Test $x \geq 0$	a	Sortie
Jeu 1 : $x = -7$				
1 (Lire)	-7	-	-	-
2 (Test)	-7	F (Non)	-	-
3 (Sinon)	-7	-	7	-
4 (Écrire)	-7	-	7	7
Jeu 2 : $x = 3$				
1 (Lire)	3	-	-	-
2 (Test)	3	V (Oui)	-	-
3 (Alors)	3	-	3	-
4 (Écrire)	3	-	3	3

Dans les deux cas, la valeur affichée est bien la valeur absolue : $|-7| = 7$ et $|3| = 3$.

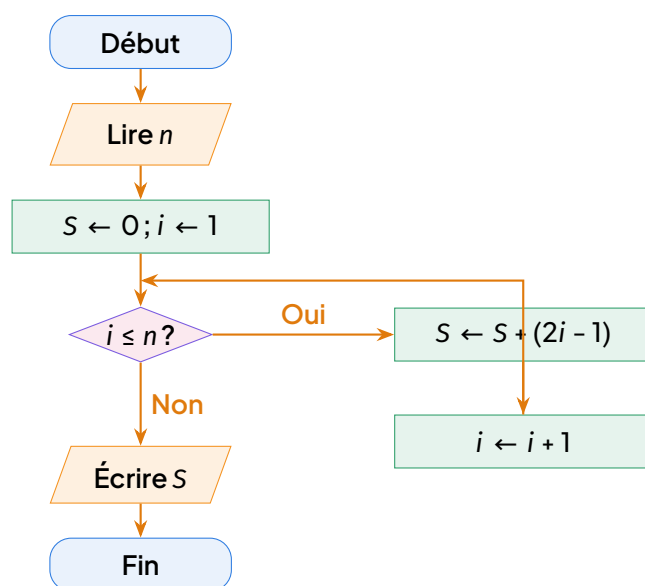
CORRIGÉ Exercice 6 — Table de trace : somme des entiers impairs

1. Table de trace pour $n = 5$ (à chaque tour : $S \leftarrow S + (2i - 1)$) :

Étape	i	Borne $i \leq 5$	S	Calcul effectué
0 (Init)	1	–	0	$S \leftarrow 0$
1	1	V	1	$S \leftarrow 0 + (2 \times 1 - 1) = 1$
2	2	V	4	$S \leftarrow 1 + 3 = 4$
3	3	V	9	$S \leftarrow 4 + 5 = 9$
4	4	V	16	$S \leftarrow 9 + 7 = 16$
5	5	V	25	$S \leftarrow 16 + 9 = 25$
6	6	F	25	Sortie de boucle
7	–	–	25	Écrire : 25

2. Valeur affichée : 25. On remarque que $25 = 5^2 = n^2$: la somme des n premiers entiers impairs vaut n^2 (propriété classique : $1 + 3 + 5 + \dots + (2n - 1) = n^2$).

3. Algorithme (boucle Pour décomposée : initialisation, test, corps, incrémentation) :



Algorithme : somme des n premiers entiers impairs

CORRIGÉ Exercice 7 — Recherche du minimum dans un tableau

1. Table de trace pour $T = [12, 5, 9, 3, 7, 8]$:

i	T[i]	Test $T[i] < \min$	min	Commentaire
1 (Init)	12	–	12	$\min \leftarrow T[1]$
2	5	V ($5 < 12$)	5	Mise à jour de min
3	9	F ($9 > 5$)	5	Aucun changement
4	3	V ($3 < 5$)	3	Mise à jour de min
5	7	F ($7 > 3$)	3	Aucun changement
6	8	F ($8 > 3$)	3	Aucun changement

- Valeur affichée : 3, qui est bien le plus petit élément du tableau.
- Nombre de comparaisons : 5, soit exactement $n - 1$ (les indices 2 à 6). Ce nombre ne dépend pas du contenu du tableau : la boucle `Pour` parcourt systématiquement tous les éléments du second au dernier, quelles que soient les valeurs rencontrées.
- Nombre d'affectations $\min \leftarrow T[i]$: 2 (pour $i = 2$ et $i = 4$). Ce nombre dépend du contenu du tableau : il est égal au nombre de fois où l'on rencontre une valeur strictement inférieure au minimum provisoire. Il vaut au minimum 0 (tableau déjà croissant, ex. $[1, 2, 3]$) et au maximum $n - 1$ (tableau strictement décroissant, ex. $[8, 7, 3, 2, 1, 0]$).

Point de vigilance : bien distinguer ce qui est fixe (nombre de comparaisons, imposé par la structure de la boucle) de ce qui est variable (nombre d'affectations, imposé par les données).

CORRIGÉ Exercice 8 — Fonction récursive : factorielle

- Cas de base :** Si $n = 0$ Alors Retourner 1 — réponse connue sans appel récursif. **Cas général :** Retourner $n * \text{fact}(n - 1)$ — appel récursif avec un argument strictement plus petit, qui rapproche du cas de base.
- Exécution de $\text{fact}(4)$:
Phase descendante (empilement) :

$$\text{fact}(4) = 4 \times \text{fact}(3)$$

$$\text{fact}(3) = 3 \times \text{fact}(2)$$

$$\text{fact}(2) = 2 \times \text{fact}(1)$$

$$\text{fact}(1) = 1 \times \text{fact}(0)$$

$$\text{fact}(0) = 1 \quad (\text{cas de base})$$

Phase montante (dépilement) :

$\text{fact}(0) = 1$
 $\text{fact}(1) = 1 \times 1 = 1$
 $\text{fact}(2) = 2 \times 1 = 2$
 $\text{fact}(3) = 3 \times 2 = 6$
 $\text{fact}(4) = 4 \times 6 = 24$

La valeur retournée est 24 (et l'on vérifie : $4! = 4 \times 3 \times 2 \times 1 = 24$).

3. Appel $\text{fact}(-3)$: le calcul enchaîne $\text{fact}(-3) \rightarrow \text{fact}(-4) \rightarrow \text{fact}(-5) \rightarrow \dots$. L'argument décroît indéfiniment et n'atteint jamais le cas de base $n = 0$: la récursion est infinie et provoque un débordement de pile (stack overflow), donc le plantage du programme. Correction proposée : traiter les entiers négatifs comme un cas d'erreur.

```
Fonction fact(n : Entier) : Entier
Début
  Si n < 0 Alors
    Écrire("Erreur : n doit être positif ou nul")
    Retourner -1
  Sinon
    Si n = 0 Alors
      Retourner 1
    Sinon
      Retourner n * fact(n - 1)
    Fin Si
  Fin Si
Fin
```

Point de vigilance : toute fonction récursive doit garantir que toute valeur d'entrée admissible conduit, par appels successifs, au cas de base.

CORRIGÉ Exercice 9 — Salaire net à la menuiserie « Le Bon Bois » (type Probatoire)

1. Tableau complété :

Symbole	Nom du symbole	Rôle
Ovale	Début / Fin	Marque le point d'entrée et le point de sortie de l'algorithme
Parallélogramme	Entrée / Sortie	Lecture des données (Lire a, b) et affichage du résultat (Écrire S)
Rectangle	Traitement	Affectations et calculs ($S \leftarrow 60000$, $S \leftarrow S + 20000$, etc.)
Losange	Test / Décision	Évalue une condition ($a \geq 10?$, $a \geq 5?$, $b > 3?$) avec deux sorties Oui/Non

2. Structures de contrôle : une séquence (Début → Lire → initialisation) et trois alternatives matérialisées par les trois losanges ($a \geq 10?$, $a \geq 5?$, $b > 3?$), dont les branches « Oui » contiennent un rectangle de traitement et les branches « Non » court-circuitent le traitement. Il n'y a aucune boucle.

3. Déroulements à la main :

Jeu 1 : $a = 12$, $b = 2$.

Étape	a	b	S	R	Action / Test
1	12	2	-	-	Lire
2	12	2	60000	0	Initialisation
3	12	2	60000	0	$12 \geq 10?$ Oui
4	12	2	80000	0	$S \leftarrow 60000 + 20000$
5	12	2	80000	0	$2 > 3?$ Non
6	12	2	80000	0	$S \leftarrow 80000 - 0$
7	12	2	80000	0	Écrire : 80000

Jeu 2 : $a = 7$, $b = 5$.

Étape	a	b	S	R	Action / Test
1	7	5	-	-	Lire
2	7	5	60000	0	Initialisation
3	7	5	60000	0	$7 \geq 10$? Non
4	7	5	60000	0	$7 \geq 5$? Oui
5	7	5	70000	0	$S \leftarrow 60000 + 10000$
6	7	5	70000	0	$5 > 3$? Oui
7	7	5	70000	5000	$R \leftarrow 5000$
8	7	5	65000	5000	$S \leftarrow 70000 - 5000$
9	7	5	65000	5000	Écrire : 65000

Jeu 3 : $a = 2, b = 0$.

Étape	a	b	S	R	Action / Test
1	2	0	-	-	Lire
2	2	0	60000	0	Initialisation
3	2	0	60000	0	$2 \geq 10$? Non
4	2	0	60000	0	$2 \geq 5$? Non
5	2	0	60000	0	$0 > 3$? Non
6	2	0	60000	0	$S \leftarrow 60000 - 0$
7	2	0	60000	0	Écrire : 60000

4. Traduction en pseudo-code :

```

Algorithme SalaireNet
Variable a, b, S, R : Entier
Début
    Lire(a, b)
    S ← - 60000
    R ← - 0
    Si a >= 10 Alors
        S ← - S + 20000
    Sinon
        Si a >= 5 Alors
            S ← - S + 10000
        Fin Si
    Fin Si
    Si b > 3 Alors
        R ← - 5000
    Fin Si
    S ← - S - R
    Écrire(S)
Fin
  
```

5. Analyse de l'affirmation du comptable : elle est fausse. L'algorithme est construit avec des tests exclusifs : si $a \geq 10$ (branche « Oui » du premier losange), le traitement

$S \leftarrow S + 20000$ est exécuté, puis le flux rejoint directement le test sur les absences — le second losange $a \geq 5$? n'est jamais traversé. Le déroulement du jeu 1 le confirme : S passe de 60000 à 80000 FCFA, soit une prime de 20000 FCFA seulement, et non $10000 + 20000 = 30000$ FCFA. La prime versée est celle du palier le plus élevé atteint, pas la somme des paliers.

Barème indicatif (sur 10) : tableau des symboles 2 pts ; structures de contrôle 1 pt ; trois tables de trace 3 pts (1 pt chacune) ; pseudo-code 2 pts ; analyse de l'affirmation 2 pts. Points de vigilance : exiger les étiquettes Oui/Non dans la lecture de l'algorithme ; pour la question 5, la justification doit s'appuyer explicitement sur le chemin suivi dans l'algorithme (la branche « Oui » de $a \geq 10$ bypass le test $a \geq 5$), pas seulement sur le résultat chiffré.

CORRIGÉ Exercice 10 — Produit par additions successives (type Probatoire)

1. Table de trace pour $a = 4, b = 3$:

Étape	i	Test $i \leq 3$	P	Nb instr. cumulé	Commentaire
1	-	-	-	1	Lire(a, b)
2	-	-	0	2	$P \leftarrow 0$
3	1	-	0	3	$i \leftarrow 1$
4	1	V	0	4	Test 1
5	1	-	4	5	$P \leftarrow 0 + 4$
6	2	-	4	6	$i \leftarrow 2$
7	2	V	4	7	Test 2
8	2	-	8	8	$P \leftarrow 4 + 4$
9	3	-	8	9	$i \leftarrow 3$
10	3	V	8	10	Test 3
11	3	-	12	11	$P \leftarrow 8 + 4$
12	4	-	12	12	$i \leftarrow 4$
13	4	F	12	13	Test 4 (sortie)
14	-	-	12	14	Écrire : 12

La valeur affichée est $12 = 4 \times 3$. ✓

2. Dénombrement général (pour b quelconque, la boucle tourne b fois) :

Avant la boucle : Lire + $P \leftarrow 0 + i \leftarrow 1 = 3$ instructions

Corps de boucle : $b \times \left(\underbrace{1}_{\text{test V}} + \underbrace{2}_{P \leftarrow P+a, i \leftarrow i+1} \right) = 3b$

Sortie de boucle : 1 test faux

Après la boucle : Écrire = 1 instruction

$$N = 3 + 3b + 1 + 1 = 3b + 5$$

Vérification avec la question 1 : $b = 3 \Rightarrow N = 3 \times 3 + 5 = 14$, conforme au comptage de la table. ✓

3. Pour $a = 7, b = 100$: $N = 3 \times 100 + 5 = 305$ instructions.

4. Algorithme amélioré :

```

Algorithme ProduitMalin
Variable a, b, i, P, temp : Entier
Début
  Lire(a, b)
  Si a < b Alors
    temp ← a
    a ← b
    b ← temp
  Fin Si
  P ← 0
  i ← 1
  Tant que i ≤ b Faire
    P ← P + a
    i ← i + 1
  Fin Tant que
  Écrire(P)
Fin
  
```

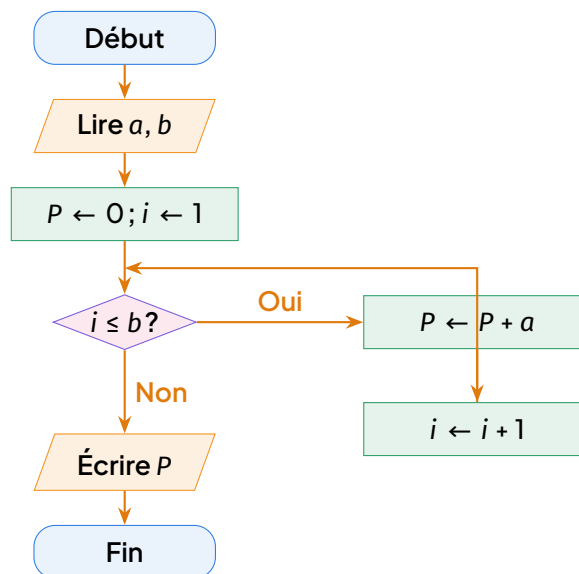
Après l'échange éventuel, b contient bien $\min(a,b)$ et a l'autre valeur ; le produit $a \times b$ est inchangé (commutativité).

5. Nombre d'instructions de l'algorithme amélioré : on ajoute 1 test et 3 affectations (l'échange) au décompte précédent, et la boucle tourne désormais $m = \min(a,b)$ fois :

$$N' = 3m + 5 + 4 = 3m + 9$$

Pour $a = 7, b = 100$: $m = 7$, donc $N' = 3 \times 7 + 9 = 30$ instructions, contre 305 pour la version initiale. Gain : $305 - 30 = 275$ instructions économisées, soit une exécution environ 10 fois plus courte. Conclusion : en algorithmique, un petit raisonnement préalable (boucler sur le plus petit opérande) peut réduire considérablement le coût d'exécution.

6. Algorithme de Produit :

Algorithme : produit $a \times b$ par additions successives

Barème indicatif (sur 12) : table de trace 3 pts ; démonstration $N = 3b + 5$ 2 pts ; calcul de N 1 pt ; algorithme `ProduitMalin` 2 pts ; N' et conclusion 2 pts ; algorithme 2 pts.

Points de vigilance : pour la démonstration, justifier séparément les 3 instructions d'initialisation, le facteur b du corps de boucle, le test de sortie et l'écriture finale ; pour l'échange, exiger la variable intermédiaire `temp` (l'échange $a \leftarrow b$; $b \leftarrow a$ sans variable temporaire est une erreur classique qui écrase a).

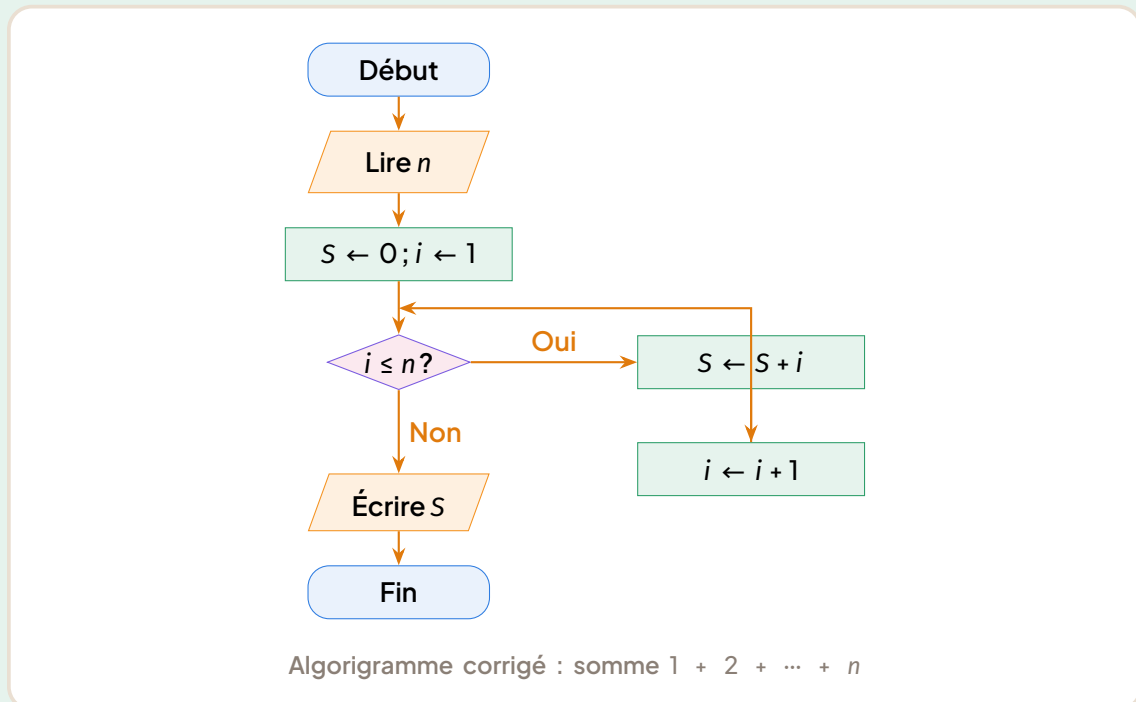
CORRIGÉ**Exercice 11 — Algorithme defectueux et suite de Fibonacci (type Probatoire)**

Partie A : chasse aux erreurs.

1. Les trois défauts :

- Défaut 1 — variable non initialisée : le rectangle $S \leftarrow S + i$ utilise S , qui n'a jamais reçu de valeur initiale (ni lecture, ni affectation). Le résultat est imprévisible : il manque $S \leftarrow 0$ avant la boucle.
- Défaut 2 — boucle infinie : la variable de contrôle i n'est jamais modifiée dans le corps de la boucle (il manque $i \leftarrow i + 1$). Le test $i \leq n$? reste vrai éternellement : l'algorithme ne s'arrête jamais.
- Défaut 3 — losange incomplet : le losange $i \leq n$? ne possède qu'une branche étiquetée (« Oui ») ; la sortie « Non » vers `Écrire S` n'est pas étiquetée. Or tout losange doit avoir deux sorties explicitement étiquetées Oui/Non.

2. Algorithme corrigé :



Corrections apportées : ajout de $S \leftarrow 0$ (défaut 1), ajout de $i \leftarrow i + 1$ dans le corps de boucle (défaut 2), étiquetage des deux sorties du losange (défaut 3).

3. Pseudo-code :

```

Algorithme SommeN
Variable n, i, S : Entier
Début
  Lire(n)
  S ← 0
  i ← 1
  Tant que i ≤ n Faire
    S ← S + i
    i ← i + 1
  Fin Tant que
  Écrire(S)
Fin
  
```

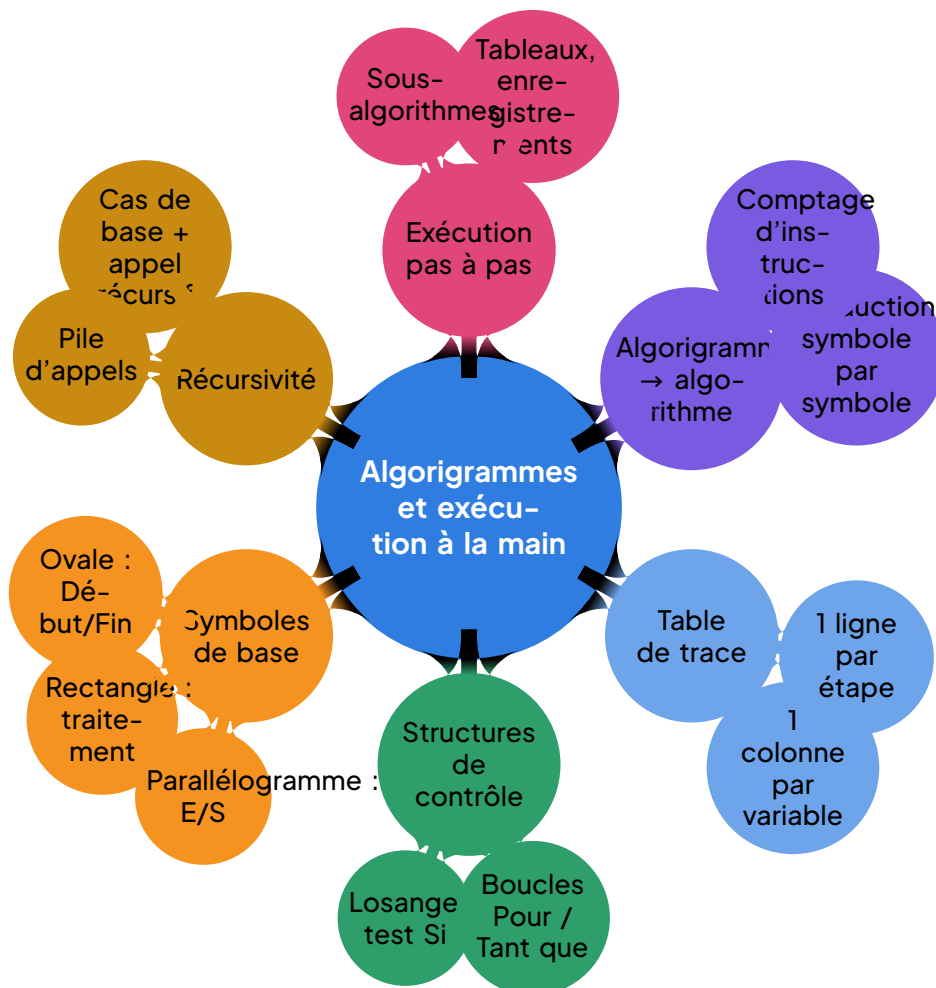
4. Déroulement pour $n = 4$:

des appels sur des valeurs qui contournent 5 ou deviennent négatives), conduisant à une récursion sans fin ou à des résultats aberrants. Le cas de base doit donc être à la fois correct et atteignable depuis toute entrée admissible.

Barème indicatif (sur 14) : identification des 3 défauts 3 pts ; algorithme corrigé 2 pts ; pseudo-code 1 pt ; trace pour $n = 4$ 1 pt ; cas de base / cas général 1 pt ; arbre des appels 3 pts ; nombre d'appels 1 pt ; explication du cas de base 2 pts.

Points de vigilance : pour le défaut 2, la justification attendue est « la variable de contrôle n'est pas modifiée dans le corps de la boucle, donc la condition reste vraie indéfiniment » ; pour l'arbre de `fibonacci(5)`, exiger que tous les nœuds soient dessinés (15 nœuds) et que les feuilles soient uniquement des `f(0)` et `f(1)` ; pour la dernière question, citer explicitement le débordement de pile (stack overflow).

14 FICHE BILAN



Carte mentale de la leçon : algorithmes et exécution des algorithmes à la main.

FICHE BILAN L'essentiel à connaître pour l'examen

Définitions clés

- **Algorithme** : représentation graphique d'un algorithme à l'aide de symboles normalisés reliés par des flèches indiquant l'ordre d'exécution.
- **Organigramme** : schéma représentant l'enchaînement des tâches ou des étapes d'un processus (plus général qu'un algorithme, souvent utilisé en gestion).
- **Table de trace** : tableau donnant, à chaque étape de l'exécution, la valeur de chaque variable ; c'est l'outil du déroulement à la main.

- **Fonction récursive** : fonction qui s'appelle elle-même, avec un cas de base (condition d'arrêt) et un appel récursif sur un problème plus petit.

Symboles normalisés à connaître par cœur

Symbole	Rôle
Ovale (terminateur)	Début / Fin de l'algorithme
Parallélogramme	Entrée (Lire) / Sortie (Écrire)
Rectangle	Traitement (affectation, calcul)
Losange	Test / décision (deux flèches : Oui / Non)
Flèche	Sens de l'enchaînement des étapes
Cercle	Connecteur (liaison entre parties du schéma)

Structures de contrôle en algorithme

- **Séquence** : rectangles empilés, exécutés de haut en bas.
- **Alternative** (Si ... Alors ... Sinon) : un losange, deux branches qui se rejoignent.
- **Répétitive Tant que** : losange dont la branche « Oui » remonte vers l'amont (le test est fait avant le corps de boucle).
- **Répétitive Pour** : initialisation du compteur, test de fin, corps, incrémentation, retour au test.

Méthodes express

1. Dérouler un algorithme : partir de l'ovale « Début », suivre les flèches, noter chaque affectation dans la table de trace, s'arrêter à « Fin ».
2. Transformer en algorithme : un symbole = une instruction ; losange → Si/Tant que ; remontée de flèche → boucle.
3. Compter les instructions : compter chaque affectation, lecture, écriture et test ; dans une boucle, multiplier le corps par le nombre de tours.
4. Exécuter une fonction récursive : empiler les appels jusqu'au cas de base, puis dépiler en calculant les résultats en sens inverse.

À RETENIR

Checklist avant le Bac

- ☐ Je sais dessiner et nommer les 6 symboles de base d'un algorithme.
- ☐ Je distingue algorithme (solution algorithmique) et organigramme (processus général).
- ☐ Je reconnais dans un algorithme une séquence, une alternative et une boucle.
- ☐ Je sais qu'un test se dessine dans un losange avec exactement deux flèches sor-

tantes.

- ☐ Je sais construire une table de trace : une colonne par variable, une ligne par étape.
- ☐ Je sais dérouler un algorithme contenant une boucle sans me tromper dans le nombre de tours.
- ☐ Je sais traduire un algorithme simple en algorithme en pseudo-code.
- ☐ Je sais compter le nombre d'instructions exécutées, y compris dans une boucle.
- ☐ Je sais exécuter à la main un algorithme utilisant un tableau ou un sous-algorithme.
- ☐ Je connais les deux ingrédients obligatoires d'une fonction récursive : cas de base et appel récursif.
- ☐ Je sais dérouler la pile d'appels d'une fonction récursive simple (factorielle, somme).
- ☐ Je vérifie toujours que ma récursion possède une condition d'arrêt (sinon : boucle infinie!).



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+